

EtherCAT[®] Protocol Enhancements

Document: ETG.1020 S (R) V1.5.0

Nomenclature:

ETG-Number	ETG.1020
Type	S (Standard)
State	R (Release)

Created by:	ETG
Contact:	info@ethercat.org
Filename:	ETG1020_v1i5i0_S_R_ProtocolEnhancements.docx
Date:	02.06.2025

LEGAL NOTICE

Trademarks and Patents

EtherCAT® is a registered trademark and patented technology, licensed by Beckhoff Automation GmbH, Germany. Other designations used in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owners.

Disclaimer

The documentation has been prepared with care. The technology described is, however, constantly under development. For that reason the documentation is not in every case checked for consistency with performance data, standards or other characteristics. In the event that it contains technical or editorial errors, we retain the right to make alterations at any time and without warning. No claims for the modification of products that have already been supplied may be made on the basis of the data, diagrams and descriptions in this documentation.

Copyright

© EtherCAT Technology Group

The reproduction, distribution and utilization of this document as well as the communication of its contents to others without express authorization is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design.

DOCUMENT HISTORY

Version	Comment
1.0.0.	First Release of PDF version
1.1.0	Changes according to ETG1020_CommentResolution_V1i0i0R.zip
1.2.0	Changes according to ETG1020_CommentFormV1i1i0D_2015-12-01.doc
1.3.0	Changes according to ETG1020_CommentResolutionV1i2i0R_2019-11-20.doc
1.4.0	Changes according to ETG1020_CommentResolutionV1i3i0R_2024-03-18.doc
1.5.0	Changes according to ETG.1020_CommentList_V1i4i0R.pdf and ETG.1020_CommentList_V1i4i1D.pdf

CONTENTS

1	Scope	1
2	References	2
3	Terms, Definitions and Word Usage	3
	3.1 Terms and Definitions	3
	3.2 Word usage: shall, should, may, can	3
4	Description of AL Status Codes	4
5	EEPROM	8
	5.1 EEPROM Access	8
	5.2 EEPROM Content / SubDevice Information Interface (SII)	8
6	Warning indication	9
	6.1 General	9
	6.2 Description of Warning Code	9
7	Dynamic Process Data Channel	11
	7.1 Scope	11
	7.2 DPC specific terms and definitions	11
	7.3 Functional principle	11
	7.4 State machine	12
	7.4.1 General	12
	7.4.2 DPC errors	14
	7.4.3 DPC primitives	14
	7.4.4 DPC variables	16
	7.4.5 DPC macros	16
	7.4.6 DPC state table	17
8	Mailbox	19
	8.1 Mailbox InitCmds	19
	8.2 Mailbox Header	19
9	ADS over EtherCAT (AoE)	21
	9.1 AoE Mailbox Frame	21
	9.2 AoE Commands	23
	9.2.1 AoE Read	23
	9.2.2 AoE Write	24
	9.2.3 AoE WriteControl	24
	9.2.4 AoE ReadWrite	25
	9.3 AoE Fragmentation	25
	9.4 AoE Initialization	26
	9.5 Mailbox Services with AoE	26
	9.5.1 CoE Services	27
	9.5.2 FoE Services	27
	9.5.3 SoE Services	28
	9.5.4 VoE Services	28
10	CoE Abort Codes (extension)	29
11	Command Object	30
12	CoE Communication Area	31
	12.1 Object 0x100B: Bootloader Version	31
	12.2 Object 0x1010: Store parameters	31
	12.3 Object 0x1011: Restore default parameters	32
	12.4 Object 0x1020: Device Statistics	33
	12.5 Object 0x10E2: Manufacturer-specific Identification Code	34

13	Change of Mapping Objects.....	36
14	SDO Services	37
	14.1 SDO access single entry with flexible length	37
	14.2 SDO Complete Access	37
	14.3 SDO Information services	38
	14.4 SDO Upload with max response size	39
15	ENUM Definition	41
	15.1 ENUM rules.....	41
	15.2 ENUM used as an entry of an ARRAY data type	41
16	Diagnosis Handling	42
	16.1 General	42
	16.2 Object 0x10F3: Diagnosis History Object.....	42
	16.3 Diagnosis Message	46
	16.4 Diagnosis History Operation Modes	48
	16.4.1 Overwrite Mode	48
	16.4.2 Acknowledge Mode	49
	16.5 Object 0x10FA: Diagnosis History Management object	50
	16.6 Object 0x10F8: Timestamp Object	51
	16.7 Object 0x10F9: Time Distribution Object	51
17	PDO Parameter Description.....	53
	17.1 General	53
	17.2 RxPDO Parameter	53
	17.3 TxPDO Parameter	54
	17.4 PDO Mapping entries larger than 31 Byte	54
	17.5 PDO Mapping attribute of object definitions	55
18	FoE Extension	57
	18.1 FoE read and write clarification	57
	18.2 FoE read sequence with busy	57
	18.3 FoE error Codes	58
19	EoE Extension.....	60
	19.1 General	60
	19.2 Get IP Parameter	60
	19.3 Get Address Filter	62
20	VoE Coding	64
	20.1 PDU structure	64
21	Explicit Device Identification.....	65
	21.1 Intention	65
	21.2 Terms and Definitions.....	65
	21.3 Device Identification Value	65
	21.4 SubDevice implementation	66
	21.4.1 Requesting ID – Complex SubDevice with application microcontroller and ID-Selector	66
	21.4.2 Direct ID – Simple SubDevice w/o microcontroller and with ID-Selector	68
	21.4.3 Set Device Identification by Configuration Tool	69
	21.5 MainDevice behaviour	69
	21.5.1 Configuration Tool	69
	21.5.2 MainDevice	69
	21.6 Object 0x10E0: Device Identification Reload Object	69
22	Error Handling	71

22.1	Error Reasons.....	71
22.2	Error Detection.....	71
22.2.1	Mechanism	71
22.2.2	Object 0x10F1: Error Settings.....	74
22.2.3	Error Detection in the SubDevice	75
22.2.4	Error Detection in the MainDevice	76
22.3	Error Reaction.....	76
22.3.1	Error Reactions in the SubDevice	76
22.3.2	Error Reactions in the MainDevice	79
22.4	Restart Behaviour	80
22.4.1	Restart Behaviour in the SubDevice	80
22.4.2	Restart Behaviour in the MainDevice	82
23	Synchronization.....	84
23.1	Synchronization in the SubDevice	85
23.1.1	Synchronization Modes in the SubDevice.....	85
23.1.2	Objects 0x1C32 and 0x1C33: SyncManager Parameter objects.....	105
23.1.3	Diagnosis of the Synchronization Mode	109
23.2	Synchronization in the MainDevice	111
23.2.1	Cyclic Mode	111
23.2.2	DC Mode, MainDevice follows Reference Clock.....	111
23.2.3	Sync-Unit: DcOffset correction	113
24	External Synchronization.....	114
24.1	External Synchronization of EtherCAT Networks, Reference Clock follows MainDevice Clock	114
24.1.1	Synchronization with an external clock using IEEE 1588	115
24.1.2	Synchronization by a Bridge device	116
24.2	External Synchronization of an EtherCAT SubDevice	117
24.2.1	Object 0x10F4: External Synchronization Status.....	117
24.2.2	Object 0x10F5: External Synchronization Settings	117
24.2.3	TxPDO Mapping	118
24.2.4	MainDevice's Control Loop	119
25	ESM behaviour	121
25.1.1	Start of process data exchange	121
25.1.2	Details on SubDevice ESM handler in MainDevice	121
26	Base Data Types.....	122

TABLES

Table 1: Description of AL Status Codes usage.....	4
Table 2: Description of Warning Codes.....	9
Table 3: Primitives issued by DPCSM to DL	15
Table 4: Primitives issued by DL to DPCSM	15
Table 5: Primitives issued by DPCSM to ESM	15
Table 6: Primitives issued by ESM to DPCSM	15
Table 7: Primitives issued by DPCSM to application	15
Table 8: Primitives issued by application to DPCSM	15
Table 9: DPC variables.....	16
Table 10: DPC macros	16
Table 11: DPCSM state table	17
Table 12: Mailbox Header definition (referenced from ETG.1000.6)	19
Table 13: Mailbox frame with AoE	21
Table 14: AoE Header	22
Table 15: AoE Commands	22
Table 16: AoE Result codes (low word)	23
Table 17: AoE Read Request Header	23
Table 18: AoE Read Response Header	24
Table 19: AoE Write Request Header	24
Table 20: AoE Write Response Header	24
Table 21: AoE WriteControl Request Header	24
Table 22: AoE WriteControl Response Header.....	24
Table 23: AoE ReadWrite Request Header	25
Table 24: AoE ReadWrite Response Header.....	25
Table 25: AoE Header for AoE Fragmentation command.....	25
Table 26: AoE Fragmentation Example	26
Table 27: AoE Header for the AoE Initialization Write Command	26
Table 28: AoE Write Request Header for the AoE Initialization Write Command	26
Table 29: Index Group of the CoE Services	27
Table 30: AoE Mapping of FoE Services	27
Table 31: AoE Mapping of SoE Services	28
Table 32: AoE Mapping of VoE Services	28
Table 33: CoE Abort Codes (extension).....	29
Table 34: Command object structure	30
Table 35: Bootloader Version object structure	31
Table 36: 0x1010 Store parameters object	31
Table 37: Entries of 0x1010 Store parameters.....	32
Table 38: 0x1011 Restore default parameters object	33
Table 39: Entries of 0x1011 Restore default parameters.....	33
Table 40: 0x1020 Device Statistics	34
Table 41: Entries of object 0x1020 Device Statistics	34
Table 42: 0x10E2 Manufacturer-specific Identification Code	34
Table 43: Entries of object 0x10E2 Manufacturer-specific Identification Code	35
Table 44: SDO Complete Access Example with SubIndex 1	38
Table 45: SDO Complete Access Example with SubIndex 0	38
Table 46: SDO Upload Expedited Request (extension)	40
Table 47: ENUM Definition – Textual description of allowed values	41
Table 48: 0x10F3 Diagnosis History Object	42
Table 49: Entries of 0x10F3 Diagnosis History Object.....	43
Table 50: Diagnosis Message	46
Table 51: 0x10FA Diagnosis History Management object	50
Table 52: Entries of 0x10FA Diagnosis History Management Object	51
Table 53: 0x10F8 Timestamp Object	51
Table 54: 0x10F9 Time Distribution Object	52
Table 55: Entries of 0x10F9 Time Distribution Object.....	52
Table 56: RxPDO Parameter Objects	53
Table 57: RxPDO Parameter.....	53
Table 58: TxPDO Parameter Objects.....	54
Table 59: TxPDO Parameter	54
Table 60: Meaning of attribute Rx/TxPDO mapping.....	56

Table 61: Error codes for FoE	59
Table 62: EoE Get IP Parameter service request	60
Table 63: EoE Get IP Parameter service response	61
Table 64: EoE Get Address Filter service request	62
Table 65: EoE Get Address Filter service response	63
Table 66: VoE elements	64
Table 67: 0x10E0 Device Identification Reload Object	70
Table 68: Entries of 0x10E0 Device Identification Reload Object	70
Table 69: AL Status Codes used for Synchronization	72
Table 70: 0x10F1 Error Settings Object	74
Table 71: 0x10F1 Error Settings	74
Table 72: Example for weighted Error Counter	75
Table 73: Error Detection Mechanisms by SubDevice	75
Table 74: Error Detection of MainDevice Errors	75
Table 75: Error Detection Mechanisms by MainDevice	76
Table 76: Error Detection of SubDevice Errors	76
Table 77: Local Reactions SubDevice Errors	76
Table 78: Bus Reactions SubDevice Errors	77
Table 79: Relation Error Detection and Error Reaction in the SubDevice	77
Table 80: Local Reactions MainDevice Errors	79
Table 81: Bus Reactions MainDevice Errors	79
Table 82: Relation Error Detection and Error Reaction in the MainDevice	80
Table 83: SubDevice Restart Behaviour	80
Table 84: MainDevice Restart Behaviour	82
Table 85: SyncManger Assignment with Mailbox	84
Table 86: SyncManger Assignment without Mailbox	84
Table 87: Synchronization Mode Identification	86
Table 88: 0x1C32 Copy and Prepare Outputs	87
Table 89: 0x1C33 Get and Copy Inputs	87
Table 90: 0x1C32 Free Run	89
Table 91: 0x1C33 Free Run	90
Table 92: 0x1C32 Synchronous with SM 2/3 Event	91
Table 93: 0x1C33 Synchronous with SM 2/3 Event	92
Table 94: 0x1C32 Synchronous with SM 2/3 Event, Shift Input Latch	93
Table 95: 0x1C33 Synchronous with SM 2/3 Event, Shift Input Latch	94
Table 96: 0x1C32 DC Mode	95
Table 97: 0x1C33 DC Mode	96
Table 98: 0x1C32 DC Mode, Shift of Outputs/ Inputs	97
Table 99: 0x1C33 DC Mode, Shift of Outputs/ Inputs	98
Table 100: 0x1C32 DC Mode, Shift of Input Latch with Sync1	99
Table 101: 0x1C33 DC Mode, Shift of Input Latch with Sync1	100
Table 102: 0x1C32 DC Mode (Sync1)	101
Table 103: 0x1C33 DC Mode (Sync1)	102
Table 104: 0x1C32 DC Mode, subordinated μ C cycles, Shift Outputs/ Inputs	103
Table 105: 0x1C33 DC Mode, subordinated μ C cycles, Shift Outputs/ Inputs	104
Table 106: 0x1C32 / 0x1C33 SyncManager Parameter objects	105
Table 107: Output SyncManager Parameter (0x1C32)	105
Table 108: Input SyncManager Parameter (0x1C33)	108
Table 109: Error Behaviour of the SubDevice	110
Table 110: MainDevice Settings for use of DCs with SubDevice related settings	113
Table 111: 0x10F4 External Synchronization Status	117
Table 112: Entries of 0x10F4 External synchronization Status	117
Table 113: 0x10F5 External Synchronization Settings	118
Table 114: Entries of 0x10F5 External Synchronization Settings	118
Table 115: TxPDO Mapping for SYNC follower, 64 bit Time Stamp	118
Table 116: TxPDO Mapping for SYNC follower, 32 bit Time Stamp	119
Table 117: TxPDO Mapping for SYNC follower, Status Only	119
Table 118: TxPDO Mapping for SYNC Reference	119
Table 119: Base Data Types	122
Table 120: Base Data Types with variable length	124
Table 121: Pre-defined records	125

FIGURES

Figure 1: Dynamic Process Data Channel within the SubDevice reference model	12
Figure 2: DPC State Machine	13
Figure 3: DPC primitives.....	14
Figure 4: MainDevice-to-SubDevice mailbox communication	19
Figure 5: SubDevice-to-SubDevice mailbox communication	20
Figure 6: Acknowledge Mode: Example 1.....	49
Figure 7: Acknowledge Mode: Example 2.....	49
Figure 8: Clearing Message History	50
Figure 9 – FoE read sequence with success	57
Figure 10 – FoE write sequence with success	57
Figure 11: FoE read sequence with busy	58
Figure 12: VoE general structure.....	64
Figure 13: Complex SubDevice with application microcontroller	66
Figure 14: Timing Example: Request Device ID	67
Figure 15: Timing Example: Error Indication while Device ID loaded	67
Figure 16: Timing Example: Error Acknowledge with Device ID Request	67
Figure 17: Simple SubDevice w/o microcontroller.....	68
Figure 18: Set Device Identification by Configuration Tool	69
Figure 19: Restart Behaviour of the SubDevice	81
Figure 20: Restart Behaviour of a Sync Unit in the MainDevice	82
Figure 21: Differences between synchronization in the SubDevices (simplified).....	85
Figure 22: Local Cycle with Free Run synchronization, with local timer	88
Figure 23: Local Cycle with Free Run synchronization, without local timer	88
Figure 24: Local Cycle synchronous with SM2 Event	90
Figure 25: Local Cycle synchronous with SM2/3 Event, Shift Input Latch	92
Figure 26: Local Cycle synchronous to Sync0 Event with support of SM2 event	94
Figure 27: Local Cycle synchronous to Sync0 Event without support of SM2 event	95
Figure 28: Local Cycle synchronous to Sync0 Event, Shift of Outputs/ Inputs with support of SM2 event	96
Figure 29: Local Cycle synchronous to Sync0 Event, Shift of Outputs/ Inputs without support of SM2 event	97
Figure 30: Local Cycle synchronous to Sync0 Event, Shift of Input Latch with Sync1 with support of SM2 event.....	99
Figure 31: Local Cycle synchronous to Sync0 Event, Shift of Input Latch with Sync1 without support of SM2 event.....	99
Figure 32: Local Cycle synchronous to Sync1 Event.....	101
Figure 33: DC, subordinated μ C cycles, Outputs/ Inputs delayed	103
Figure 34: MainDevice synchronized to DC Base.....	111
Figure 35: User Shift.....	112
Figure 36: MainDevice Settings for use of DCs	112
Figure 37: MainDevice Settings.....	113
Figure 38: Synchronization of several EtherCAT Segments	114
Figure 39: External IEEE 1588 Clock	115
Figure 40: Control Algorithm for Time difference loaded in an EtherCAT SubDevice	116
Figure 41: Synchronization of EtherCAT Segments using a Bridge device	116

ABBREVIATIONS

μC	Microcontroller
C	Conditional
CMD	Command
CoE	CANopen over EtherCAT
DC	Distributed Clock
DPC	Dynamic Process Data Channel
DPRAM	Dual-Ported RAM
ENI	EtherCAT Network Information (EtherCAT XML MainDevice Configuration)
EoE	Ethernet over EtherCAT
ESC	EtherCAT SubDevice Controller
ESI	EtherCAT SubDevice Information (EtherCAT Devices Description)
ESM	EtherCAT State Machine
ETG	EtherCAT Technology Group
FMMU	Fieldbus Memory Management Unit
FoE	File Access over EtherCAT
FPMR	Configured Address Physical Read Multiple Write
FPRD	Configured Address Physical Read
FPRW	Configured Address Physical ReadWrite
FPWR	Configured Address Physical Write
I/O	Input/Output
IDN	Identification Number (Servo Profile Identifier)
IEC	International Electrotechnical Commission
INT	Integer
IRQ	Interrupt Request
LRD	Logical Read
LRW	Logical ReadWrite
LSB	Least Significant Bit
LWR	Logical Write
M	Mandatory
MAC	Media Access Controller
MI	(PHY) Management Interface
MII	Media Independent Interface
MSB	Most Significant Bit
NIC	Network Interface Card
NOP	No Operation
ns	nanoseconds (10 ⁻⁹ seconds)
O	Optional
OD	Object Dictionary
OS	Oversampling
PDO	Process Data Object
PREOP	Pre-Operational
RO	Read Only
RD	Read
SDO	Service Data Object
SI	SubIndex
SM	SyncManager
SoE	Servo Profile over EtherCAT
SOF	Start of Frame
SPI	Serial Peripheral Interface
SU	Sync Unit
WD	Watchdog
WKC	Working Counter
WO	Write Only
WR	Write
XML	eXtensible Markup Language

1 Scope

This specification contains enhancements of the EtherCAT Protocol. These enhancements are part of the EtherCAT specification and may become part of the ETG.1000 series in the future.

2 References

The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

ETG Standards

- [1] ETG.1000.2: Physical Layer service definition and protocol specification
- [2] ETG.1000.3: Data Link Layer service definition
- [3] ETG.1000.4: Data Link Layer protocol specification
- [4] ETG.1000.5: Application Layer service definition
- [5] ETG.1000.6: Application Layer protocol specification
- [6] ETG.2000: EtherCAT SubDevice Information specification
- [7] ETG.2010: EtherCAT SubDevice Information Interface (SII) specification
- [8] ETG.5001: EtherCAT Modular Device Profiles
- [9] ETG.5100: Safety over EtherCAT specification

Other References

- [10] IEC 61158-x-12 (all parts for type 12): Industrial communication networks – Fieldbus specifications
- [11] IEC 61784-2: Industrial communication networks – Profiles – Part 2: Additional fieldbus profiles for real-time networks based on ISO/IEC 8802-3
- [12] IEC 61800-7: Adjustable speed electrical power drives systems – Part 7: Generic interface and use of profiles for power drive systems
- [13] CiA 301: CANopen application layer and communication profile, www.can-cia.org

3 Terms, Definitions and Word Usage

3.1 Terms and Definitions

The terms and definitions of ETG.1000 series shall be fully valid, unless otherwise stated.

3.2 Word usage: shall, should, may, can

The word *shall* is used to indicate mandatory requirements strictly to be followed in order to conform to the standard and from which no deviation is permitted (*shall* equals *is required to*).

The word *should* is used to indicate that among several possibilities one is recommended as particularly suitable, without mentioning or excluding others; or that a certain course of action is preferred but not necessarily required; or that (in the negative form) a certain course of action is deprecated but not prohibited (*should* equals *is recommended that*).

The word *may* is used to indicate a course of action permissible within the limits of the standard (*may* equals *is permitted to*).

The word *can* is used for statements of possibility and capability, whether material, physical, or causal (*can* equals *is able to*).

NOTE 1: EtherCAT Medium Access Control and Terminology

The EtherCAT medium access control method follows the master/slave principle: only the main device sends frames, the subordinate devices process them. While it is considered ethically acceptable for one electronic device to impose communication behavior on another electronic device, there are people and institutions that have concerns about the use of these terms in technical descriptions and specifications. Since this document is not intended to offend any sensibilities, the term MainDevice (abbreviated MainDevice) replaces “master” and SubordinateDevice (abbreviated SubDevice) replaces “slave”.

4 Description of AL Status Codes

AL Status Codes are defined in ETG.1000.6, [5]. This chapter defines additional codes and provides additional information for already existing AL Status Codes, see Table 1.

Table 1: Description of AL Status Codes usage

Code	Meaning	Description	Current state or state change	Resulting state	Reference
0x0000	No error	No error	Any	Current	ETG.1000.6
0x0001	Unspecified error	General Error which is not defined in the following list	Any	Any +E	ETG.1000.6
0x0002	No Memory	Local Application runs out of memory (e.g. dynamic memory allocation for emergency messages)	Any	Any +E	ETG.1000.6
0x0003	Invalid Device Setup	This AL Status code should be used if the EtherCAT SubDevice has no valid setup of application specific settings. E.g. if a bus-coupler has no physical modules connected	PS	P +E	Additional code
0x0004	Invalid Revision	Output/Input mapping is not valid for this hardware or software revision (0x1018:03). Solution: Change mapping or use different hardware.	PS	P +E	Additional code
0x0006	SII/EEPROM information does not match firmware	if the SubDevice checks if the SII/EEPROM content matches the firmware, e.g. process data description or revision number, and detects a mismatch.	IP	I +E	Additional code
0x0007	Firmware update not successful. Old firmware still running	Error occurred during firmware update	Boot	I +E	Additional code
0x000E	License error	HW/SW license invalid or evaluation period expired Use case: if ESC IP-Core runs in a time-limited evaluation version, e.g. with a license chip. Using this in a product should be very carefully and noticeably documented because it may stop a EtherCAT system in some applications.	Any	I +E	Additional code
0x0011	Invalid requested state change	Requested state change is invalid (e.g. IS, IO,PO,PB,SB,OB)	Any	Current +E (not O +E)	ETG.1000.6
0x0012	Unknown requested state	Requested state change is unknown (e.g. if the requested state code is not 1,2,3,4,8)	Any	Current +E (not O +E)	ETG.1000.6
0x0013	Bootstrap not supported	The device doesn't support Bootstrap state	IB	I +E	ETG.1000.6
0x0014	No valid firmware	The downloaded file is no valid firmware file	IP	I +E	ETG.1000.6
0x0015	Invalid mailbox configuration	The mailbox SyncManager configuration is not valid in Bootstrap state	IB	I +E	ETG.1000.6
0x0016	Invalid mailbox configuration	The mailbox SyncManager configuration is not valid in PREOP state	IP	I +E	ETG.1000.6
0x0017	Invalid sync manager configuration	Sync manager configuration is not valid	PS, SO	Current +E (not O +E)	ETG.1000.6
0x0018	No valid inputs available	SubDevice application cannot provide valid inputs	O, SO	S +E	ETG.1000.6
0x0019	No valid outputs	SubDevice application cannot receive valid outputs	O, SO	S +E	ETG.1000.6

Code	Meaning	Description	Current state or state change	Resulting state	Reference
0x001A	Synchronization error	Multiple synchronization errors. Device is not synchronized any more (shall be used if no specific Synchronization error can be distinguished)	O, SO	S +E	ETG.1000.6
0x001B	Sync manager watchdog	No process data received yet (S->O) or not received within a specified timeout value	O, S	S +E	ETG.1000.6
0x001C	Invalid Sync Manager Types		O, S, PS	P +E	ETG.1000.6
0x001D	Invalid Output Configuration	SyncManger configuration for output process data is invalid	O, S, PS	P +E	ETG.1000.6
0x001E	Invalid Input Configuration	SyncManger configuration for input process data is invalid	O, S, PS	S +E	ETG.1000.6
0x001F	Invalid Watchdog Configuration	Watchdog Settings are invalid (e.g. SyncManger watchdog trigger is enabled but no watchdog timeout is defined)	O, S, PS	P +E	ETG.1000.6
0x0020	SubDevice needs cold start	SubDevice device requires a power off – power on reset	Any	Current +E (not O +E)	ETG.1000.6
0x0021	SubDevice needs INIT	SubDevice application requests INIT state	B, P, S, O	Current +E (not O +E)	ETG.1000.6
0x0022	SubDevice needs PREOP	SubDevice application requests PREOP state	S, O	Current +E (not O +E)	ETG.1000.6
0x0023	SubDevice needs SAFEOP	SubDevice application requests SAFEOP state	O	Current +E (not O +E)	ETG.1000.6
0x0024	Invalid Input Mapping	Input process data mapping does not match to expected mapping	PS	P +E	ETG.1000.6
0x0025	Invalid Output Mapping	Output process data mapping does not match to expected mapping	PS	P +E	ETG.1000.6
0x0026	Inconsistent Settings	General settings mismatch	PS	P +E	ETG.1000.6
0x0027	Freerun not supported		PS	P +E	ETG.1000.6
0x0028	Synchronization not supported		PS	P +E	ETG.1000.6
0x0029	Freerun needs 3 Buffer Mode	FreeRun Mode, sync manager has to run in 3-Buffer Mode	PS	P +E	ETG.1000.6
0x002A	Background Watchdog		S, O	P +E	ETG.1000.6
0x002B	No Valid Inputs and Outputs		O, SO	S +E	ETG.1000.6
0x002C	Fatal Sync Error	Fatal Sync Error: Sync0 or Sync1 are not received any more	O	S +E	ETG.1000.6
0x002D	No Sync Error	Sync not received: In SAFEOP the SubDevice waits for the first Sync0/Sync1 events before switching to OP, if these events were not received during the SAFEOP to OP-Timeout time the SubDevice should refuse the state transition to OP with this AL Status Code (SystemTimeOffset too big, no DC event received)	SO	S +E	ETG.1000.6
0x002E	Cycle time too small	EtherCAT cycle time is smaller than Minimum Cycle Time supported by SubDevice	SO	S +E	Additional Code
0x0030	Invalid DC SYNC Configuration	Distributed Clocks configuration is invalid due to application requirements	O, SO, PS	P +E, S +E	ETG.1000.6

Code	Meaning	Description	Current state or state change	Resulting state	Reference
0x0031	Invalid DC Latch Configuration	DC Latch configuration is invalid due to application requirements	O, SO, PS	P +E, S +E	ETG.1000.6
0x0032	PLL Error	MainDevice not synchronized, at least one DC event received	S, O	S +E	ETG.1000.6
0x0033	DC Sync IO Error	Multiple synchronization errors. IO is not synchronized any more	O, SO	S +E	ETG.1000.6
0x0034	DC Sync Timeout Error	Multiple synchronization errors. Too much SM Events missed	O, SO	S +E	ETG.1000.6
0x0035	DC Invalid Sync Cycle Time		PS	P +E	ETG.1000.6
0x0036	DC Sync0 Cycle Time	DC Sync0 Cycle time does not fit to application requirements	PS	P +E	ETG.1000.6
0x0037	DC Sync1 Cycle Time	DC Sync1 Cycle time does not fit to application requirements	PS	P +E	ETG.1000.6
0x0041	MBX_AOE		B, P, S, O	Current +E S +E	ETG.1000.6
0x0042	MBX_EOE		B, P, S, O	Current +E S +E	ETG.1000.6
0x0043	MBX_COE		B, P, S, O	Current +E S +E	ETG.1000.6
0x0044	MBX_FOE		B, P, S, O	Current +E S +E	ETG.1000.6
0x0045	MBX_SOE		B, P, S, O	Current +E S +E	ETG.1000.6
0x004F	MBX_VOE		B, P, S, O	Current +E S +E	ETG.1000.6
0x0050	EEPROM No Access	EEPROM not assigned to PDI	Any	Any +E (not O +E)	ETG.1000.6
0x0051	EEPROM Error	EEPROM access error	Any	Any +E (not O +E)	ETG.1000.6
0x0052	External Hardware not ready	This AL Status Code should be used if the EtherCAT-SubDevice refused the state transition due to an external connection to another device or signal is missing	Any	Any +E (not O +E)	Additional code
0x0060	SubDevice Restarted Locally		Any	I	ETG.1000.6
0x0061	Device Identification value updated	The device identification value was updated and is now valid	P	P +E	ETG.1000.6
0x0070	Detected Module Ident List does not match	Configured Module Ident List (0xF030) and Detected Module Ident list (0xF050) do not match Should be used for devices in which the physical hardware configuration (0xF050) determines the expected configuration	PS	P +E	Additional code
0x0080	Supply voltage too low		Any	Any +E (not O +E)	Additional code
0x0081	Supply voltage too high		Any	Any +E (not O +E)	Additional code
0x0082	Temperature too low		Any	Any +E (not O +E)	Additional code

Code	Meaning	Description	Current state or state change	Resulting state	Reference
0x0083	Temperature too high		Any	Any +E (not O +E)	Additional code
0x00F0	Application Controller available	The local application releases the application controller which is now available and services the EtherCAT state machine and all other device features This optional AL Status Code shall only be used for devices that have a different power supply for the ESC and the application controller and which cannot define a maximum timeout value for transition I→P NOTE: Use case can be if ESC is powered before application controller. Err Indication can be used by MainDevice to indicate that SubDevice is ready for boot-up (instead of cyclic polling)	I	I +E	Additional code

5 EEPROM

5.1 EEPROM Access

By default, EtherCAT MainDevice side has the access rights to the EEPROM. The EtherCAT MainDevice shall give the EEPROM access rights to the application controller by writing 0x0500.0 = 1 in the following transitions and states:

- in transition INIT – PREOP
- in transition INIT – BOOT and in state BOOT.
- If the Element “AssignToPdi” is set in the ESI-File the MainDevice shall grant the access right to the PDI in all states except INIT.

During power-up the ESC loads the first seven words from the EEPROM to configure its PDI. ESC is always accessible via EtherCAT, even if PDI Configuration Data (or even the whole EEPROM content), is filled with wrong data. That means the MainDevice/configuration logic can still rewrite EEPROM with updated information.

During production the EEPROM shall be loaded with the correct values; at least the first seven words of the EEPROM.

NOTE: To allow the application controller to check EEPROM content for correct content it might be useful to connect the two EEPROM I2C-signals to the controller. The controller can use the access to the EEPROM during power-up, before releasing the reset signal to the ESC.

5.2 EEPROM Content / SubDevice Information Interface (SII)

The basic content of the EEPROM is defined in ETG.1000.6, [5]. The ETG.2010, [7], extends the content and provides a comprehensive overview of the SubDevice Information Interface (SII).

6 Warning indication

6.1 General

The AL Control Response and AL State Changed parameter 'AL Status Code' is used to indicate ESM errors (see ETG.1000.6 [5]) and the Device Identification value (see ETG.1020, clause Explicit Device Identification).

According to ETG.1000.6 an error is a discrepancy between an observed and the correct value. One way to indicate errors is through the error indication (AL Status register 0x0130 bit 4) and the corresponding AL Status Code (register 0x0134), which usually leads to an AL state change.

Certain EtherCAT resources, such as the DPC (see clause 7, Dynamic Process Data Channel), are intended to be available with a high degree of independence from the AL state and no effect on the current operation mode of the application.

For this reason, the error handling of such resources should have a similar degree of independence from the AL state. Thus, in these cases errors are indicated via a warning indication instead of an error indication.

The warning indication is set through register 0x0130 bit 6. In case the warning indication is set, register 0x0134 shall provide a Warning Code.

The warning indication shall be acknowledged via the warning acknowledge (register 0x0120 bit 6). Upon a warning acknowledge, the warning indication bit shall be reset and the AL Status Code should be set to 0.

The error indication, warning indication and ID Loaded bits shall not be set at the same time. The error indication has a higher priority than the warning indication and ID Loaded. The warning indication has a higher priority than the ID Loaded. In case an error is indicated to the ESM, a pending warning indication shall be reset before the error indication is set. The warning indication and Warning Code shall be locally stored by the ESM and indicated again after the error indication has been acknowledged. The handling between warning indication and ID Loaded is analogous to the handling between error indication and ID Loaded (see ETG.1020, clause 21, Explicit Device Identification).

6.2 Description of Warning Code

Table 2 describes Warning Codes.

Table 2: Description of Warning Codes

Code	Meaning	Description	Source	Reference
0x0m02	Invalid mapping (triggered by the application)	Process data mapping of channel m is invalid or does not match with corresponding SyncManager length (offset +0x2 in the SyncManager structure).	DPC	ETG.1020
0x0m03	Invalid SyncManager length	SyncManager length of channel m (offset +0x2 in the SyncManager structure) is not valid.	DPC	ETG.1020
0x0m04	Invalid SyncManager settings	SyncManager settings (direction and buffer type) of channel m (offset +0x4 in the SyncManager structure) are not valid.	DPC	ETG.1020
0x0m05	Invalid SyncManager address range	SyncManager start address of channel m (offset +0x0 in the SyncManager structure) is not within a valid range.	DPC	ETG.1020
0x0m06	SyncManager overlapping memory	The SyncManager of channel m overlaps with other SyncManagers.	DPC	ETG.1020
0x0m07	No valid data available (triggered by the application)	Data values are not valid for the application	DPC	ETG.1020
0x0m09	Invalid SyncManager address align	SyncManager start address of channel m (offset +0x0 in the SyncManager structure) has invalid 16-bit or 32-bit alignment.	DPC	ETG.1020

Code	Meaning	Description	Source	Reference
0x0m11	ESM state not valid	Channel m cannot be open in Init, Pre-Operational or Bootstrap.	DPC	ETG.1020
0x0m12	ESM state not matching application requirements (triggered by the application)	Channel m cannot be open due to application-specific requirements in current ESM state (Safe-Operational or Operational).	DPC	ETG.1020
0x0m1A	Synchronization error (triggered by the application)	Channel m is not synchronized with the MainDevice cycle.	DPC	ETG.1020
0x0m1B	SyncManager watchdog (triggered by the application)	No process data received by channel m within a specified timeout.	DPC	ETG.1020
Other codes	reserved	All not defined Warning Codes shall be reserved.		

NOTE: Value 0 of the most significant nibble of the Warning Code indicates that the error source is the Dynamic Process Data Channel (DPC).

7 Dynamic Process Data Channel

7.1 Scope

According to ETG.1000.6 a channel is a management object to convey data. According to ETG.1000.6 process data are application objects to transfer data. A process data channel is a management object to convey process data.

A Dynamic Process Data Channel (DPC) is a process data channel which can be opened and closed between MainDevice and SubDevice when required by the application, without the need of an ESM state transition.

This clause specifies how to configure, open and close a Dynamic Process Data Channel and how to handle related errors.

A possible use case for the Dynamic Process Data Channel is the monitoring and tuning of the frequency behaviour of a servo drive. The Dynamic Process Data Channel is opened to start a frequency measurement when the SubDevice is in Operational state and the axis is enabled. The Dynamic Process Data Channel is closed once the measurement has been completed.

Another use case is the cyclically reading of application-specific diagnostic data with high priority from a SubDevice upon request of a client application.

7.2 DPC specific terms and definitions

Dynamic Process Data Channel

Process data channel which can be opened and closed to convey process data between MainDevice and SubDevice without requiring an ESM state transition

Dynamic Process Data Channel handler

Component of the application layer (AL) of the SubDevice connecting the data link layer (DL) of a Dynamic Process Data Channel with the application

Warning Code

Optional attribute controlled by the application reporting the error related to the Dynamic Process Data Channel

7.3 Functional principle

A Dynamic Process Data Channel (DPC) differs from a process data channel in its configuration and error handling. A DPC is configured, opened, and closed without requiring an ESM state transition. Configuration or operation errors of a DPC are indicated via a warning indication instead of an error indication, and hence do not require an ESM state change.

A DPC can be open in all ESM states in which a process data channel can be enabled (Safe-Operational, Operational). The DPC may be open even if the ESM error indication is set. Possible further restrictions in the operation of the DPC depending on the ESM state are given by the application. The application-specific settings, including the PDO mapping and assignment, shall be applied before the DPC is opened.

The component connecting the data link layer of a DPC to the application is called DPC handler. The DPC handler is part of the application layer in the ISO/IEC 7498 OSI Basic Reference Model (see Figure 1).

At data link layer each DPC is associated with one SyncManager and one corresponding FMMU. The DPC has the same direction (read or write) as the associated SyncManager and FMMU. The SyncManagers and FMMUs used for a DPC are defined with specific types both in the ESI file (see ETG.2000) and in the SII (see ETG.2010).

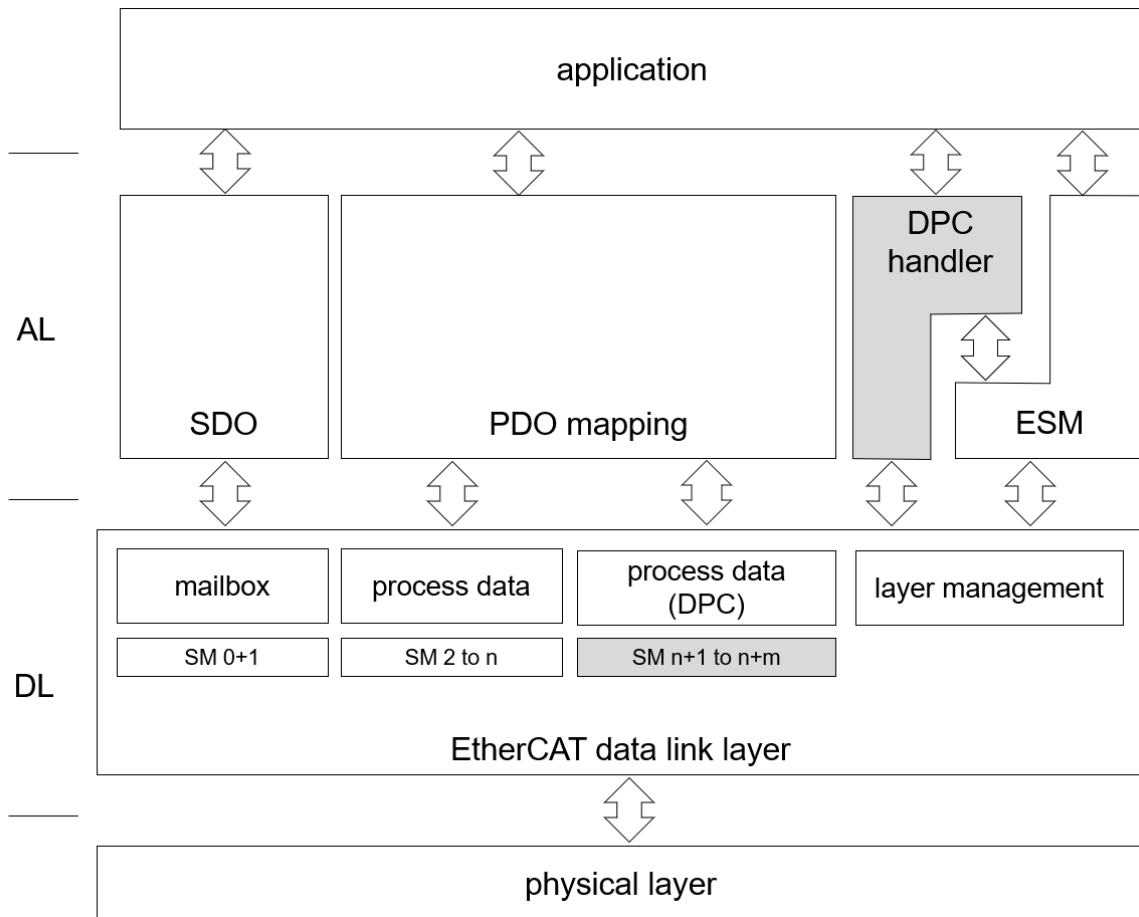


Figure 1: Dynamic Process Data Channel within the SubDevice reference model

The MainDevice requests to open/close the DPC by enabling/disabling the corresponding SyncManager. This issues a SyncManager change event (SM_Chg) in the data link layer of the SubDevice. Upon this SM_Chg event, the DPC handler checks the SyncManager settings and informs the application. The application requests the DPC handler to open/close the channel and the DPC handler activates/deactivates the corresponding SyncManager accordingly.

The DPC handler may close the DPC without a request from the MainDevice by deactivating the corresponding SyncManager due to communication errors or for reasons indicated by the application of the SubDevice.

Errors in the DPC, either triggered by the DPC handler or by the application, are indicated by issuing a warning indication towards the ESM (refer to ETG.1020, clause 6, Warning Indication).

7.4 State machine

7.4.1 General

The DPC operation is based on a SyncManager channel as specified by ETG.1000.4.

The set-up of the DPC includes the application-specific configuration and the configuration of the DPC itself. The application-specific configuration includes the PDO mapping and assignment, and optionally other settings. The set-up of the DPC includes the configuration of the related SyncManager channel.

The set-up sequence shall be as follows:

1. application-specific configuration
2. configuration of the DPC
3. enabling of the DPC

The MainDevice enables the SyncManager by setting the corresponding Channel enable bit (offset 0x6.0 within the SyncManager structure) to 1. By writing the Channel enable bit of the SyncManager, the SyncManager change event in bit .4 of DL-user Event register 0x0220 is set. Upon this event, the

DPC handler checks the settings of the SyncManager. In case of matching settings, the DPC handler sets an indication to the application. When the request to open the DPC is received from the application, the DPC handler activates the SyncManager by setting the corresponding SyncManager Deactivate Channel PDI bit (offset 0x7.0 within the SyncManager structure) to 0. By activating the SyncManager, the DPC is opened.

The timeout between the enable request and the activation of the DPC is application-specific and shall be monitored by the application of the MainDevice. As default, the application of the MainDevice may apply the timeout defined for the transition from Pre-Operational to Safe-Operational (see ETG.2000).

The SyncManager may be deactivated by the DPC handler by setting the SyncManager Deactivate Channel PDI bit (offset 0x7.0 within the SyncManager structure) to 1, or disabled by the MainDevice by setting the SyncManager Channel enable bit (offset 0x6.0 within the SyncManager structure) to 0.

The DPC State Machine (DPCSM) is described in Figure 2.

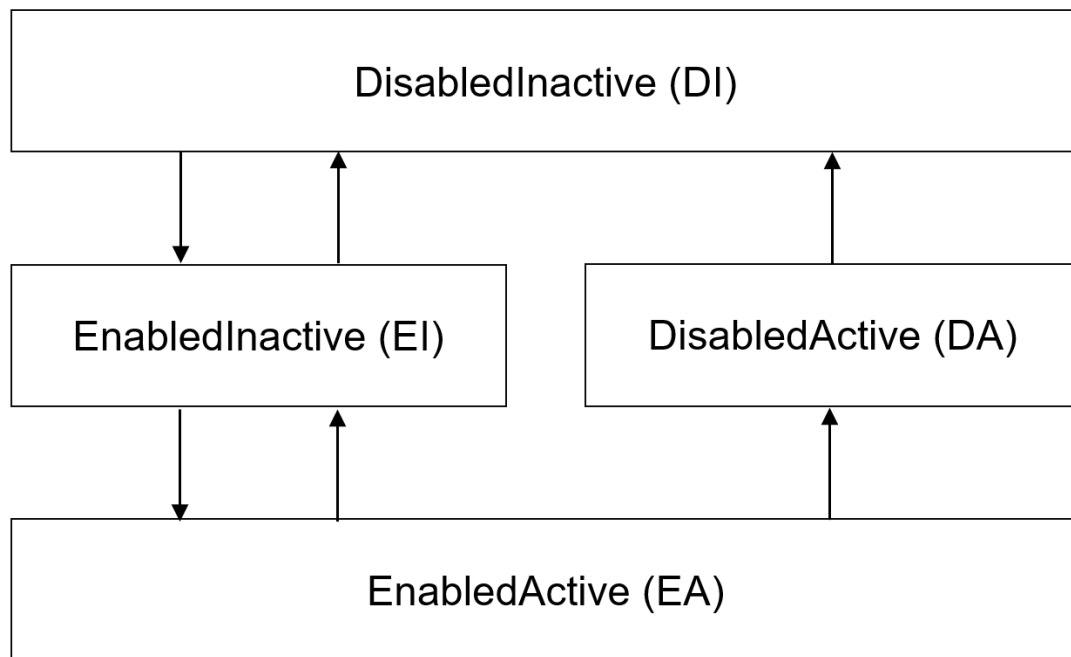


Figure 2: DPC State Machine

The DPC State Machine defines the transitions between the following four states.

DisabledInactive (DI)

In DisabledInactive state the SyncManager has not been enabled by the MainDevice and the DPC is closed.

EnabledInactive (EI)

In EnabledInactive state the SyncManager has been enabled by the MainDevice but not activated by the DPC handler, and the DPC is closed.

In this state the DPC handler checks the SyncManager settings and the ESM state. If the SyncManager settings match and the ESM state is a valid state for opening the DPC (Safe-Operational or Operational), the DPC handler informs the application about the validity of the settings and the SyncManager length.

The application checks the SyncManager length against the process data size according to the PDO mapping and assign objects. Additionally, the application checks the PDO mapping and assignment itself and other optional application-specific settings. In case these match, the application issues a request towards the DPC handler to open the DPC.

The DPC handler activates the SyncManager and the DPCSM state changes to EnabledActive state.

In case the settings checked by the DPC handler or by the application do not match, a warning indication is issued towards the ESM (see ETG.1020, clause Warning Indication) and the DPCSM remains in EnabledInactive state.

As the registers of the SyncManager structure can be changed only when the SyncManager is disabled, the MainDevice shall set the DPCSM state back to DisabledInactive before writing the SyncManager settings with different values.

EnabledActive (EA)

In EnabledActive state the SyncManager has been enabled by the MainDevice and activated by the DPC handler. The DPC is open and process data can be exchanged.

In EnabledActive state, the application may monitor the watchdog and synchronization of process data exchange, as well as ESM state transitions to states not matching application-specific requirements, and request the DPC handler to close the DPC in case of error.

In case the ESM performs a transition to an invalid state for DPC operation, or upon a request of the application, the DPC handler closes the DPC by deactivating the corresponding SyncManager. In this case, the DPCSM state changes to EnabledInactive and a warning indication towards the ESM is issued (see ETG.1020, clause Warning Indication).

The DPC can be closed from the MainDevice by disabling the SyncManager. In this case, the DPCSM state changes to DisabledActive state without a warning indication.

DisabledActive (DA)

In DisabledActive state the SyncManager has been disabled by the MainDevice but is still active. The DPC handler shall deactivate the SyncManager and the DPCSM state shall change to DisabledInactive.

7.4.2 DPC errors

Errors related to the DPC like invalid SyncManager settings or communication errors occurring during operation, as well as application-specific errors related to the DPC, are indicated through a warning indication (see ETG.1020, clause 6, Warning Indication). Warning Codes related to the operation of the DPC are defined in the same clause.

7.4.3 DPC primitives

Figure 3 shows the DPCSM primitives.

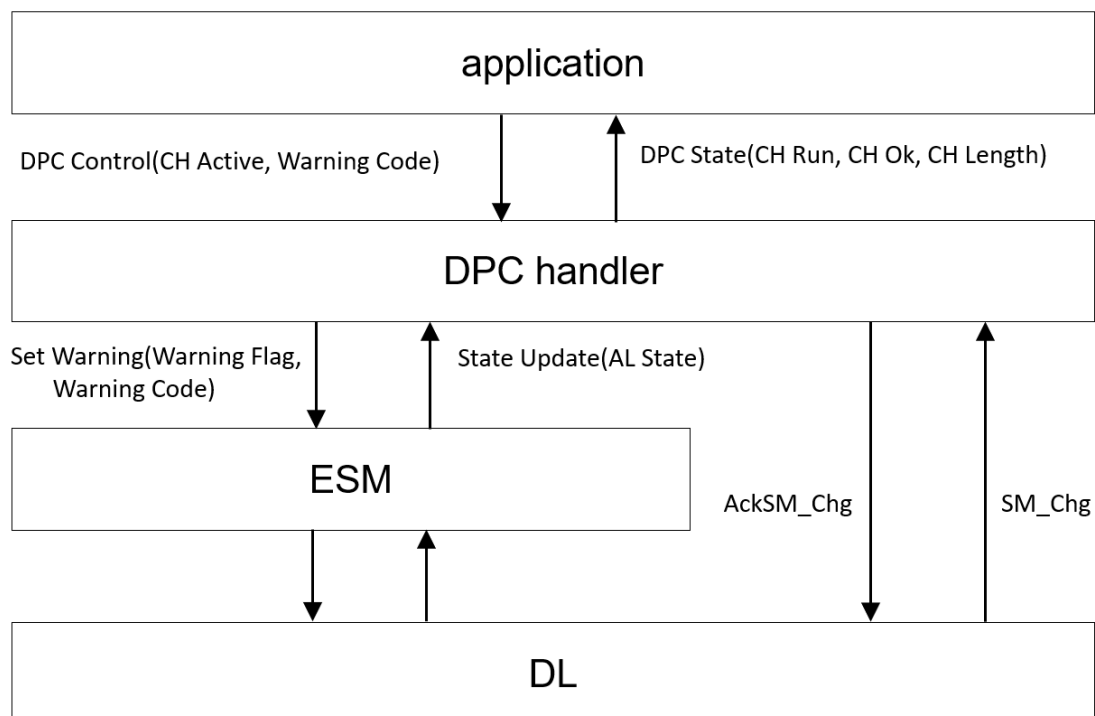


Figure 3: DPC primitives

7.4.3.1 Primitives exchanged between DL and DPCSM

Table 3 shows the service primitives including their associated parameters issued by the DPCSM and received by the DL.

Table 3: Primitives issued by DPCSM to DL

Primitive Name	Associated parameters	Functions
AckSM_Chg		SyncManager change event shall be acknowledged by the SubDevice.

Table 4 shows the service primitives including their associated parameters issued by the DL and received by the DPCSM.

Table 4: Primitives issued by DL to DPCSM

Primitive Name	Associated parameters	Functions
SM_Chg		Event service of DL to indicate that the SyncManager settings were written by the MainDevice

7.4.3.2 Primitives exchanged between ESM and DPCSM

Table 5 shows the service primitives including their associated parameters issued by the DPCSM and received by the ESM.

Table 5: Primitives issued by DPCSM to ESM

Primitive Name	Associated parameters	Functions
Set Warning	Warning Flag Warning Code	Sends warning indication and Warning Code to request the ESM to write AL Status and AL Status Code register

Table 6 shows the service primitives including their associated parameters issued by the ESM and received by the DPCSM.

Table 6: Primitives issued by ESM to DPCSM

Primitive Name	Associated parameters	Functions
State Update	AL State	Gets the information of the ESM state

7.4.3.3 Primitives exchanged between application and DPCSM

Table 7 shows the service primitives including their associated parameters issued by the DPCSM and received by the application.

Table 7: Primitives issued by DPCSM to application

Primitive Name	Associated parameters	Functions
DPC State	CH Run CH Ok CH Length	Informs the application if the DPC is open/closed, if the SyncManager settings are correct and about the configured SyncManager length

Table 8 shows the service primitives including their associated parameters issued by the application and received by the DPCSM.

Table 8: Primitives issued by application to DPCSM

Primitive Name	Associated parameters	Functions
DPC Control	CH Active Warning Code	Sends the request to de-/activate the DPC, sets an application error or acknowledges that the application error is removed

7.4.4 DPC variables

Table 9 defines the variables used in the DPCSM.

Table 9: DPC variables

Variable	Description
localState	State of the DPC State Machine
ChannelOk	Internal variable which indicates that the settings of the SyncManager used for the DPC are correct

7.4.5 DPC macros

Table 10 defines the macros used in the DPCSM.

Table 10: DPC macros

Macro	Description
SM_SETTINGS_MATCH	Local function checking that the settings requested for the SyncManager used for the DPC match to the local settings.
SM_INVALID_LENGTH	Local function that signals an invalid SyncManager Length (register offset 0x02) of the SyncManager used for the DPC.
SM_INVALID_CONTROL	Local function that signals an invalid SyncManager Control (register offset 0x04) of the SyncManager used for the DPC.
SM_INVALID_ADDRESS_RANGE	Local function that signals an invalid SyncManager Address Range (register offset 0x00) of the SyncManager used for the DPC.
SM_OVERLAPPING_MEMORY	Local function that signals if the memory offset range of the SyncManager used for the DPC overlaps with that of other SyncManagers.
SM_INVALID_ADDRESS_ALIGN	Local function that signals an invalid Address Align (concerning SyncManager register offset +0x0) of the SyncManager used for the DPC.
SM_ENABLED	Local function that checks that the SyncManager is enabled.
ACTIVATE_SM_CHANNEL	Local function that activates the SyncManager.
DEACTIVATE_SM_CHANNEL	Local function that deactivates the SyncManager.
ESM_STATE_MATCH	Local function that checks that the actual ESM state is Safe-Operational or Operational (further application-specific limitations shall be checked by the application)
SM_LENGTH	Local function that gets the SyncManager Length (register offset 0x02) of the SyncManager used for the DPC.
CONF_WARNING_CODE	If SM_INVALID_LENGTH then Warning Code = 0x0m03 else If SM_INVALID_CONTROL then Warning Code = 0x0m04 else if SM_INVALID_ADDRESS_RANGE then Warning Code = 0x0m05 else if SM_OVERLAPPING_MEMORY then Warning Code = 0x0m06 else if SM_INVALID_ADDRESS_ALIGN then Warning Code = 0x0m09 end_if

7.4.6 DPC state table

Table 11 contains the complete description of the DPC State Machine.

Table 11: DPCSM state table

#	Current State	Event /Condition => Action	Next State
1	DI	State Update (AL State) => ignore	DI
2	DI	DPC Control (CH Active, Warning Code) => ignore	DI
3	DI	SM_Chg / not SM_ENABLED => AckSM_Chg	DI
4	DI	SM_Chg / SM_ENABLED => localState = EnabledInactive	EI
101	EI	State Update (AL State) => ignore	EI
102	EI	DPC Control (CH Active, Warning Code) => ignore	EI
103	EI	SM_Chg / SM_ENABLED and SM_SETTINGS_MATCH and not ESM_STATE_MATCH => Warning Flag = true Warning Code = 0x0m11 Set Warning (Warning Flag, Warning Code) AckSM_Chg	EI
104	EI	SM_Chg / SM_ENABLED and not SM_SETTINGS_MATCH and ESM_STATE_MATCH => Warning Flag = true Warning Code = CONF_WARNING_CODE Set Warning (Warning Flag, Warning Code) AckSM_Chg	EI
105	EI	SM_Chg / SM_ENABLED and SM_SETTINGS_MATCH and ESM_STATE_MATCH => ChannelOk = true Ch Run = false CH Ok = true CH Length = SM_LENGTH DPC State (CH Run, CH Ok, CH Length) AckSM_Chg	EI
106	EI	DPC Control (CH Active, Warning Code) / CH Active = false and not Warning Code = 0 => Warning Flag = true Set Warning (Warning Flag, Warning Code) CH Ok = false CH Length = 0 DPC State (CH Run, CH Ok, CH Length)	EI
107	EI	DPC Control (CH Active, Warning Code) / Warning Code = 0 and CH Active = true and ChannelOk = true => ACTIVATE_SM_CHANNEL localState = EnabledActive CH Run = true DPC State (CH Run, CH Ok, CH Length)	EA

#	Current State	Event /Condition => Action	Next State
108	EI	SM_Chg / not SM_ENABLED => ChannelOk = false Warning Flag = false Warning Code = 0 localState = DisabledInactive CH Ok = false CH Length = 0 DPC State (CH Run, CH Ok, CH Length) AckSM_Chg	DI
201	EA	State Update (AL State) / ESM_STATE_MATCH => ignore	EA
202	EA	SM_Chg / SM_ENABLED => AckSM_Chg	EA
203	EA	State Update (AL State) / not ESM_STATE_MATCH => ChannelOk = false Warning Flag = true Warning Code = 0x0m11 DEACTIVATE_SM_CHANNEL localState = EnabledInactive Set Warning (Warning Flag, Warning Code) CH Run = false CH Ok = false CH Length = 0 DPC State (CH Run, CH Ok, CH Length)	EI
204	EA	DPC Control (CH Active, Warning Code) / CH Active = false and not Warning Code = 0 => ChannelOk = false Warning Flag = true DEACTIVATE_SM_CHANNEL localState = EnabledInactive Set Warning (Warning Flag, Warning Code) CH Run = false CH Ok = false CH Length = 0 DPC State (CH Run, CH Ok, CH Length)	EI
205	EA	SM_Chg / not SM_ENABLED => ChannelOk = false localState = DisabledActive CH Run = false CH Ok = false CH Length = 0 DPC State (CH Run, CH Ok, CH Length)	DA
301	DA	State Update (AL State) => ignore	DA
302	DA	DPC Control (CH Active, Warning Code) => ignore	DA
303	DA	SM_Chg => DEACTIVATE_SM_CHANNEL localState = DisabledInactive AckSM_Chg	DI

8 Mailbox

8.1 Mailbox InitCmds

During start-up the MainDevice can send Initialization commands to SubDevice's register (Register InitCmd) and to the mailbox (Mailbox InitCmd).

Sequence of Mailbox InitCmds and Register InitCmds depends on state transition:

- State transition from INIT:
Register InitCmds shall be send first and Mailbox InitCmds afterwards.
Mailbox InitCmds can be sent in state transition INIT-PREOP and INIT-BOOT if the sequence is kept. This allows configuration of Mailbox objects, e.g. to send EoE InitCmds in transition INIT-PREOP.
- other transitions:
Mailbox InitCmds shall be sent first and afterwards the Register InitCmds.
This allows the SubDevice to verify the configuration.

8.2 Mailbox Header

The mailbox header (defined in ETG.1000.6) contains an address field that allows the MainDevice to redirect services, see Table 12.

Table 12: Mailbox Header definition (referenced from ETG.1000.6)

Frame part	Data Field	Data Type	Description
Mailbox Header	Length	WORD	Length of Mailbox Service Data = 4
	Address	WORD	Station Address of the source, if a MainDevice is client, Station Address of the destination, if a SubDevice is client
	Channel	Unsigned6	Reserved for future use
	Priority	Unsigned2	Reserved for future use
	Type	Unsigned4	Mailbox protocol type (CoE, EoE, FoE, ...)
	Counter	Unsigned3	Counter for Mailbox Data Link layer
	Reserved	Unsigned1	Reserved for future use

Address:

If a MainDevice is sending the mailbox message:

Station address of the source

If a SubDevice is sending the mailbox message:

Station address of the destination

For MainDevice-to-SubDevice mailbox communication the MainDevice is starting the Mailbox request as shown in Figure 4.

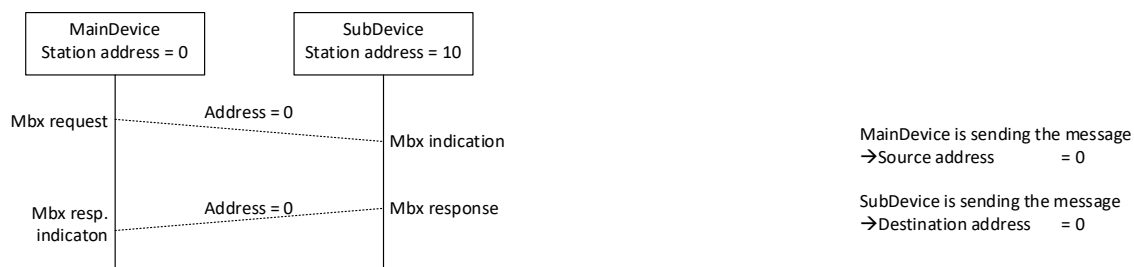


Figure 4: MainDevice-to-SubDevice mailbox communication

For SubDevice-to-SubDevice mailbox communication the MainDevice device acts like a router. SubDevice 1 request mailbox information from SubDevice 2, as shown in Figure 5.

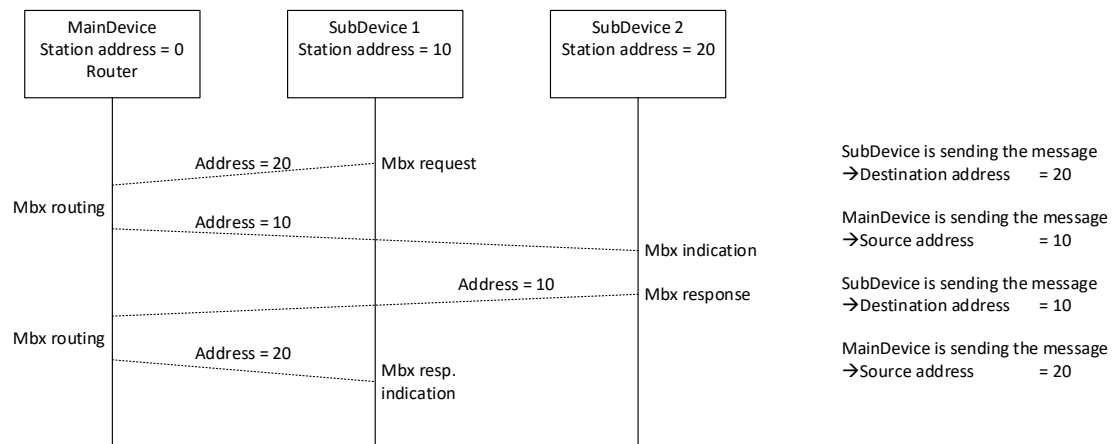


Figure 5: SubDevice-to-SubDevice mailbox communication

9 ADS over EtherCAT (AoE)

An acyclic service to a fieldbus SubDevice normally needs request and response data so that a SDO Download/SDO Upload combination would be necessary. This could be done with a command object but there is always an extended communication overhead because at least two complete SDO services have to be transmitted to get a successful acyclic service transmission.

To reach a good enough performance parallel services are needed that acyclic service requests to different fieldbus SubDevices can be transmitted at the same time, but the SDO service cannot be used in parallel.

Therefore, it makes sense to use the AoE services (ADS over EtherCAT) for accessing fieldbus SubDevices (or modules) with acyclic services. AoE services can be transmitted in parallel because every AoE command contains an Invoke ID which identifies the request.

For AoE services the mailbox type 1 is used.

9.1 AoE Mailbox Frame

The mailbox frame coding is defined in Table 13:

Table 13: Mailbox frame with AoE

Frame part	Data Field	Data Type	Value / Description
Mailbox Header	Length	WORD	Length of the mailbox data
	Address	WORD	Station Address of the source, if a MainDevice is client, Station Address of the destination, if a SubDevice is client
	Channel	UNSIGNED6	0x00 (reserved for future)
	Priority	UNSIGNED2	0x00 (reserved for future)
	Type	UNSIGNED4	0x01: AoE
	Cnt	UNSIGNED3	Counter of the mailbox services (0 reserved, 1 is start value, next value after 7 is 1)
	Reserved	UNSIGNED1	0x00
AoE Header		OCTET-STRING[32]	see Table 14
AoE Command		OCTET-STRING	Command structure according to AoE Command in AoE Header

The AoE Header is defined in Table 14:

Table 14: AoE Header

AoE Header	Offset (in Bytes)	Data Type	Description
Target Net ID ¹	0	OCTET-STRING[6]	Destination Net-ID
Target Port ²	6	UNSIGNED16	Destination Port
Sender Net ID	8	OCTET-STRING[6]	Source Net-ID
Sender Port	14	UNSIGNED16	Source Port
Command ID	16	UNSIGNED16	Command ID of the AoE service, see Table 15
State Flags	18	UNSIGNED16	Bit 0: 0 = Request, 1 = Response Bit 2: 1 Bit 1, 3...15: 0
Data Size	20	UNSIGNED32	size of the data following the AoE header
Error Code	24	UNSIGNED32	error code (only for responses), see Table 16
Invoke ID	28	UNSIGNED32	Invoke ID (will be copied unchanged from request to response)

¹The Net ID defines a unique identification of a device used for the AoE communication.

²The Port number shall be unique within the device.

The AoE Commands are defined in Table 15.

Table 15: AoE Commands

AoE Command	Command ID	Description
AoE Read	2	AoE Read command to read data
AoE Write	3	AoE Write command to write data
AoE WriteControl	5	AoE Write Control command change ADS state and device state and write data
AoE ReadWrite	9	AoE ReadWrite command to exchange data in both directions (read and write)

The "Result" field of the AoE services consists of two parts. The low word (16 bit) is the specific error code. The high word (16 bit) contains an additional info of the error code. The standard entry for the high word is "ads device error (0x0700)".

The AoE Result codes are defined in Table 16.

Table 16: AoE Result codes (low word)

Error Code	Description
0x00	Error class < device error >
0x01	Service is not supported by server
0x02	invalid indexGroup
0x03	invalid indexOffset
0x04	reading/writing not permitted
0x05	parameter size not correct
0x06	invalid parameter value(s)
0x07	device is not in a ready state
0x08	device is busy
0x09	Reserved
0x0A	out of memory
0x0B	invalid parameter value(s)
0x0C	not found (files, ...)
0x0D	syntax error in comand or file
0x0E	objects do not match
0x0F	object already exists
0x10..0x18	Reserved
0x19	device has a timeout
0x20..0x22	Reserved
0x23	access denied
0x24..0xFF	Reserved

9.2 AoE Commands

9.2.1 AoE Read

The AoE Read Request Header is defined in Table 17.

Table 17: AoE Read Request Header

AoE Read Request Header	Offset (in Bytes)	Data Type	Description
Index Group	0	UNSIGNED32	Index Group to address the requested data see clause 9.5
Index Offset	4	UNSIGNED32	Index Offset to address the requested data see clause 9.5
Read Length	8	UNSIGNED32	length of the data to be read

The AoE Read Response Header is defined in Table 18.

Table 18: AoE Read Response Header

AoE Read Response Header	Offset (in Bytes)	Data Type	Description
Result	0	UNSIGNED32	Result (0 = Success) of the read operation Otherwise see Table 16 for error information
Read Length	4	UNSIGNED32	length of the following data
Read Data	8	OCTET-STRING	read data

9.2.2 AoE Write

The AoE Write Request Header is defined in Table 19.

Table 19: AoE Write Request Header

AoE Write Request Header	Offset (in Bytes)	Data Type	Description
Index Group	0	UNSIGNED32	Index Group to address the requested data see clause 9.5
Index Offset	4	UNSIGNED32	Index Offset to address the requested data see clause 4.4
Write Length	8	UNSIGNED32	length of the following data
Write Data	12	OCTET-STRING	data to be written

The AoE Write Response Header is defined in Table 20.

Table 20: AoE Write Response Header

AoE Write Response Header	Offset (in Bytes)	Data Type	Description
Result	0	UNSIGNED32	Result (0 = Success) of the write operation Otherwise see Table 16 for error information

9.2.3 AoE WriteControl

The AoE WriteControl Request Header is defined in Table 21.

Table 21: AoE WriteControl Request Header

AoE WriteControl Request Header	Offset (in Bytes)	Data Type	Description
AoE State	0	UNSIGNED16	AoE State (standardized)
Device State	2	UNSIGNED16	device specific state
Write Length	4	UNSIGNED32	length of the following data
Write Data	8	OCTET-STRING	data to be written

The AoE WriteControl Response Header is defined in Table 22.

Table 22: AoE WriteControl Response Header

AoE WriteControl Response Header	Offset (in Bytes)	Data Type	Description
Result	0	UNSIGNED32	Result (0 = Success) of the write operation Otherwise see Table 16 for error information

9.2.4 AoE ReadWrite

The AoE ReadWrite Request Header is defined in Table 23.

Table 23: AoE ReadWrite Request Header

AoE ReadWrite Request Header	Offset (in Bytes)	Data Type	Description
Index Group	0	UNSIGNED32	Index Group to address the requested data
Index Offset	4	UNSIGNED32	Index Offset to address the requested data
Read Length	8	UNSIGNED32	length of the data to be read
Write Length	12	UNSIGNED32	length of the following data
Write Data	16	OCTET-STRING	data to be written

The AoE ReadWrite Response Header is defined in Table 24.

Table 24: AoE ReadWrite Response Header

AoE ReadWrite Response Header	Offset (in Bytes)	Data Type	Description
Result	0	UNSIGNED32	Result (0 = Success) of the read operation Otherwise see Table 16 for error information
Read Length	4	UNSIGNED32	length of the following data
Read Data	8	OCTET-STRING	read data

9.3 AoE Fragmentation

The AoE Fragmentation command uses the Invoke ID for the fragmentation of the AoE command to be transmitted (see Table 25).

Table 25: AoE Header for AoE Fragmentation command

AoE Header	Offset (in Bytes)	Data Type	Description
Target Net ID	0	OCTET-STRING[6]	Destination Net-ID
Target Port	6	UNSIGNED16	Destination Port
Sender Net ID	8	OCTET-STRING[6]	Source Net-ID
Sender Port	14	UNSIGNED16	Source Port
Command ID	16	UNSIGNED16	0x0902
State Flags	18	UNSIGNED16	0x0004
Data Size	20	UNSIGNED32	size of the data following the AoE header
Error Code	24	UNSIGNED32	error code (only for responses), see Table 16
Fragment Number	28	UNSIGNED16	increasing number with every fragment starting with 0
Packet Number	30	UNSIGNED16	increasing number with every new fragmented AoE command (shall be the same for all fragments of one AoE command)

The AoE command will be split in several AoE fragments depending on the mailbox and AoE command size. The first 16 bytes of the AoE command shall not be transmit (because they are included in the AoE fragmentation header). So an AoE command with size 500 and mailbox size 100 will be transmitted with 8 fragments as shown in Table 26.

Table 26: AoE Fragmentation Example

Fragment Number	Mailbox header	AoE Fragmentation header	AoE command
0	6 bytes	32 bytes	62 bytes (offset 16-77)
1	6 bytes	32 bytes	62 bytes (offset 78-139)
2	6 bytes	32 bytes	62 bytes (offset 140-201)
3	6 bytes	32 bytes	62 bytes (offset 202-263)
4	6 bytes	32 bytes	62 bytes (offset 264-325)
5	6 bytes	32 bytes	62 bytes (offset 326-387)
6	6 bytes	32 bytes	62 bytes (offset 388-449)

9.4 AoE Initialization

The AoE Net ID for the EtherCAT SubDevice will be transported with a special AoE Write command in state transition to PREOP as shown in Table 27 and Table 28.

Table 27: AoE Header for the AoE Initialization Write Command

AoE Header	Offset (in Bytes)	Data Type	Description
Target Net ID	0	OCTET-STRING[6]	AoE Net-ID of the EtherCAT MainDevice
Target Port	6	UNSIGNED16	EtherCAT Address of the EtherCAT SubDevice (Register 0x10-0x11)
Sender Net ID	8	OCTET-STRING[6]	AoE Net-ID of the EtherCAT MainDevice
Sender Port	14	UNSIGNED16	EtherCAT Address of the EtherCAT SubDevice (Register 0x10-0x11)
Command ID	16	UNSIGNED16	3 = AoE Write
State Flags	18	UNSIGNED16	Bit 0: 0 = Request, 1 = Response Bit 2: 1 Bit 1, 3...15: 0
Data Size	20	UNSIGNED32	12+6
Error Code	24	UNSIGNED32	0
Invoke ID	28	UNSIGNED32	Invoke ID (will be copied unchanged from request to response)

Table 28: AoE Write Request Header for the AoE Initialization Write Command

AoE Write Request Header	Offset (in Bytes)	Data Type	Description
Index Group	0	UNSIGNED32	1
Index Offset	4	UNSIGNED32	3
Write Length	8	UNSIGNED32	6
Write Data	12	OCTET-STRING[6]	AoE Net-ID of the EtherCAT SubDevice

9.5 Mailbox Services with AoE

AoE can embed other mailbox protocols. With AoE the mailbox services can be transmitted from the EtherCAT MainDevice to the EtherCAT SubDevice. The EtherCAT SubDevice can transmit the mailbox services to the underlying modules by the following alternatives:

- via a profile specific protocol (e.g. fieldbus gateways)
- via the mailbox protocol on the module's communication protocol

- by transmitting AoE to modules

The underlying modules can have their own object dictionary that can be accessed via embedded CoE protocol or a new firmware can be downloaded via embedded FoE.

A module will be addressed by its own port; the mailbox service will be selected by the index group. Details about the own dictionary and the own ports can be found in section "Modular Object Dictionary", which is part of the Modular Device Profiles, see [8].

9.5.1 CoE Services

For the CoE services the index groups shown in Table 29 are defined.

Table 29: Index Group of the CoE Services

CoE Service	AoE Service	AoE Port	AoE IndexGroup	AoE IndexOffset	AoE Data
SDO Upload	Read	EtherCAT SubDevice Address or 0xFFFF for MainDevice	0xF302	Bit 16-31: Index Bit 8: Complete Access Bit 0-7: SubIndex	Response: read data
SDO Download	Write	EtherCAT SubDevice Address or 0xFFFF for MainDevice	0xF302	Bit 16-31: Index Bit 8: Complete Access Bit 0-7: SubIndex	Request: data to be written
SDO Information Get Object List	Read	EtherCAT SubDevice Address or 0xFFFF for MainDevice	0xF3FC	Bit 16-31: List Type Request	Response: Word 0: List Type Word 1..n: List
SDO Information Get Object Description	Read	EtherCAT SubDevice Address or 0xFFFF for MainDevice	0xF3FD	Bit 16-31: Index Request	Response: Object Description see [5]
SDO Information Get Entry Description	Read	EtherCAT SubDevice Address or 0xFFFF for MainDevice	0xF3FE	Bit 16-31: Index Bit 8-15: ValueInfo Bit 0-7: SubIndex	Response: Entry Description see [5]

9.5.2 FoE Services

The mapping of the FoE services in the AoE protocol is defined in Table 30

Table 30: AoE Mapping of FoE Services

FoE Service	AoE Service	AoE Port	AoE IndexGroup	AoE IndexOffset	AoE Data
FoE OpenRead	ReadWrite	EtherCAT SubDevice Address or 0xFFFF for MainDevice	0xF401	Password	Request: File name Response: File handle (4 bytes and optional error text)
FoE Read	ReadWrite	EtherCAT SubDevice Address or 0xFFFF for MainDevice	0xF404	File handle	Request: - Response: File data
FoE OpenWrite	ReadWrite	EtherCAT SubDevice Address or 0xFFFF for MainDevice	0xF402	Password	Request: File name Response: File handle (4 bytes and optional error text)
FoE Write	ReadWrite	EtherCAT SubDevice Address or 0xFFFF for MainDevice	0xF405	File handle	Request: File data Response: -

9.5.3 SoE Services

The mapping of the SoE services in the AoE protocol is defined in Table 31

Table 31: AoE Mapping of SoE Services

SoE Service	AoE Service	AoE Port	AoE IndexGroup	AoE IndexOffset	AoE Data
SoE Read	Read	EtherCAT SubDevice Address	0xF420	Bit 0-15: IDN Bit 16: Data State Bit 17: Name Bit 18: Attribute Bit 19: Unit Bit 20: Min Value Bit 21: Max Value Bit 22: Value Bit 23: Default value Bit 24-26: Drive Bit 27-31: 0	Request: - Response: data
SoE Write	Write	EtherCAT SubDevice Address	0xF420	Bit 0-15: IDN Bit 16: Data State Bit 17: Name Bit 18: Attribute Bit 19: Unit Bit 20: Min Value Bit 21: Max Value Bit 22: Value Bit 23: Default value Bit 24-26: Drive Bit 27-31: 0	-

9.5.4 VoE Services

The mapping of the VoE services in the AoE protocol is defined in Table 32

Table 32: AoE Mapping of VoE Services

VoE Service	AoE Service	AoE Port	AoE IndexGroup	AoE IndexOffset	AoE Data
VoE Read / Write	ReadWrite	EtherCAT SubDevice Address or 0xFFFF for MainDevice	0xF430	0x00	DWORD: Vendor ID WORD: Vendor protocol Type Protocol specific data

10 CoE Abort Codes (extension)

CoE Abort Codes are defined in ETG.1000.6, [5]. Additional codes listed in Table 33 are defined here.

Table 33: CoE Abort Codes (extension)

Value	Meaning
0x06 01 00 03	Subindex cannot be written, SI0 must be 0 for write access
0x06 01 00 04	SDO Complete access not supported for objects of variable length such as ENUM object types
0x06 01 00 05	Object length exceeds mailbox size
0x06 01 00 06	Object mapped to RxPDO, SDO Download blocked This optional Abort Code is used only in states SafeOp and Op.
0x06 09 00 33	configured module list does not match detected module list It shall be used if Object 0xF03x is written but does not fit to object 0xF05x

11 Command Object

The command object should be used for actions or services (commands in the following) which cannot be adapted to a single SDO Upload or SDO Download service; for example if the command needs request and response data or if the processing of the command needs time so that the SDO timeout would expire with a standard transfer.

With the command object the command consists of at least two SDO services:

- By writing the SubIndex 1 with SDO Download of the command object the command is started.
- By reading the SubIndex 3 with SDO Upload the response is fetched. If the response is not available when reading SubIndex 3 the first byte of the response data should give information about the progress.

The SubIndex 1 is not writable until the command is finished and the response is read, so that multiple accesses from different applications cannot overwrite each other. In this case CoE Abort Code 0x08 00 00 21 "Data cannot be transferred or stored to the application because of local control" shall be used.

If a value is written to SubIndex 1 which is not supported and command validation is done in SDO download of SubIndex1, the CoE Abort Code 0x06040043 "General parameter incompatibility reason" should be returned. In this case, the command is not started. The status in SubIndex 2 remains unchanged. If a command validation is performed in command execution after SDO download, the status in SubIndex 2 should reflect the error.

The SubIndex 2 is only defined for compatibility reasons to the corresponding CANopen DS301 definition. The data type of the command object's object description shall be 0x0025.

The command object structure is shown in Table 34.

Table 34: Command object structure

SubIndex	Description	Data Type	Access	Value
1	Command	OCTET_STRING	RW	Byte 0-n: Service Request Data A write access to the command data will execute the command
2	Status	UNSIGNED8	RO	0: last command completed, no errors, no response data 1: last command completed, no errors, response data available 2: last command completed, error, no response data 3: last command completed, error, response data available 4-99: reserved for future use 100-200: indicates how much of the command has been executed (in %, 100 = 0 %, 200 = 100 %) 201-254: reserved for future use 255: command is executing (if the percentage display is not supported)
3	Response	OCTET_STRING	RO	Byte 0: see SubIndex 2 Byte 1: unused 2-n: Service Response Data

NOTE: Alternative name for SubIndex 1 "Request"

12 CoE Communication Area

12.1 Object 0x100B: Bootloader Version

The Bootloader Version object is defined in Table 35. The bootloader version value shall follow the same string representation as defined in index 0x100A for Manufacturer Software Version.

Table 35: Bootloader Version object structure

Attribute	Value
Index	0x100B
Name	Bootloader Version
Object Code	VAR
Data Type	VISIBLE_STRING
Category	Optional
Access	RO
PDO Mapping	No
Value	Vendor-specific version of the bootloader software

The bootloader version object is readable in PREOP, SAFEOP, OP.

If the object 0x100B is supported but the current bootloader version cannot be derived, the value of 0x100B shall be "N/A" (byte length of 3).

12.2 Object 0x1010: Store parameters

Parameters in the CoE object entries may be stored in non-volatile memory. This behaviour is indicated by the Backup flag for such an object entry. By default, those parameters are stored non-volatile once they are written per SDO Download. This is also called automatic storage mode.

The Store parameters object defines different storage modes: automatic storage mode and manual storage mode. Storage modes are selectable, and the currently selected storage mode is readable. The Store parameters object defines a method to store backup parameters manually.

The Store parameters object is defined in Table 1.

Table 36: 0x1010 Store parameters object

Attribute	Value
Index	0x1010
Name	Store parameters
Object Code	ARRAY
Max Subindex	highest subindex supported
Category	Optional

The entry description of 0x1010 is defined in Table 37.

Table 37: Entries of 0x1010 Store parameters

SubIndex	Description	M/O/C	Access	PDO mapping	Data Type	Value
0	Number of entries	M	R	No	UNSIGNED8	
1	Save all parameters	M	RW R, if automatic storage mode supported	No	UNSIGNED32	Save backup parameters to non-volatile memory Read: Bit 0 = vendor specific Bit 1 = 0: Manual storage mode for backup parameters Bit 1 = 1: Automatic storage mode for backup parameters Bit 2-31: reserved, shall be 0 Write: Only upon writing the value 0x65766173 ("e" "v" "a" "s" = save) all backup entries are stored in non-volatile memory of the SubDevice. This is called Manual storage. NOTE: An ESM state transition to INIT or power-cycle may be required in addition to store backup parameters.
2	Save communication parameters	O	RW R, if automatic storage mode supported	No	UNSIGNED32	As subindex 1
3	Save application parameters	O	RW R, if automatic storage mode supported	No	UNSIGNED32	As subindex 1
4..31	Save manufacturer defined parameters	O	RW R, if automatic storage mode supported	No	UNSIGNED32	As subindex 1
32..255	Reserved					

NOTE: ESI element Object/Flags@Backup refers to the SDO entry description parameter backup.

In case an invalid password is written or the object entry cannot be written in a certain ESM state the SDO Abort Code 0x08 00 00 2y (y = 0 or 1 or 2) should be returned.

In case the default parameters cannot be stored the SDO Abort Code 0x06 06 00 00 should be returned.

12.3 Object 0x1011: Restore default parameters

The Restore default parameters object defines a method to restore CoE object entries of type backup to their default values.

The Restore default parameters object is defined in Table 38.

Table 38: 0x1011 Restore default parameters object

Attribute	Value
Index	0x1011
Name	Restore default parameters
Object Code	ARRAY
Max Subindex	highest subindex supported
Category	Optional

The entry description of 0x1011 is defined in Table 39.

Table 39: Entries of 0x1011 Restore default parameters

SubIndex	Description	M/O/C	Access	PDO mapping	Data Type	Value
0	Number of entries	M	R	No	UNSIGNED8	
1	Restore all default parameters	M	RW	No	UNSIGNED32	Restore backup parameters Read: Bit 0 = vendor specific Bit 1-31: reserved, shall be 0 Write: With the value 0x64616F6C ("d" "a" "o" "l" = load) the backup entries are restored to their default values. NOTE: An ESM state transition to INIT or power-cycle may be necessary for default values to be valid.
2	Restore communication default parameters	O	RW	No	UNSIGNED32	As subindex 1
3	Restore application default parameters	O	RW	No	UNSIGNED32	As subindex 1
4..31	Restore manufacturer defined parameters	O	RW	No	UNSIGNED32	As subindex 1
32..255	reserved					

NOTE: ESI element Object/Flags@Backup refers to the SDO entry description parameter backup.

In case an invalid password is written or the object entry cannot be written in a certain ESM state the SDO Abort Code 0x08 00 00 2y (y = 0 or 1 or 2) should be returned.

In case the default parameters cannot be restored the SDO Abort Code 0x06 06 00 00 should be returned.

12.4 Object 0x1020: Device Statistics

For diagnosis of EtherCAT devices it is helpful to be able to read out device statistics.

The Device Statistics object is defined in Table 40, the entry description is given in Table 41.

Table 40: 0x1020 Device Statistics

Attribute	Value
Index	0x1020
Name	Device Statistics
Object Code	RECORD
Category	Optional
Max Subindex	3 (4...255 reserved for future)

Table 41: Entries of object 0x1020 Device Statistics

SubIndex	Description	M/O/C	Access	Data Type	Value
0	Number of entries	M	R	UINT 8	
1	Time since power on	O	R	UDINT	This is the time the device has been currently powered on, in seconds, regardless of communication presence.
2	Total time powered	O	R	UDINT	This is the cumulative amount of time the device has been powered on, in seconds. It shall be stored automatically to non-volatile memory.
3	Number of power cycles	O	R	UDINT	Number of power cycles

NOTE 1: Time counts by 1 sec

NOTE 2: Time count display of Total time powered und Time since power on in CoE object dictionary increments every second.

NOTE 3: Interval for saving to non-volatile memory depends on memory technology and its maximum number of write cycles, e.g., 1 000 000 write cycles give a write interval of 15 minutes.

12.5 Object 0x10E2: Manufacturer-specific Identification Code

Unique identification of a device can be useful for a manufacturer for various reasons, e.g. for traceability purposes. The device vendors often use manufacturer-specific identification codes. Those can be stored in the SII EEPROM (Category Type 01-09 are device specific, see ETG.2010, [7]). For simple access via CoE a representation in the object dictionary is also helpful.

The Manufacturer-specific Identification Code object is defined in Table 42, the entry description is given in Table 43.

Table 42: 0x10E2 Manufacturer-specific Identification Code

Attribute	Value
Index	0x10E2
Name	Manufacturer-specific Identification Code
Object Code	ARRAY
Data Type	VISIBLE_STRING
Category	Optional

Table 43: Entries of object 0x10E2 Manufacturer-specific Identification Code

SubIndex	Description	M/O/C	Access	Data Type	Value
0	Maximum supported subindex	M	R	UINT 8	
1	Manufacturer-specific identification code	M	R (RW)	VISIBLE_STRING	1 st manufacturer-specific identification code
2	Manufacturer-specific identification code	O	R (RW)	VISIBLE_STRING	2 nd manufacturer-specific identification code if the device consists of modules with individual identification codes
...					
254	Manufacturer-specific identification code	O	R (RW)	VISIBLE_STRING	254 th manufacturer-specific identification code if the device consists of modules with individual identification codes

NOTE: The object entries can be implemented as writeable. It is recommended to protect the write access with a password, e.g. by writing a code to another index first.

13 Change of Mapping Objects

Some objects contain mapping information that might be variable. For example

- 0x1600...0x17FF Receive PDO Mapping
- 0x1A00...0x1BFF Transmit PDO Mapping
- 0x1C10...0x1C2F Sync manager Channel PDO Assignment
- 0xF01x, 0xF02x, 0xF03x defined in ETG.5001 [8].

To change the content of these objects the following procedure shall be done, if the entries are changed separately, i.e. without CompleteAccess:

- 1) set SubIndex 0 = 0
The object is considered disabled when SubIndex 0 has the value 0.
- 2) configure the mapping information in the mapping entries SubIndex 1 ... n
- 3) set SubIndex 0 = number of used mapping entries

A write access to the SubIndex 1...n while SubIndex 0 does not have the value 0 should be handled in the following way:

- if value = stored value:
write access is allowed if object is not readonly.
This can be used to check the device configuration against the MainDevice configuration
- if value <> stored value:
Abort write access with Abort Code 0x06 01 00 03 "Subindex cannot be written, SI0 must be 0 for write access"

With CompleteAccess the whole object can be written at once (if not readonly)

The write-access to the entries might be restricted to a state, e.g. PreOp, of the EtherCAT State Machine.

14 SDO Services

14.1 SDO access single entry with flexible length

Entries with flexible length, as specified in Table 120, may have shorter length than given as the maximum size of the entry.

For entries with data type `VISIBLE_STRING` the following applies:

Write access

- MainDevice can send a `VISIBLE_STRING` without 0-terminator:
SubDevice shall add the 0-terminator if the written `VISIBLE_STRING` is shorter than the maximum entry size
- MainDevice can also write `VISIBLE_STRING` with 0-terminator

Read access:

- When the MainDevice reads a `VISIBLE_STRING` the SubDevice should return the `VISIBLE_STRING` with its full length given in the mailbox header data field "Length", i.e. including padding "0"s if the string itself is shorter .
- Bytes following the 0-terminator shall be zero.

14.2 SDO Complete Access

With the Complete Access all SubIndexes of an object are uploaded or downloaded with a single SDO service. Complete Access should be supported to minimize boot-up time of the device.

For the data of the object there are the following rules for the correct alignment:

1. SubIndex 0 is padded to 16 bit
2. SubIndexes of bit data type (`BOOLEAN` and `BIT1` to `BIT7`) will be filled over the byte border, the next non-bit data type will start at the next `BYTE` address
e.g. object `0x10F3` `SI4 = BOOL`, `SI5 = UINT16` → Bit-Offset of `SI5 = 48` (not 41)
3. To define gaps in the byte stream of a complete access, it is allowed to define gap entries (`Data_Type = 0`, `Bit_Length = gap size in bits`)
The value of gaps shall be 0.
4. SubIndexes that don't exist will not need space
5. The Complete Access can start with SubIndex 0 or SubIndex 1, other subindexes are not allowed
6. If the SubDevice supports complete access all objects which fit in the mailbox should be accessible by SDO with complete access
7. If the SubDevice supports Complete Access and Segmented SDO Transfer all objects should be accessible by SDO with Complete Access
8. Objects with dynamic entries (i.e. "`OCTET_STRING`", "`UNICODE_STRING`", "`ARRAY_OF_*`" or "`VISIBLE_STRING`") cannot be accessed by Complete access.
9. Objects that cannot be accessed shall return the error code `0x06010000` (Unsupported Access) or `0x6010004` (Complete access not supported).
10. For `ENUM` objects complete access may not be supported
11. The length of the responded Complete Access data shall either match to the current byte length of the object or to the Bit Size of the object description.
12. The current maximum subindex may exceed the number of entry descriptions of the offline object dictionary.
13. The SDO Download Complete Access data length shall always match the full current object size (defined by SubIndex0).

Entries of data type string must have a fixed length in the entry description to be used for complete access. The string itself can be shorter. Unused Bytes shall be filled with 0.

Example: EntryDescription Length = 240 Byte, actual String value = "Test" = 5 Byte with 0-Termination.

Even when the SubDevice supports complete access there still may be objects that cannot be accessed via complete access. For descriptions about complete access in ESI, see ETG2000.

- If Object/Flags@SdoAccess=CompleteAccess in ESI, the device shall support complete access to the object.

An example of SDO Complete Access starting with SubIndex 1 is shown in Table 44.

Table 44: SDO Complete Access Example with SubIndex 1

SubIndex	Data Type	Bit Offset
1	BOOLEAN	0
2	BIT2	1
3	BIT3	3
4	0 = Gap, Length = 2	6
10	UNSIGNED8	8
11	UNSIGNED16	16
12	UNSIGNED32	32
13	UNSIGNED8	64
14	OCTET-STRING[4]	72
15	VISIBLE-STRING[7]	104
16	INTEGER32	160
17	UNSIGNED64	192

An Example of SDO Complete Access starting with SubIndex 0 is shown in Table 45.

Table 45: SDO Complete Access Example with SubIndex 0

SubIndex	Data Type	Bit Offset
0	UNSIGNED8	0
1	BOOLEAN	16
2	BIT2	17
3	BIT3	19
4	0 = Gap, Length = 2	22
10	UNSIGNED8	24
11	UNSIGNED16	32
12	UNSIGNED32	48
13	UNSIGNED8	80
14	OCTET-STRING[4]	88
15	VISIBLE-STRING[7]	120
16	INTEGER32	176
17	UNSIGNED64	208

For a download CompleteAccess to an object with RW and RO entries, the RO objects shall ignore the write access. Download data should be Zero for the RO entries.

14.3 SDO Information services

SDO Information services are defined in ETG.1000.6, [5].

Some objects support a higher maximum subindex than it is given in subindex 0, e.g. PDO assign object, Diagnosis object.

In SDO Information Service “Get Entry Description Response” the ValueInfo“, „Data Type“, „Bit Length“ and “Object Access” shall be 0 for those objects.

An SDO Upload/Download service shall be aborted with SDO Abort Code “Subindex not exist” (0x06090011) for those objects.

An SDO Information request to an Index <0x1000 is not allowed and shall be aborted with Abort code “Unsupported Access” (0x06010000).

14.4 SDO Upload with max response size

A MainDevice application uses the SDO Upload with Complete Access to read information from a SubDevice’s object dictionary. The object can be enhanced in a future version of the SubDevice (SI4, SI5 in the example). Enhanced means, that there are additional entries at the end of the object.

The MainDevice application can either be enhanced as well, or the MainDevice application should be able to limit the size of data that should be read.

Therefore the SDO Upload service should be enhanced with a Response Buffer Size information, that the MainDevice can set in the SDO Upload Request. The SubDevice that supports this feature shall write the amount of data that is requested by the MainDevice (if access is possible), even if the object contains more data.

The SDO Upload Expedited Request service is defined in ETG.1000.6. The request is extended with an additional information “Response Buffer Size” of type DWORD as defined in Table 46.

Table 46: SDO Upload Expedited Request (extension)

Frame part	Data Field	Data Type	Value/Description
Mailbox Header	Length	WORD	0x0A: Length of the Mailbox Service Data
	Address	WORD	Station Address of the source, if a MainDevice is client, Station Address of the destination, if a SubDevice is client
	Channel	Unsigned6	0x00 (Reserved for future)
	Priority	Unsigned2	0x00: lowest priority ... 0x03: highest priority
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	Counter of the mailbox services (0 reserved, 1 is start value, next value after 7 is 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x02: SDO Request
SDO	Reserved	Unsigned4	0x00
	Complete Access	Unsigned1	0x00: entry addressed with index and subindex will be uploaded 0x01: complete object will be uploaded, subindex shall be zero (subindex 0 included) or one (subindex 0 excluded)
	Command Specifier	Unsigned3	0x02: Upload Request
	Index	WORD	Index of the Object
	Subindex	BYTE	Subindex of the Object, shall be zero or one, if Complete Access = 0x01
	Response Buffer Size	DWORD	> 0x00: Maximum length of the response object data. According "Complete Size" field in the response data.

To limit the size of the SDO response the MainDevice shall set the SDO upload response field "Response Buffer Size" in the SDO Upload Expedited Request service. In case that the value is greater 0 the object response data shall not exceed the specified value.

NOTE: The feature support is indicated by the ESI Flag
["EtherCATInfo/Descriptions/Devices/Device/Mailbox/CoE/@SdoUploadWithMaxLength"](#).

In case that the specified buffer size is insufficient one of the following abort codes should be returned by the SubDevice:

- 0x06 07 00 10 Data type does not match, length of service parameter does not match
- 0x06 07 00 12 Data type does not match, length of service parameter too high
- 0x06 07 00 13 Data type does not match, length of service parameter too low

15 ENUM Definition

15.1 ENUM rules

For the textual description of the allowed values of an entry, the data type of the entry description shall be a device specific data type enumeration definition in the object area 0x0800-0x0FFF. The data type enumeration definition itself is defined as a record of the ENUM type (0x28) which is read-only in the object dictionary.

Values (Byte0-3) shall be sorted in ascending order.

Table 47: ENUM Definition – Textual description of allowed values

SubIndex	Description	Data type	Value
0	Number of entries	UNSIGNED8	n
1	Allowed Value 1	Octet String	Byte 0-3: Value 1 Byte 4-n1: textual description of Value 1
2	Allowed Value 3	Octet String	Byte 0-3: Value 2 Byte 4-n2: textual description of Value 2
...			
n	Allowed Value n	Octet String	Byte 0-3: Value n Byte 4-nn: textual description of Value n

NOTE: Byte 0-3 define the values in the range 2^{32} . It is not the subindex number. Up to 255 enumeration values can be defined per object

In the EtherCAT SubDevice Information (ESI) file the ENUM definition is included in the Data Type definition of the Dictionary.

The SDO Upload service for a Data Type Index used for ENUMs is conditional (mandatory if ENUM is used in an Entry-Description). The SDO-Info service is optional for this Data Type object.

An ENUM may also be used instead of a specified entry data type. In that case the bit size shall remain as specified.

Use case: if a data type is defined for an object/entry, a device manufacturer can use an ENUM instead of the defined data type, to provide a list of possible/allowed values.

15.2 ENUM used as an entry of an ARRAY data type

In an ARRAY object all entries shall have the same base data type. However, for some ARRAY object entries, only discrete values are allowed to be assigned to an entry. To show the possible values, ENUMs may be used for individual ARRAY object entries in the offline object dictionary (ESI file). But the structure for an ARRAY object in the ESI file cannot be used if one or more entries are (different) ENUMs and other entries have a different base data type.

In this case a RECORD data type should be used instead of an ARRAY, even if the object is defined as an ARRAY in the specification.

Use Case 1: To take advantage of using ENUMs for individual entries for the drive profile objects defined as ARRAY.

16 Diagnosis Handling

16.1 General

This clause describes a unique handling for diagnosis data in an EtherCAT SubDevice. The Diagnosis Handling supports:

- a diagnosis history with up to 250 diagnosis messages
- different types of diagnosis messages: Info, Warning, Error
- status bit “New message available” which can be mapped to the cyclic data
- a Text ID for each message as a reference to a detailed diagnosis description stored in the ESI file
- two operation modes: Overwrite Mode and Acknowledge Mode

The Diagnosis History Object is described by 0x10F3.

16.2 Object 0x10F3: Diagnosis History Object

There should be a history of diagnosis messages in the EtherCAT SubDevice that an EtherCAT MainDevice can always access the latest diagnosis messages. With the Diagnosis History object 0x10F3 up to 250 diagnosis messages can be accessed.

The object is defined in Table 48, the entry description is given in Table 49.

Table 48: 0x10F3 Diagnosis History Object

Attribute	Value
Index	0x10F3
Name	Diagnosis History
Object Code	RECORD
Max SubIndex	6-255

Table 49: Entries of 0x10F3 Diagnosis History Object

Sub-Index	Description	Data Type	Access	PDO Mapping	M/O/C	Description / Default value
1	Maximum Messages	UNSIGNED8	R or RW	no	M	Read: Number of diagnosis messages which can be stored in the diagnosis history (SubIndex 6 onwards) Write: For devices with dynamic memory the number of stored diagnosis messages may be adapted by the EtherCAT MainDevice
2	Newest Message	UNSIGNED8	R	no	M	SubIndex of the newest diagnosis message (6-255) Default value = 0
3	Newest Acknowledged Message	UNSIGNED8	R(W)* *Write support is optional	no	M	Default value = 0 <u>Overwrite Mode</u> (SI5, Bit4 = 0): Read = 0: When the message queue will be overwritten, the SubDevice shall set SI3 to 0 Writing = 0: (support optional) the SubDevice will clear all messages, i.e. resetting SI2, SI3, SI4 and SI5 Bit 5 Writing = 1...5: The SubDevice shall return SDO-Abort with codes 0x06090030 (value range of parameter exceeded) or 0x06090032 (value of parameter written too low) Writing = 6...[SI0] ¹ : SI3 = Written value without checking Writing > [SI0]...255: SDO-Abort with codes 0x06090030 or 0x06090031(value of parameter written too high) <u>Acknowledge Mode</u> (SI5, Bit4 =1): Read = 0: No messages have been acknowledge so far Read <> 0: SubIndex of latest acknowledged diagnosis message (6-255) Writing = 0: (support optional) All acknowledged messages will be deleted. Writing = 1...5: The SubDevice shall return SDO-Abort with codes 0x06090030 (value range of parameter exceeded) or 0x06090032 (value of parameter written too low) Writing = 6...[SI0]: Messages are acknowledged Writing > [SI0]...255: SDO-Abort with codes 0x06090030 or 0x06090031(value of parameter written too high)

¹ [SI0] means content of subindex 0

Sub-Index	Description	Data Type	Access	PDO Mapping	M/O/C	Description / Default value
4	New Messages Available	BOOLEAN	R	(TxPDO)	<u>M</u>	<u>Overwrite Mode:</u> 0: newest message was read 1: newest message was not read <u>Acknowledge Mode:</u> 0: no unacknowledged message 1: diagnosis messages are available which can be acknowledged (SI2 <> SI3)

Sub-Index	Description	Data Type	Access	PDO Mapping	M/O/C	Description / Default value
5	Flags	UNSIGNED16	R(W)*	no	M	Flags to control the sending and storing of diagnosis messages Which functionality the SubDevice supports is described by the ESI / SII. When writing SI5, the read-only bits should match the current values. Bit5 shall be "don't care". The SubDevice should send an abort with 0x6090030 Value exceeded in case the readonly bits differ from current values
						Bit 0: *Write support is optional Enable Emergency sending (according to ETG.1000-6) 0: new diagnosis messages are not sent as Emergency message (default, if device does not support Emergency sending) 1: new diagnosis messages shall be sent as Emergency message
						Bit 1: *Write support is mandatory Disable info messages 0: Info messages are stored in the diagnosis message queue (default) 1: Info messages will not be stored in the diagnosis message queue
						Bit 2 *Write support is mandatory disable warning messages 0: Warning messages are stored in the diagnosis message queue (default) 1: Warning messages will not be stored in the diagnosis message queue
						Bit 3: *Write support is optional disable error messages 0: Error messages are stored in the message queue (default) 1: Error messages will not be stored in the diagnosis message queue
						Bit 4: *Write support is optional Mode selection for diagnosis history handling 0: Overwrite Mode old messages are overwritten by new ones when buffer is full 1: Acknowledge Mode New messages do only overwrite messages which were acknowledged before

Sub-Index	Description	Data Type	Access	PDO Mapping	M/O/C	Description / Default value	
						Bit 5: *readonly	Overwrite/Discard Information In Overwrite Mode: 1: unread messages have been overwritten (=buffer overrun) (SI3 is set to 0, too) In Acknowledge Mode: 1: message buffer is full with unacknowledged messages and a new message is discarded.
						Bit 6-15	reserved for future use
6-255	Diagnosis message	OCTET-STRING	R	no	SI6 M SI 7-255 O	Diagnosis message buffer Depending on SI1 the EtherCAT SubDevice can store up to 250 diagnosis messages; the first message is stored in SubIndex 6, the second in SubIndex 7 and so on. When the queue is full, the EtherCAT SubDevice shall overwrite SubIndex 6 and so on, that always the latest maximum messages (SI1) shall be accessible by the EtherCAT MainDevice	

16.3 Diagnosis Message

The Diagnosis Message shall contain the parameters shown in Table 50.

Table 50: Diagnosis Message

Parameter	Data type	USE	Description	
Diag Code	UNSIGNED32	M	Diagnosis code to identify the diagnosis message	
			Bit 0-15 = 0x0000-0xDFFF	not used
			Bit 0-15 = 0xE000-0xE7FF	Bit 16-31: can be used manufacturer specific
			Bit 0-15 = 0xE800	Bit 16-31: Emergency Error Code as defined in ETG.1000.6
			Bit 0-15 = 0xE801	reserved
			Bit 0-15 = 0xE802	module related messages (incl. module number assignment) Bit 16-23: Slot No, 1 = 1 st Slot Bit 24-31: 0x00
			Bit 0-15 = 0xE803	Device related messages in a modular device Bit 16-31: 0x0000
			Bit 0-15 = 0xE804-0xEDFF	reserved for future standardization
			Bit 0-15 = 0xEE00-0xEFFF	Bit 16-31: Profile specific
			Bit 0-15 = 0xF000-0xFFFF	not used
Flags	UNSIGNED16	M	Bit 0-3	Diag type:
				0 Info message
				1 Warning message
				2 Error message
				3-15 reserved for future standardization

Parameter	Data type	USE	Description	
			Bit 4	Time Stamp is a local time stamp, the global time stamp can be calculated by reading the actual local time stamp (object 0x10F8) and calculating the age of the message
			Bit 5	Time Stamp is a global, yet not DC based timestamp (based on Time Distribution object 0x10F9, see 16.7)
			Bit 6-7	reserved for future standardization
			Bit 8-15	Number of parameters in this Diagnosis Message
Text ID	UNSIGNED16	M	Text ID as reference to Diagnosis text as defined in the ESI file	
			0	no Text ID
			1-65535	Text ID reference to ESI file Text ID shall be unique in combination with VendorID, Product Code, Revision Number
Time Stamp	UNSIGNED64	M	Time Stamp in ns. Time derived from DC unit or local time stamp according to Bit 4 and Bit 5 of entry Flags	
			0	no time stamp
			<> 0	time stamp
Flags Parameter 1	UNSIGNED16	O	Describes the type of Parameter 1	
			Bit 12-15 = 0	Bit 0-11 = Data type Index of the data type of parameter 1 0x0001: BOOLEAN 0x0002: INTEGER8 0x0003: INTEGER16 0x0004: INTEGER32 0x0005: UNSIGNED8 0x0006: UNSIGNED16 0x0007: UNSIGNED32 0x0008 : REAL32 0x0011 : REAL64 0x0015 : INTEGER64 0x001B : UNSIGNED64 The corresponding text parameters and formatting are specified in the ETG.2000
			Bit 12-15 = 1	Bit 0-11 = size of BYTE-Array in bytes, parameter 1 is a BYTE-Array shall be used for following Data Type: OCTET_STRING (index: 0x00A), specifier %s
			Bit 12-15 = 2	Bit 0-11 = size of ASCII-String (without ending 0) in bytes, parameter 1 is an ASCII-string shall be used for following Data Type: VISIBLE_STRING (index: 0x009), specifier %s
			Bit 12-15 = 3	Bit 0-11 = size of Uni-Code-String in bytes, parameter 1 is an Uni-Code-string shall be used for following Data Type: UNICODE_STRING (index: 0x00B), specifier %s
			Bit 12-15 = 4	Bit 0-11 = 2 (size of parameter 1 in bytes), parameter 1 number of TextID as referenced in ESI specifier %s
			Bit 12-15 = 5-15	reserved for future standardization

Parameter	Data type	USE	Description
Parameter 1	depend on Flags Parameter 1	C	value of parameter 1 (Mandatory if Flags Parameter 1 exist)
Flags Parameter 2	UNSIGNED16	O	see flags parameter 1
...			

16.4 Diagnosis History Operation Modes

The Diagnosis Message buffer (0x10F3:6-255) works like a ring buffer. This ring buffer can be handled in Overwrite Mode or Acknowledge Mode.

The mode can be read/ written via 0x10F3:SI5:Bit4.

16.4.1 Overwrite Mode

In Overwrite Mode new messages overwrite older messages once the buffer is full even when they were not acknowledged before.

16.4.1.1 Acknowledging Messages in Overwrite Mode

When writing the Newest Acknowledged Message (0x10F3:SI3) all messages up to the age of the message which is in the written SubIndex are acknowledged. The SubDevice does not check if those messages had been read before.

Indication:

0x10F3:SI5:BIT5 = 0

The SubDevice shall return SDO-Abort with codes 0x06090030 (value range of parameter exceeded) or 0x06090031 (value of parameter written too high) in the following case:

- If SI3 is written with a value of a Subindex which does not hold a message

16.4.1.2 Clearing Messages in Overwrite Mode

Write 0 to Newest Acknowledged Message (0x10F3:SI3) will delete all messages even if the messages were not acknowledged or read before.

Indication:

0x10F3:SI2 = 0; 0x10F3:SI3 = 0; 0x10F3:SI4 = 0 and 0x10F3:SI5:BIT5 = 0

16.4.2 Acknowledge Mode

Messages which have not been acknowledged are not overwritten within the ring buffer. New messages are not stored and go lost.

16.4.2.1 Acknowledging Messages in Acknowledge Mode

When writing the Newest Acknowledged Message (0x10F3:SI3) all messages up to the age of the message which is in the written SubIndex are acknowledged. The SubDevice does not check if those messages had been read before.

Indication:

0x10F3:SI5:BIT5 = 0

The SubDevice shall return SDO-Abort with codes 0x06090030 (value range of parameter exceeded) or 0x06090031 (value of parameter written too high) in the following case:

- If SI3 is written with a value of a Subindex which does not hold a message

The following examples with writing different values to the Acknowledge (SI3) show the behaviour.

SI	history	status		status
6	MESSAGE 12	read	write SI3 = 8	acknowledge
7	MESSAGE 13	read		acknowledge
8	MESSAGE 14	read		acknowledge
9	MESSAGE 15	read		
10	MESSAGE 16	read		
11	MESSAGE 17			
12	MESSAGE 18			
13	NEWEST 19			
14	MESSAGE 09	read		acknowledge
15	MESSAGE 10	read		acknowledge
16	MESSAGE 11	read		acknowledge

Figure 6: Acknowledge Mode: Example 1

SI	history	status		status
6	MESSAGE 12	read	write SI3 = 12	acknowledge
7	MESSAGE 13	read		acknowledge
8	MESSAGE 14	read		acknowledge
9	MESSAGE 15	read		acknowledge
10	MESSAGE 16	read		acknowledge
11	MESSAGE 17			acknowledge
12	MESSAGE 18			acknowledge
13	NEWEST 19			
14	MESSAGE 09	read		acknowledge
15	MESSAGE 10	read		acknowledge
16	MESSAGE 11	read		acknowledge

Figure 7: Acknowledge Mode: Example 2

16.4.2.2 Clearing Message History in Acknowledge Mode

To gain a better overview within the diagnosis message history all messages which have been acknowledged before can be deleted from the message history by writing "0" to the Acknowledge (SI3). Messages which had not been acknowledged before are shifted to the beginning of the message history queue.

Figure 8 shows an example.

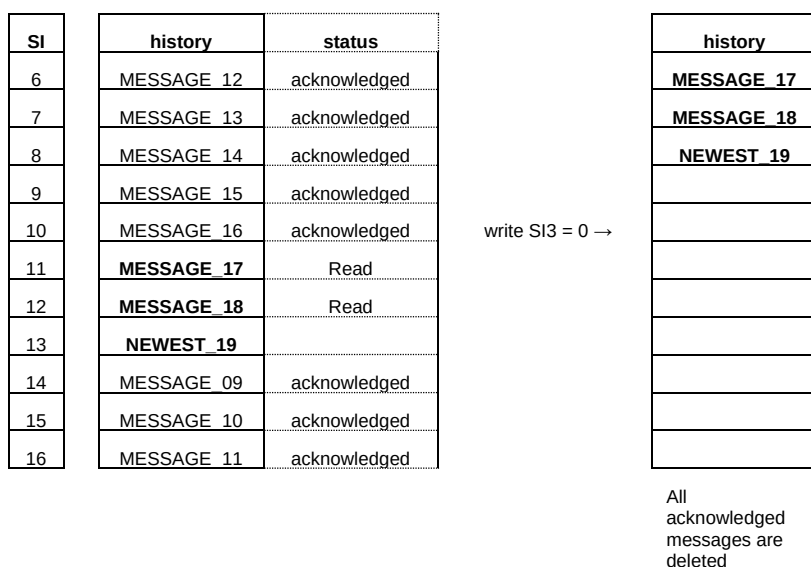


Figure 8: Clearing Message History

16.5 Object 0x10FA: Diagnosis History Management object

The Diagnosis History Management object, defined in Table 51, is intended for additional management functions for the Diagnosis History object 0x10F3.

It can be used to acknowledge only specific diagnosis history messages.

Table 51: 0x10FA Diagnosis History Management object

Attribute	Value
Index	0x10FA
Name	Diagnosis History Management
Object Code	RECORD
Max SubIndex	1 (2.. 255 reserved for future)
Category	Optional
PDO Mapping	No

The Diagnosis History Management entries are defined by Table 52.

Table 52: Entries of 0x10FA Diagnosis History Management Object

Sub-Index	Description	Data Type	Access	M/O/C	Description / Default value
1	Slot Ack	UNSIGNED16	RW	O	All diagnosis messages reported by the module assigned to the Slot n are acknowledged. Default Value: 0
2...255	Res	--	--	--	Reserved for future enhancements

If 0x10FA subindex 1 is supported and the value is unequal zero, only the diagnosis messages reported by the module assigned to the corresponding slot shall be acknowledged. If the subindex 1 is zero, no slot specific filtering is active. The acknowledge of the messages is triggered with the mechanisms described in 14.2. The MainDevice writes the entry 0x10F3:03 "Newest Acknowledged Message". The value of the entry is checked to filter which message(s) shall be acknowledged. When the SubDevice has acknowledged the diagnosis message(s) the SubDevice shall reset the subindex 1 to zero. The MainDevice can use this as a feedback information.

Subindex 2-255 are reserved for future enhancements.

16.6 Object 0x10F8: Timestamp Object

The Timestamp object holds the current local Time of the SubDevice

The Timestamp object is defined in Table 53.

Table 53: 0x10F8 Timestamp Object

Attribute	Value
Index	0x10F8
Name	Timestamp Object
Object Code	VAR
DataType	UNSIGNED64
Category	optional
Access	ro or rw
PDO Mapping	(TxPDO)
Value	Local Timestamp of the device in ns

The time source used for object 0x10F8 is arbitrary and individual for each SubDevice. Only differences between values of object 0x10F8 from the same SubDevice have an absolute meaning.

If the SubDevice is set to DC synchronous mode, the time source for object 0x10F8 may be the local copy of the DC System Time. If a device supports only 32-bit DC register set, the Timestamp object can be writeable. The MainDevice can write the global 64-Bit DC-time. This time is used by the SubDevice to update the higher 32-Bit of the object. The MainDevice shall only write to the object, if Bit 31 of the DC time is 0, to prevent that the MainDevice writes in the moment of an overflow of the 32-Bit register.

The SubDevice increments the higher 32-bit with every overflow of the 32-Bit DC-Time register; the lower 32-Bits represents the value of the DC unit.

NOTE: When the SubDevice boots, it does not yet have a DC time, but may already be generating Diagnosis messages. After DC initialisation, the time may jump and the Diagnosis History will be inconsistent.

NOTE: If bit4=true in the Diagnosis message, the MainDevice should read the 0x10F8 of each SubDevice to calculate the corresponding global time of the message. Only if Bit4 and Bit5 = FALSE the MainDevice can expect the DC time to be used for the Diagnosis message.

16.7 Object 0x10F9: Time Distribution Object

The Time Distribution Object is an element of the CoE Communication Area of the CoE Object Dictionary.

The Time Distribution Object provides the means for a system-wide (“global”) timestamp, without requiring that the SubDevice supports Distributed Clocks (DC). It is used when the DC accuracy is not required, e.g. for the message time stamp of the Diagnosis History Object (0x10F3).

MainDevice (application) writes an absolute time reference via CoE SDO Download to the entry 0x10F9:01. The SubDevice adjusts its internal time base accordingly.

The Time Distribution object is defined in Table 54.

Table 54: 0x10F9 Time Distribution Object

Attribute	Value
Index	0x10F9
Name	Time Distribution Object
Object Code	RECORD
Max SubIndex	1
Category	Optional
PDO Mapping	No

The Distributed Time Value is defined by Table 55.

Table 55: Entries of 0x10F9 Time Distribution Object

Sub-Index	Description	Data Type	Access	M/O/C	Description / Default value
1	Distributed Time Value	UNSIGNED64	RW	M	Absolute time distributed by the MainDevice via SDO Download in ns. Default Value: 0

17 PDO Parameter Description

17.1 General

The PDO Parameter object contains information about the PDOs. Each PDO Mapping object (0x1600-0x17FF, 0x1A00-0x1BFF) is related to a PDO Parameter object (0x1400-0x15FF, 0x1800-0x19FF). The subindexes 1-5 of the PDO Parameter object are reserved to keep compatibility to CANopen.

17.2 RxPDO Parameter

Table 56 defines the RxPDO Parameter objects. Table 57 describes the object entries.

Table 56: RxPDO Parameter Objects

Attribute	Value
Index	0x1400...0x15FF
Name	RxPDO Parameter
Max SubIndex	9
Category	Optional
Access	Read Write
PDO Mapping	Yes

Table 57: RxPDO Parameter

SubIndex	Description	Data Type	Description
6	RxPDO Exclude RxPDOs	OCTET_STRING	Includes the Indexes of the RxPDO Mapping object Indexes which could not be assigned together with this RxPDO
7	RxPDO State*	BOOLEAN	Shall be set (TRUE) from the SubDevice if the output data of this RxPDO could not be put to the hardware, mappable in a TxPDO
8	RxPDO Control	BOOLEAN	Shall be set (TRUE) from the MainDevice if the output data of this RxPDO do not have valid values, mappable in a RxPDO
9	RxPDO Toggle* Optional: OutputUpdateCounter	BOOLEAN Optional: BIT2	Shall toggle/increment with every update of the corresponding RxPDO (written by the MainDevice and mappable in a RxPDO)
10	RxPDO Exclude SM	UNSIGNED32	If Bit n = 1, the RxPDO shall not be mappable into SyncManager n.
11	RxPDO Mapping Changed	UNSIGNED16	Incremented each time SIO of the RxPDO Mapping object is changed from 0 to a value > 0 Mappable in TxPDO.

* RxPDO State and RxPDO Toggle/OutputUpdateCounter might be defined module specific in a corresponding device profile, such as ETG.5001 [8] . If such a profile is used, the profile specific entries should be mapped to the process data.

17.3 TxPDO Parameter

Table 58 defines the TxPDO Parameter objects. Table 59 describes the object entries.

Table 58: TxPDO Parameter Objects

Attribute	Value
Index	0x1800...0x19FF
Name	TxPDO Parameter
Max SubIndex	9
Category	Optional
Access	Read Write
PDO Mapping	optional

Table 59: TxPDO Parameter

SubIndex	Description	Data Type	Description
6	TxPDO Exclude TxPDOs	OCTET_STRING	Includes the Indexes of the TxPDO Mapping object Indexes which could not be assigned together with this TxPDO
7	TxPDO State*	BOOLEAN	Shall be set (TRUE) from the SubDevice if the input data of this TxPDO is not valid, mappable in a TxPDO Should be used, if the input data from the application are not valid, e.g. because a measurement hardware is not connected properly or it can break. The SubDevice stays in OP.
9	TxPDO Toggle* Optional: InputUpdateCounter	BOOLEAN Optional: Bit2	Shall toggle/increment with every update of the corresponding TxPDO (written by the SubDevice and mappable in a TxPDO)
10	TxPDO Exclude SM	UNSIGNED32	If Bit n = 1, the TxPDO shall not be mappable into SyncManager n.
11	TxPDO Mapping Changed	UNSIGNED16	Incremented each time SIO of the TxPDO Mapping object is changed from 0 to a value > 0 Mappable in TxPDO.

* TxPDO State and TxPDO Toggle/InputUpdateCounter might be defined module specific in a corresponding device profile, such as ETG.5001 [8] . If such a profile is used, the profile specific entries should be mapped to the process data.

17.4 PDO Mapping entries larger than 31 Byte

Since the bit length of a mapping entry is restricted to 255 bit (UINT8), PDO Mapping entries >31 Byte shall be separated into several parts with maximum 30 Byte (240 Bit) length. The first PDO Mapping entry shall contain the entries' index and subindex information with a bit size of 240. Following parts shall be used as padding entries with index = 0x0000 and subindex = 0x00 and with bit size of 240, expect the last block with a bit size <240.

To distinguish between padding entries used for such kind of large mapping entries and alignment information (also using index = 0x0000 and subindex = 0x00) the configuration tool shall evaluate the data type of the PDO.

Following rules shall be fulfilled:

- Bitsize=240 and DataType:Bitlength <= 240
 - all following 0000:00:yy are alignment information
- Bitsize=248 and DataType:Bitlength = 248
 - all following 0000:00:yy are alignment information

3. Bitsize=240 and DataType:Bitlength > 240

- Pdo entry needs padding information
All padding Pdo mapping entries must be "0000:00:240",
except the last Pdo entry may be "0000:00:nn" where nn should be <=240

4. The PDO entry bit size and data type may be smaller than the definition in the OD.

Examples:

32 byte entry (0x7000:01) + no alignment + 2 byte entry (0x7000:02):

0x1600.1	0x7000: 01: 240	Data type ARRAY[0.. 31] OF BYTE
0x1600.2	0x0000: 00: 16	last two Bytes
0x1600.3	0x7000: 02: 16	Data type e.g. UINT

30 byte entry (0x7000:01) + 2 byte alignment + 2 byte-entry (0x7000:02):

0x1600.1	0x7000: 01: 240	Data type ARRAY[0.. 29] OF BYTE
0x1600.2	0x0000: 00: 16	2 Byte alignment
0x1600.3	0x7000: 02: 16	Data type e.g. UINT

32 byte entry (0x7000:01) + 2 byte alignment + 2 byte entry (0x7000:02):

0x1600.1	0x7000: 01: 240	Data type ARRAY[0.. 31] OF BYTE
0x1600.2	0x0000: 00: 16	last two bytes
0x1600.3	0x0000: 00: 16	2 Byte alignment
0x1600.4	0x7000: 02: 16	Data type e.g. UINT

Map only 30 byte of a 40 byte entry (ARRAY[0..**39**] OF BYTE) + 2 byte alignment + 2 byte entry (0x7000:02):

0x1600.1	0x7000: 01: 240	Data type ARRAY[0.. 29] OF BYTE
0x1600.2	0x0000: 00: 16	2 Byte alignment
0x1600.3	0x7000: 02: 16	Data type e.g. UINT

60 Byte-entry (fits exactly in two 240 bit entry) (0x7000:01) + no alignment + 2 Byte entry (0x7000:02)

0x1600.1	0x7000:01:240	Data type ARRAY[0.. 59] OF BYTE
0x1600.2	0x0000:00:240	last 30 Byte
0x1600.3	0x7000:02:16	Data type e.g. UINT

60 Byte-entry (fits exactly in two 240 bit entry) (0x7000:01) + alignment + 2 Byte entry (0x7000:02)

0x1600.1	0x7000:01:240	Data type ARRAY[0.. 59] OF BYTE
0x1600.2	0x0000:00:240	last 30 Byte
0x1600.3	0x0000:00:16	2 Byte alignment
0x1600.4	0x7000:02:16	Data type e.g. UINT

The configuration tool always has to take care of the corresponding data type.

17.5 PDO Mapping attribute of object definitions

EtherCAT specifications ETG.1000, ETG.1020 and several EtherCAT profile specifications define the PDO Mapping options of CoE object entries into PDOs. The attribute "PDO Mapping" is used to

distinguish between mandatory, optional and not intended mapping of a CoE object entry. Allowed values are described in the Table 60.

Table 60: Meaning of attribute Rx/TxPDO mapping

Value	Meaning
Not defined	Entry is not intended to be mapped to PDOs ⁽¹⁾
No ⁽²⁾	Entry is not intended to be mapped to PDOs ⁽¹⁾
Optional ⁽³⁾	Mapping to TxPDO and/or RxPDO optional
TxPDO	Mapping to TxPDO mandatory and not intended to RxPDO
RxPDO	Mapping to RxPDO mandatory and not intended to TxPDO
(TxPDO)	Mapping to TxPDO optional and not intended to RxPDO
(RxPDO)	Mapping to RxPDO optional and not intended to TxPDO
RxPDO/(TxPDO)	Mapping to RxPDO mandatory and to TxPDO optional
(RxPDO)/TxPDO	Mapping to RxPDO optional and to TxPDO mandatory

NOTES

⁽¹⁾ however, mapping the entry does not violate the specification.

⁽²⁾ attribute value “No” is used in some specifications with the same meaning as “not defined”.

⁽³⁾ attribute values “possible” and “yes” are used in some specifications with the same meaning as “optional”.

18 FoE Extension

18.1 FoE read and write clarification

If the last FoE Data service primitive includes data that exactly fits into the Mailbox, indicated by the parameter Size = FullSize, then an empty Data service primitive must be used to terminate the FoE Write or FoE Read service.

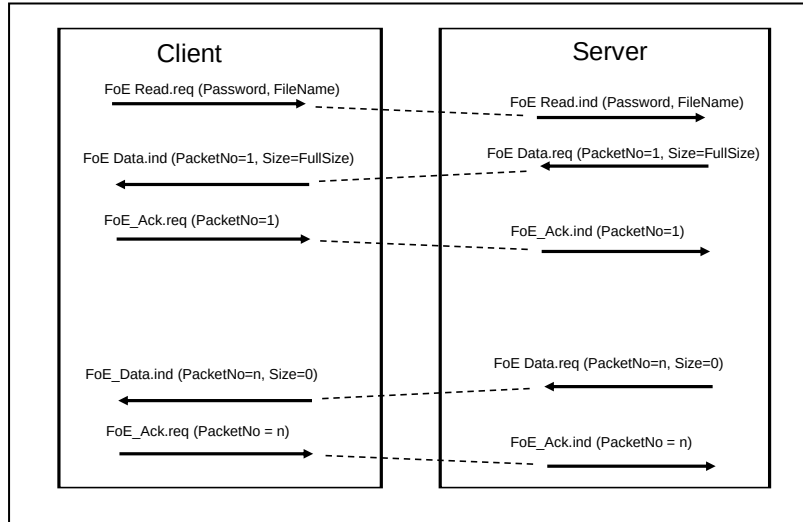


Figure 9 – FoE read sequence with success

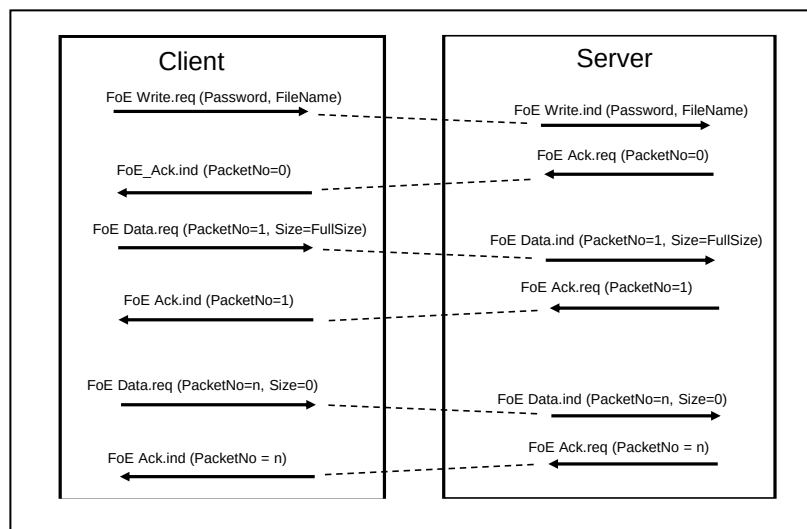


Figure 10 – FoE write sequence with success

The last FoE Data indication with Size < FullSize is used to indicate the end of the FoE Service.

18.2 FoE read sequence with busy

FoE services are defined in ETG.1000.5, corresponding protocols in ETG.1000.6.

For an FoE read request with Busy the sequence shown in Figure 11 shall be used.

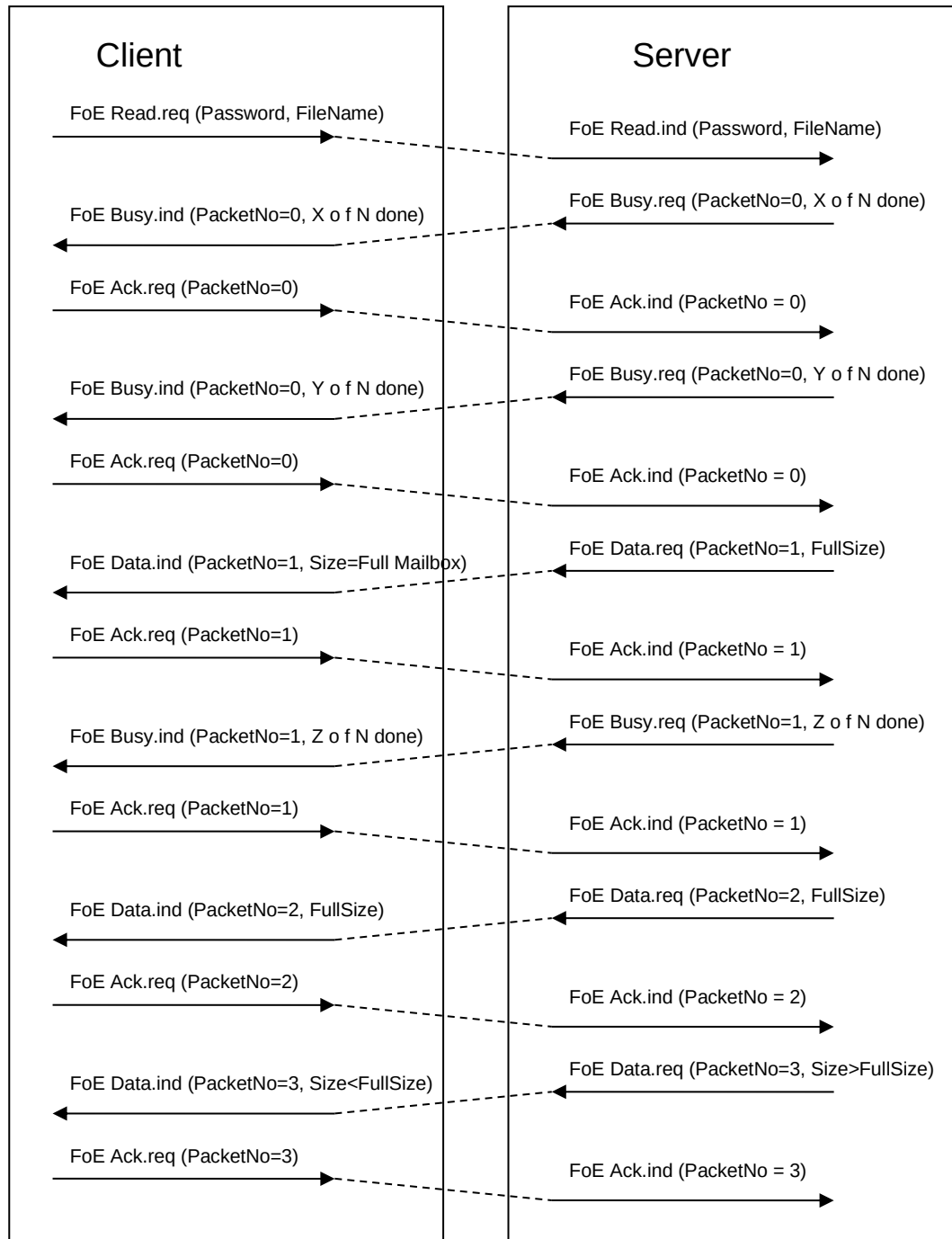


Figure 11: FoE read sequence with busy

If the last data segment of the file fits exactly into the mailbox an additional FoE Data.req (PacketNo=3, Size=0) without data must be sent. The last FoE Data must include a Size < FullSize.

A FoE_Write request with Busy shall always be handled as defined in ETG.1000.5. That means that the SubDevice shall always send the FoE_ACK.req as response to the FoE_Write.ind. Even if the internal procedure needs additional time to check the Write request from the MainDevice. After FoE_Data.ind the SubDevice can send FoE_Busy.req or FoE_Error.req in case of an error (e.g. if the file is not known).

18.3 FoE error Codes

FoE Error Codes are defined in ETG.1000.6, [5].

The FoE error codes are derived from TFTP error codes. Several implementations use the TFTP error codes without the Offset of 0x8000. Both codes can be used arbitrarily.

Table 61 shows already existing error codes and additional error codes that should be used for FoE.

Table 61: Error codes for FoE

Error code	Meaning	Description	Reference
0x8000 or 0x0000	Not defined	See error message (if any)	ETG.1000.6
0x8001 or 0x0001	Not found	The file requested by an FoE upload service could not be found on the server	ETG.1000.6
0x8002 or 0x0002	Access denied	Read or write access to this file not allowed (e.g. due to local control).	ETG.1000.6
0x8003 or 0x0003	Disk full	Disk to store file is full or memory allocation exceeded	ETG.1000.6
0x8004 or 0x0004	Illegal	Illegal FoE operation, e.g. service identifier invalid	ETG.1000.6
0x8005 or 0x0005	Packet number wrong	FoE packet number invalid	ETG.1000.6
0x8006 or 0x0006	Already exists	The file which is requested to be downloaded does already exist	ETG.1000.6
0x8007 or 0x0007	No user	No user	ETG.1000.6
0x8008 or 0x0008	Bootstrap only	FoE only supported in Bootstrap	ETG.1000.6
0x8009 or 0x0009	Not Bootstrap	This file may not be accessed in BOOTSTRAP state	ETG.1000.6
0x800A or 0x000A	No rights	Password invalid	ETG.1000.6
0x800B or 0x000B	Program Error	Generic programming error. Should only be returned if error reason cannot be distinguished	ETG.1000.6
0x800C or 0x000C	Checksum wrong	A checksum included in the file is invalid	Additional code
0x800D or 0x000D	Firmware does not fit for Hardware	The hardware does not support the downloaded firmware	Additional code
0x800E or 0x000E	reserved	--	Additional code
0x800F or 0x000F	No file to read	Do not use (identical with 0x8001)	Additional code
0x8010 or 0x0010	File header does not exist	Missing file header of error in file header	Additional code
0x8011 or 0x0011	Flash problem	Flash cannot be accessed	Additional code
0x8012 or 0x0012	File incompatible	Do not use (identical with 0x800D)	Additional code

19 EoE Extension

19.1 General

EoE Timestamp information can be used for synchronization protocols like IEEE 1588.

SubDevice → MainDevice: Time Stamp contains exact receive time

MainDevice → SubDevice: Time Stamp contains desired send time

19.2 Get IP Parameter

With this service the IP parameter could be read, the service response is similar to the Set IP parameter request as shown in Table 62.

Table 62: EoE Get IP Parameter service request

Frame part	Data Field	Data Type	Description
Mailbox Header	Length	WORD	Length of Mailbox Service Data = 4
	Address	WORD	Station Address of the source, if a MainDevice is client, Station Address of the destination, if a SubDevice is client
	Channel	Unsigned6	Reserved for future use
	Priority	Unsigned2	Reserved for future use
	Type	Unsigned4	EoE = 2
	Counter	Unsigned3	Counter for Mailbox Data Link layer
	Reserved	Unsigned1	Reserved for future use
	Frame Type	Unsigned4	0x06
EoE Header	Port	Unsigned4	0x00: send to no specific port 0x01-0x0F: specific ports selected
	Last Fragment	Unsigned1	0x01: last Data part
	Time Appended	Unsigned1	0x00: no time stamp value will be appended
	Time Request	Unsigned1	0x00: no time stamp value of the send time requested
	Reserved	Unsigned5	reserved for future use
	Fragment number	Unsigned6	0x00
	Offset	Unsigned6	0x00
	Frame number number	Unsigned4	0x00

The Set IP parameter response service is shown in Table 63.

Table 63: EoE Get IP Parameter service response

Frame part	Data Field	Data Type	Description
Mailbox Header	Length	WORD	Length of Mailbox Service Data >= 10
	Address	WORD	Station Address of the source, if a MainDevice is client, Station Address of the destination, if a SubDevice is client
	Channel	Unsigned6	Reserved for future use
	Priority	Unsigned2	Reserved for future use
	Type	Unsigned4	EoE = 2
	Counter	Unsigned3	Counter for Mailbox Data Link layer
	Reserved	Unsigned1	Reserved for future use
EoE Header	Frame Type	Unsigned4	0x07
	Port	Unsigned4	0x00: send to no specific port 0x01-0x0F: specific ports selected
	Last Fragment	Unsigned1	0x01: last Data part
	Time Appended	Unsigned1	0x00: no time stamp value will be appended
	Time Request	Unsigned1	0x00: no time stamp value of the send time requested
	Reserved	Unsigned5	reserved for future use
	Fragment number	Unsigned6	0x00
	Offset	Unsigned6	0x00
	Frame number	Unsigned4	0x00
Get IP parameter Header	MAC included	Unsigned1	MAC address according to ISO/IEC 8802-3
	IP address included	Unsigned1	IP address according to IETF RFC 791
	Subnet Mask included	Unsigned1	Subnet mask according to IETF RFC 791
	Default Gateway included	Unsigned1	Default Gateway address according to IETF RFC 791
	DNS Server IP Address included	Unsigned1	IP address of DNS server according to IETF RFC 1035
	DNS Name included	Unsigned1	DNS name according to IETF RFC 1035
	reserved	Unsigned26	reserved for future use
(conditional)	MAC	BYTE[6]	MAC address according to ISO/IEC 8802-3
(conditional)	IP address	BYTE[4]	IP address according to IETF RFC 791
(conditional)	Subnet Mask	BYTE[4]	Subnet mask according to IETF RFC 791
(conditional)	Default Gateway	BYTE[4]	Default Gateway address according to IETF RFC 791
(conditional)	DNS Server IP Address	BYTE[4]	IP address of DNS server according to IETF RFC 1035
(conditional)	DNS Name	char[32]	DNS name according to IETF RFC 1035

19.3 Get Address Filter

With this service the Address Filter can be read, the service response is similar to the Set Address Filter request. Table 64 defines the service request and Table 65 the service response.

Table 64: EoE Get Address Filter service request

Frame part	Data Field	Data Type	Description
Mailbox Header	Length	WORD	Length of Mailbox Service Data = 4
	Address	WORD	Station Address of the source, if a MainDevice is client, Station Address of the destination, if a SubDevice is client
	Channel	Unsigned6	Reserved for future use
	Priority	Unsigned2	Reserved for future use
	Type	Unsigned4	EoE = 2
	Counter	Unsigned3	Counter for Mailbox Data Link layer
	Reserved	Unsigned1	Reserved for future use
EoE Header	Frame Type	Unsigned4	0x08
	Port	Unsigned4	0x00: send to no specific port 0x01-0x0F: specific ports selected
	Last Fragment	Unsigned1	0x01: last Data part
	Time Appended	Unsigned1	0x00: no time stamp value will be appended
	Time Request	Unsigned1	0x00: no time stamp value of the send time requested
	Reserved	Unsigned5	reserved for future use
	Fragment number	Unsigned6	0x00
	Offset	Unsigned6	0x00
	Frame number	Unsigned4	0x00

Table 65: EoE Get Address Filter service response

Frame part	Data Field	Data Type	Description
Mailbox Header	Length	WORD	Length of Mailbox Service Data >= 6
	Address	WORD	Station Address of the source, if a MainDevice is client, Station Address of the destination, if a SubDevice is client
	Channel	Unsigned6	Reserved for future use
	Priority	Unsigned2	Reserved for future use
	Type	Unsigned4	EoE = 2
	Counter	Unsigned3	Counter for Mailbox Data Link layer
	Reserved	Unsigned1	Reserved for future use
EoE Header	Frame Type	Unsigned4	0x09
	Port	Unsigned4	0x00: send to no specific port 0x01-0x0F: specific ports selected
	Last Fragment	Unsigned1	0x01: last Data part
	Time Appended	Unsigned1	0x00: no time stamp value will be appended
	Time Request	Unsigned1	0x00: no time stamp value of the send time requested
	Reserved	Unsigned5	reserved for future use
	Fragment number	Unsigned6	0x00
	Offset	Unsigned6	0x00
	Frame number	Unsigned4	0x00
Get Address Filter Header	MAC filter count	Unsigned4	Count of MAC address according to ISO/IEC 8802-3 which are accepted by Ethernet Ports on this SubDevice
	MAC filter mask	Unsigned2	Count of MAC address masks according to ISO/IEC 8802-3 which are combined with filter by Ethernet Ports on this SubDevice
	Reserved	Unsigned1	reserved for future use
	Inhibit Broadcast	Unsigned1	Filter Broadcast messages
	Reserved	Unsigned8	reserved for future use
(conditional, used if MAC filter count <> 0)	List of MAC Address	List of BYTE [6]	MAC address according to ISO/IEC 8802-3
(conditional, used if MAC filter mask <> 0)	List of MAC Address Filter	List of BYTE [6]	MAC address according to ISO/IEC 8802-3 A set bit means that this address bit of Destination MAC Address is compared with the corresponding entry in the List of MAC Address.

20 VoE Coding

20.1 PDU structure

The general attribute types of VoE are described in Figure 12.

```
typedef struct
{
    unsigned32      VendorID;
    unsigned16      VendorType;
} TVOEHEADER;

typedef struct
{
    TMBXHEADER      MbxHeader;
    TVOEHEADER      VoeHeader;
    BYTE            Data[MBX_DATA_SIZE-2];
} TVOEMBX;
```

Figure 12: VoE general structure

The VoE coding is specified in Table 66.

Table 66: VoE elements

Frame part	Data Field	Data Type	Value/Description
Mailbox Header	Length	WORD	Length of the Mailbox Service Data
	Address	WORD	Station Address of the source, if a MainDevice is client, Station Address of the destination, if a SubDevice is client
	Channel	Unsigned6	0x00 (Reserved for future)
	Priority	Unsigned2	0x00: lowest priority ... 0x03: highest priority
	Type	Unsigned4	0x0F: VoE
	Cnt	Unsigned3	Counter of the mailbox services (0 reserved, 1 is start value, next value after 7 is 1)
	Reserved	Unsigned1	0x00
VoE Header	VendorID	Unsigned32	Vendor ID
	Vendor Type	Unsigned16	Vendor specific Protocol Type
	Data		Protocol specific data

21 Explicit Device Identification

21.1 Intention

The use of EtherCAT Device identification is to identify an EtherCAT SubDevice explicitly. This is necessary for the following use cases:

- **Hot Connect applications**
Within some applications it might be useful to connect or disconnect parts of the network. In this case the MainDevice must have the possibility to identify which part of the network is available.
- **Prevention against cable swapping**
If at least two identical devices are used in one application it might be necessary to prevent the mix-up of these devices by cable swapping.
Example Scenario: Within a machining center there might be two identical drives to work in X and Y direction. To avoid that the drives receive wrong process data, for example after a device replacement, an explicit identification of the devices can be used.

The Device Identification value can be used optionally for unique addressing.

21.2 Terms and Definitions

Device Identification Value

Explicit Device Identification value that is set locally on the device

ID-Selector

Local non-volatile selector to set the Device Identification Value. The ID-Selector shall exclusively be used to set the Device Identification Value.

SII

SubDevice Information Interface – content of EEPROM

SII Configured Station Alias

Parameter Configured Station Alias stored in the SII and loaded during boot-up to register 0x0012 of the ESC

Configured Station Alias Register – referred to as Register 0x0012

Register 0x0012 in the ESC that is loaded from EEPROM during boot-up of the ESC

21.3 Device Identification Value

The Device Identification value should be set uniquely within a network configuration.

The following properties shall be fulfilled by the Device Identification value:

- 0 shall be the default value of the SII Configured Station Alias
- The Device Identification Value shall have a length of 2 Byte

The Device Identification value can be set

- by an **ID-Selector**, locally on the SubDevice
This can be a Switch (e.g. Rotary- or DIP-switch) or set via a display control stored non-volatile. The ID-Selector should have a length of 16 Bit.
- by a configuration tool in the SII Configured Station Alias
The value of the SII is automatically loaded during boot-up of the ESC to the Configured Station Alias Register (0x0012).

21.4 SubDevice implementation

Three different device types for setting the Device ID are defined in this specification:

- Complex SubDevice with application microcontroller and ID-Selector
→ Requesting ID mechanism
- Simple SubDevice w/o microcontroller and with ID-Selector
→ Direct ID mechanism
- Set Device Identification by Configuration Tool
 - o SubDevice w/o ID-Selector
 - o Remote settable Device ID

ETG.2000 defines the elements “IdentificationAdo” and “IdentificationReg134” to distinguish between Requesting ID and Direct ID mechanism.

21.4.1 Requesting ID – Complex SubDevice with application microcontroller and ID-Selector

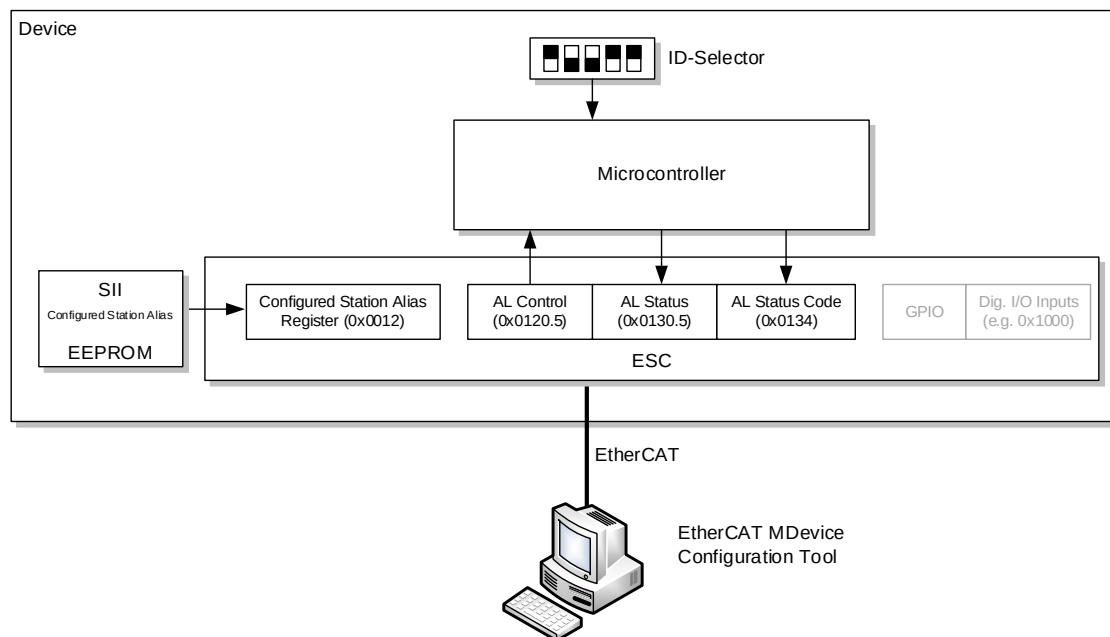


Figure 13: Complex SubDevice with application microcontroller

Handling of Requesting ID mechanism within EtherCAT state machine (ESM) is specified in ETG.1000.5 and ETG.1000.6. Following clauses are for description only.

A complex SubDevice with application Microcontroller and ID-Selector should use AL Status Code register 0x0134 for Device Identification value.

The EtherCAT MainDevice can request the Device Identification value by setting Bit 5 of AL Control register (0x0120.5 = *ID Request*). The SubDevice shall load the current Device Identification Value to the AL Status Code register and set Bit 5 of AL Status register (0x0130.5 = *ID loaded*) to indicate that its Device Identification Value is loaded. A changed Device Identification value, e.g. because the ID-Selector is set to a different value, shall be loaded only with a new ID Request by the EtherCAT MainDevice.

Figure 14 describes a timing example of an ID request by the MainDevice.

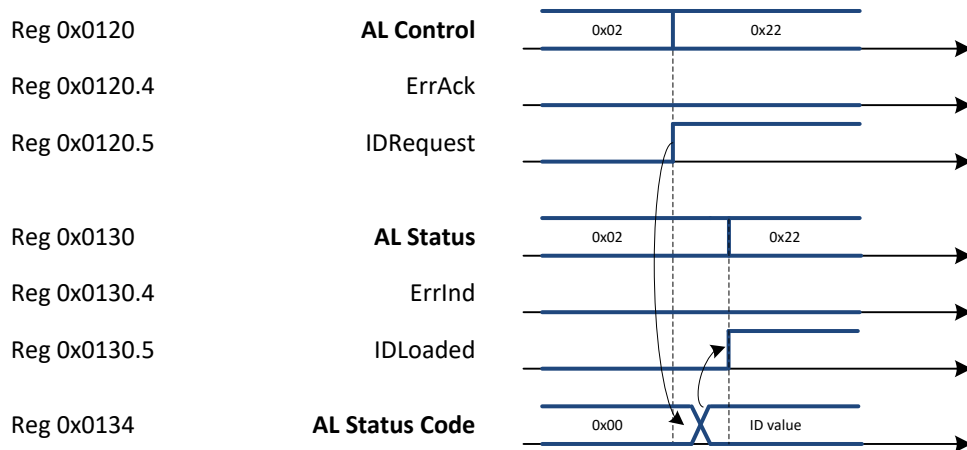


Figure 14: Timing Example: Request Device ID

If the SubDevice has to indicate an Error at the same time at which the ID Value is requested the SubDevice shall reset *ID Loaded* Flag (0x0130.5), load the *Error code* to the AL Status Code register and set the *Error Indication* Flag (0x0130.4), see Figure 15.

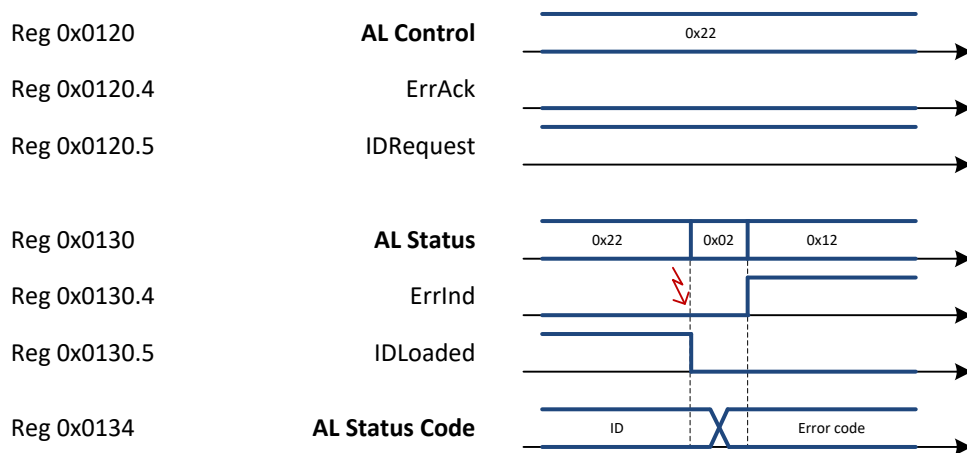


Figure 15: Timing Example: Error Indication while Device ID loaded

If the SubDevice has an active *Error Indication* but the Error is not pending and the MainDevice sends an *Error Acknowledge* and *ID Request* at the same time, the SubDevice shall reset the *Error Indication* Flag and confirm noError, then load the Device Identification Value in the AL Status Code register and set *ID Loaded* again, see Figure 16.

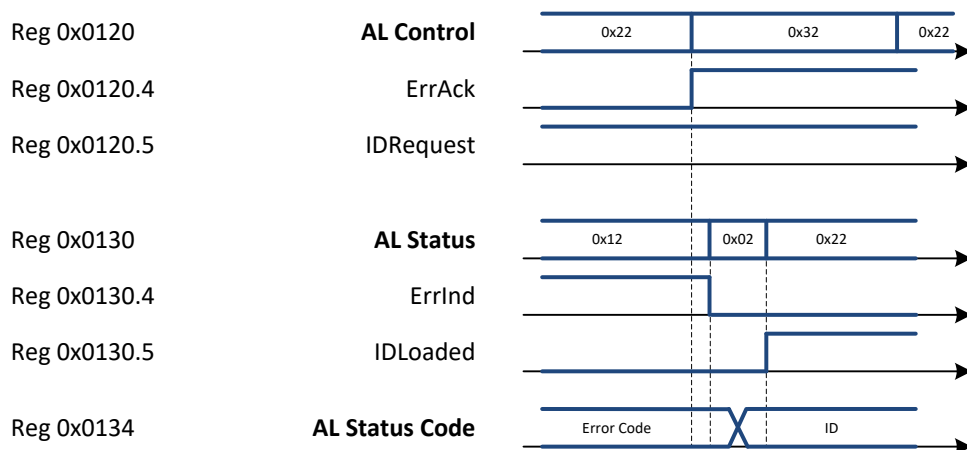


Figure 16: Timing Example: Error Acknowledge with Device ID Request

If the device supports the Requesting ID mechanism it shall support this mechanism in all states except Boot. A request of the Device ID in Boot state shall be ignored.

Timeouts for the Requesting ID mechanism are the same as used for ESM transitions.

For simple identification process The MainDevice can use following command sequence (for example as part of the ENI file) to identify the SubDevice:

- Write register 0x0120 = 0x0021 (Bit 0 → Init; Bit 5 → Load Device ID)
- Poll registers 0x0130...0x0135 and mask to compare with expected Device ID.
MainDevice shall use one datagram for data consistency to read register 0x0130 and 0x0134.
- Write register 0x0120 = 0x0001 (Bit 0 → Init)

NOTE: The Requesting ID mechanism might be ignored by the MainDevice / Configuration Tool and Configured Station Alias may be used alternatively. The ID-Selector should be zero in that case.

NOTE: Configured Station Alias Register 0x0012 should be used for a complex SubDevice with microcontroller only for compatibility reasons, since this register is also loaded during power-on from SII Configured Station Alias. This might lead to a race condition.

If a SubDevice uses Register 0x0012 for ID-Selector values, following functionality shall be used:

- the host controller reads the ID value from the ID switch immediately after start-up (when powered-on). The ID value for the legacy mechanism is only updated after start-up.
- If ID-Selector value == 0 nothing shall be done (value in register 0x0012 is loaded from SII, independent if "0" or different value)
- If ID-Selector value != 0 AND SII value == 0: value from ID-Selector shall be copied to register 0x0012
- If ID-Selector value != 0 and SII value != 0: a race condition failure procedure shall be started:
 - o copy ID-Selector value to register 0x0012
 - o set internal error flag (ERRint = 1)
 - o overwrite SII value with "0" during state change from Init to PreOp (SII access to PDI)
 - o depending on further MainDevice action:
 - a) if state change to SafeOp is requested: reject state change (AI Status = 0x12) and set error "Device Identification value updated" (AI Status Code = 0x0061)
 - b) if state Init is requested OR power cycle OR successful SDO write to Reload object: delete internal error, proceed normal

21.4.2 Direct ID – Simple SubDevice w/o microcontroller and with ID-Selector

The ID-Selector can be connected directly to I/O Inputs of the ESC. There is no need for a microcontroller.

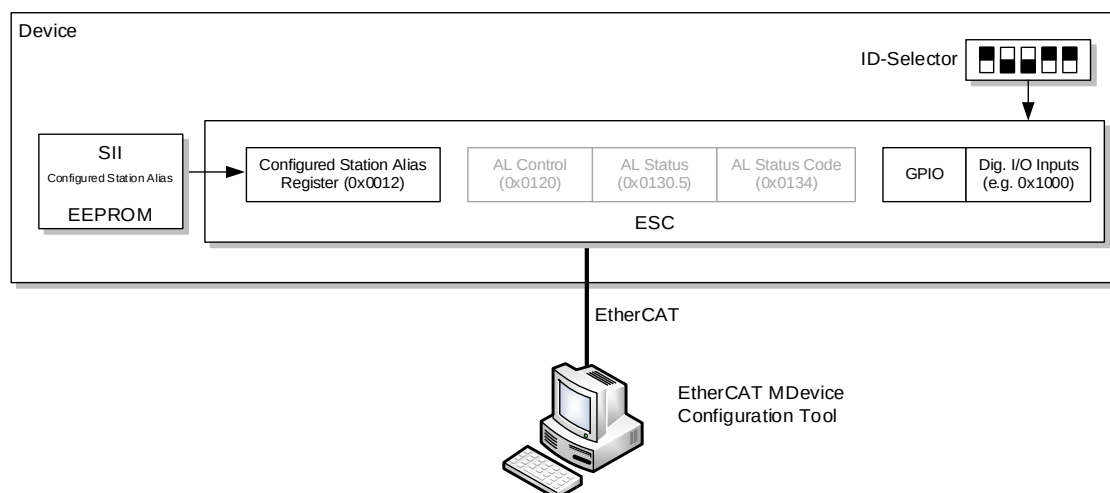


Figure 17: Simple SubDevice w/o microcontroller

The ID-Selector value is copied to the address given in the IdentificationAdo ESI /SII element and can directly be read by the MainDevice (Direct ID mechanism). A change of the ID-Selector is directly copied to the Device Identification value.

The local address of the Device Identification Value shall be within one of the following memory ranges in the ESC:

- General Purpose Inputs (e.g. 0x0F18 ... 0x0F1F)
- Digital I/O Input Data (PDI Configuration: Digital I/O Interface, 0x1000...0x1003)

NOTE: availability of memory ranges depends on ESC type

NOTE: Do not use the User RAM area (0x0F80...0x0FFF) or Process Data RAM (0x1000...0xFFFF, if not initialized by hardware on power-up) since RAM comes up with undefined values after boot-up.

NOTE: The Direct ID mechanism might be ignored by the MainDevice / Configuration Tool and Configured Station Alias may be used alternatively. The ID-Selector should be zero in that case.

21.4.3 Set Device Identification by Configuration Tool

If no local ID-Selector is available or the ID-Selector value is Zero the configuration tool can use Register 0x0012 for explicit device identification.

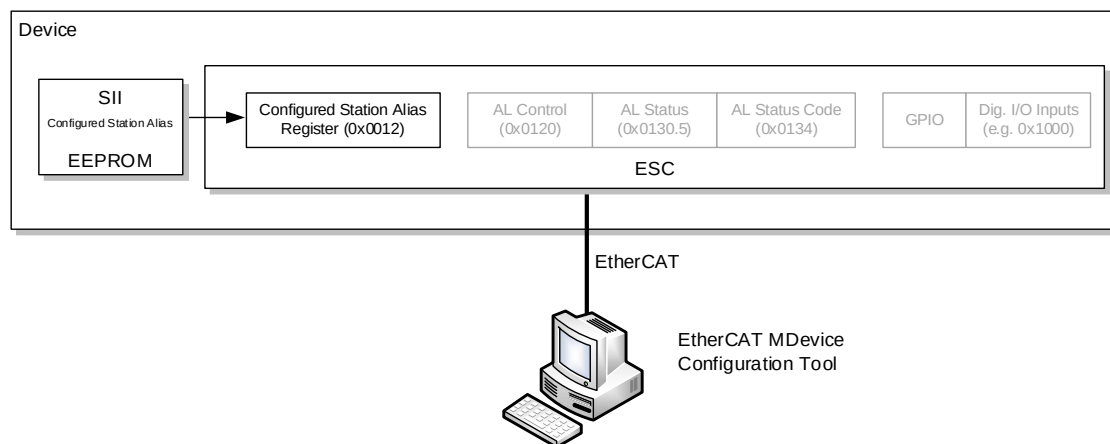


Figure 18: Set Device Identification by Configuration Tool

The configuration tool must write the SII Configured Station Alias and the device needs a power cycle to load the new value to the Register 0x0012.

NOTE: a reload command to the ESC does not reload the Register 0x0012

To load Register 0x0012 remotely by the MainDevice the device should implement the Device Identification Reload Object, see chapter 21.6. This avoids the power cycle after change of Configured Station Alias.

21.5 MainDevice behaviour

21.5.1 Configuration Tool

The configuration tool should use the ESI /SII elements IdentificationAdo/IdentificationReg134 for configuration.

If no such information is available the Configured Station Alias Register 0x0012 can be used for Explicit Device Identification.

21.5.2 MainDevice

Before reading the device ID the MainDevice should identify the SubDevice, to determine which Identification mechanism shall be used. For example:

- read VendorID and ProductCode first
- then read Explicit Device ID

21.6 Object 0x10E0: Device Identification Reload Object

The Device Identification Reload object should be used to explicitly reload registers in the EtherCAT SubDevice Controller that are available from the PDI.

The Device Identification Reload object is defined in Table 67 the entry description is given in Table 68.

Table 67: 0x10E0 Device Identification Reload Object

Attribute	Value
Index	0x10E0
Name	Device Identification Reload Object
Object Code	RECORD
Max SubIndex	3

Table 68: Entries of 0x10E0 Device Identification Reload Object

SubIndex	Description	Data Type	Value
0	Maximum supported Subindex	UINT 8	
1	Configured Station Alias register value	UINT16	Write: write value into register 0x0012 Read: read current value of register 0x0012
2	Write Configured Station Alias persistent	BOOL	FALSE: Write access to SI 1 will write value to 0x0012 only TRUE: Write access to SI 1 will write value to 0x0012 and to SII
3	Reload ID-selector value	UINT16	Write: writing 0x0000 to the subindex updates the current ID-selector value into register 0x0012 Read: read current value of ID-selector

The subindex 1 “Configured Station Alias register value” offers the possibility to explicitly reload the register 0x0012. Write access might be restricted to PreOp state.

The purpose of this entry is to handle following use case:

- Remote change of Devcie Identification value in Configured Station Alias register (0x0012) without power-cycle of device

NOTE: A “Reload EEPROM” command to the ESC register 0x0502 is not sufficient, since all registers except register 0x0012 and 0x0140.9 are reloaded.

If subindex 2 “Write Configured Station Alias persistent” is supported and set to TRUE, the SubDevice shall also write the “Configured Station Alias” in the SII. The SubDevice needs access to the EEPROM, therefore it shall set the Flag “AssignToPdi” in the ESI/SII.

Write access to Index 3 shall only be supported if the device uses Register 0x0012 for explicit device identification with ID-Selector.

22 Error Handling

The motivation of a standard EtherCAT Error Handling is a unique behaviour of EtherCAT devices (MainDevices and SubDevices) in cases of communication problems.

In the following there will be a systematic classification of:

- Error Reasons
- Error Detection
- Error Reaction
- Restart Behaviour

22.1 Error Reasons

There are the following SubDevice specific Error Reasons:

- Inputs or part of Inputs are invalid (e.g. I/O-Error)
- Inputs are not updated
- No communication of the Application Controller with the ESC
- Application Controller restarted
- Cycle Time exceeded
- SubDevice powered off
- SubDevice disconnected
- Inputs are OK, but Outputs cannot be set (e.g. I/O-Error)

The following MainDevice specific Error Reasons can be distinguished:

- Outputs or part of Outputs are invalid (e.g. PLC-Stop)
- Outputs are not updated (e.g. Application exception)
- MainDevice Restart
- Timing Error (e.g. sending of cyclic datagram was not finished when the next cycle started)
- MainDevice disconnected
- MainDevice powered off
- SubDevice Timing Error
- Distributed Clocks were not started correctly (e.g. System Time Offset is not written)

The following bus specific Error Reasons can be distinguished:

- Single bus faults
- Multiple bus faults
- Bus disconnected

22.2 Error Detection

22.2.1 Mechanism

22.2.1.1 EtherCAT State

The MainDevice should check the actual EtherCAT state of the SubDevice by reading the SubDevice's AL Status Register (0x0130). If the read value does not match with the expected value(s) an error is detected.

The SubDevice shall check the requested EtherCAT state from the MainDevice by reading the SubDevice's AL Control Register (0x0120). If the read value does not match with the expected value(s) an error is detected.

Table 69: AL Status Codes used for Synchronization

AL Status Code	Description
0x1A	Multiple synchronization errors. Device is not synchronized any more (shall be used if the following causes cannot be distinguished)
0x2C	Fatal Sync Error: Sync0 or Sync1 are not received any more
0x2D	Sync not received: In SAFEOP the SubDevice waits for the first Sync0/Sync1 events before switching to OP, if these events were not received during the SAFEOP to OP-Timeout time the SubDevice should refuse the state transition to OP with this AL Status Code
0x32	Multiple synchronization errors: PLL is not synchronized any more
0x33	Multiple synchronization errors. IO is not synchronized any more
0x34	Multiple synchronization errors. Too much SM Events missed

22.2.1.1.1 SM Change

An error is detected when the SM Change Event is received and as consequence of the event the SM-Settings are not correct (e.g. SyncManager was disabled).

22.2.1.1.2 Watchdog

The MainDevice should initialize the SubDevice's watchdog before switching the SubDevice to the OPERATIONAL state (usually done in transition Init-to-PreOp) by writing the SubDevice's watchdog registers (0x400 and 0x420, the default values represent a 100 ms watchdog). If the watchdog expires an error is detected.

The watchdog will be retriggered every time the SM2-Event is received (SM3-Event if the output size is 0).

22.2.1.1.3 PDO State

If the SubDevice supports to send the TxPDO State (Index 0x1800+PDO Number-1, SubIndex 7, Data Type BOOLEAN) in a TxPDO (in that case the TxPDO State shall be included in the PDO Mapping of this TxPDO). The MainDevice should check the TxPDO State. If the TxPDO State changes to TRUE (set by the SubDevice), the data of the related TxPDO is invalid and an error is detected.

If the SubDevice supports to send the RxPDO State (Index 0x1400+PDO Number-1, SubIndex 7, Data Type BOOLEAN) in a TxPDO (in that case the RxPDO State shall be included in the PDO Mapping of this TxPDO). The MainDevice should check the RxPDO State. If the RxPDO State changes to TRUE (set by the SubDevice), the data of the related RxPDO could not be put to the hardware and an error is detected.

22.2.1.1.4 PDO Control

If the SubDevice supports to receive the RxPDO Control (Index 0x1400+PDO Number-1, SubIndex 8, Data Type BOOLEAN) in a RxPDO (in that case the RxPDO Control shall be included in the PDO Mapping of this RxPDO) the SubDevice should check the RxPDO Control. If the RxPDO Control changes to TRUE (set by the MainDevice), the data of the related RxPDO is invalid and an error is detected.

22.2.1.1.5 PDO Toggle

If the SubDevice supports to send the TxPDO Toggle (Index 0x1800+PDO Number-1, SubIndex 9, Data Type BOOLEAN) in a TxPDO (in that case the TxPDO Toggle shall be included in the PDO Mapping of this TxPDO) the MainDevice should check the TxPDO Toggle. If the TxPDO Toggle does not toggle with every sync cycle (toggled by the SubDevice), the data of the related TxPDO is old and an error is detected.

If the SubDevice supports to receive the RxPDO Toggle (Index 0x1400+PDO Number-1, SubIndex 9, Data Type BOOLEAN) in a RxPDO (in that case the RxPDO Toggle shall be included in the PDO

Mapping of this RxPDO) the SubDevice should check the RxPDO Toggle. If the RxPDO Toggle does not toggle (done by the MainDevice), the data of the related RxPDO is old and an error is detected.

A Profile may support a counter similar to the PDO Toggle (1-Bit Counter) which could be used instead of the PDO Toggle if MainDevice and SubDevice support this Profile.

22.2.1.1.6 Synchronization Monitoring

4. In SM-synchronous mode an error is detected if the SM-Event is received again before the local cycle is finished.
5. In SM-synchronous mode an error is detected if the SM2-event jitters too much and the application cannot synchronize itself to the SM2 Event any more.
6. In DC Mode an error is detected if the Sync0 event is received and no SM2 Event was received before.
7. In DC Mode an error is detected if the Sync1 event is received or the Output Shift Time is expired before the Output CalcAndCopy is finished.
8. In DC Mode an error is detected if the Sync0 is not received within the Sync0 Cycle Time.
9. In DC Mode an error is detected if the Sync1 is not received within the Sync1 Cycle Time.
10. In DC Mode an error is detected if the Sync0 and/or Sync1 event is never detected

22.2.1.1.7 Working Counter

The MainDevice should check the Working Counter of a received EtherCAT datagram. If the Working Counter does not match with the expected value an error is detected.

22.2.1.1.8 Lost SubDevices

The MainDevice may check the number of connected SubDevices with a BRD datagram for example. If the number of counted SubDevices does not match with the expected value an error is detected.

22.2.1.1.9 Lost Frames

The MainDevice may use the Index (IDX) of the EtherCAT datagram header to check if all sent EtherCAT datagrams will be received. If EtherCAT datagrams are lost an error is detected.

22.2.1.1.10 Local Error

The MainDevice and SubDevice may detect a local error like a stop of the application or a fault in the connected hardware.

22.2.1.1.11 Error Counter in the SubDevice

An error might be detected by the MainDevice if an error counter of the SubDevice increments.

22.2.1.1.12 Sync Error

The MainDevice may detect a synchronization error in the SubDevice when the Sync Error Flag (0x1C33:20) is TRUE.

22.2.2 Object 0x10F1: Error Settings

Table 70 defines the Error Settings object used for Error Reaction behaviour of the SubDevice. Table 71 describes object entries.

Table 70: 0x10F1 Error Settings Object

Attribute	Value
Index	0x10F1
Name	Error Settings
Max SubIndex	2
Category	Optional
Access	Read Write
PDO Mapping	No

Table 71: 0x10F1 Error Settings

SubIndex	Description	Data Type	Access	Description / Default value
1	Local Error Reaction	UNSIGNED32	R or RW	0 = PDO State SubDevice changes the value of the PDO State Bit (either in the PDO Parameter or if it is part of the process data description then there. SubDevice remains in OP 1 = Disable SyncManager (outputs) from PDI side and go to ErrSafeOp 2 = Device specific state Input PDOs contains a device specific value, e.g. profile/device specific error code
2	Sync Error Counter Limit	UNSIGNED16	RW	Limit set by the MainDevice when SubDevice shall change its EtherCAT State to SAFEOP with AL Status Code 0x1A. The SubDevice shall reset its internal error counter when it is synchronized again and confirms the state transition from SAFEOP to OP.

22.2.2.1 0x10F1:1 Local Error Reaction

If a SubDevice supports several error reactions it shall support the object 0x10F1. It describes which error reaction shall be used in case of an error. The MainDevice can write this object. If the device supports only one local error reaction mechanism the entry should be read-only.

Default error reaction shall be the setting of the Rx/Tx PDO State if supported. A device that cannot detect a local error may not support the entry 0x10F1:01.

22.2.2.2 0x10F1:2 Sync Error Counter Limit

The SM Event Missed Error Counter increments every time by one when an SM Event was missed. But it does not give any information about the relation between received and missed SM Events. While a single fault can be tolerated multiple errors shall reset the communication until the error is removed.

A weighted "internal" SM Event Missed Counter (in the following Internal Sync Error Counter) is used to count missed and received SM Events. The SubDevice shall increment its Internal Sync Error Counter by 3 in case of a missed event. Every time a SM Event is received the Internal Sync Error Counter shall be decremented by 1.

The smallest value for the Internal Sync Error Counter is 0.

A Sync Error Counter Limit can be set by the MainDevice via 0x10F1:2 Sync Error Counter Limit. If the Internal Sync Error Counter exceeds the Sync Error Counter Limit a multiple error is detected and the SubDevice changes its EtherCAT state to SAFEOP with AL Status Code 0x1A. The Sync Error Counter should be reset by the SubDevice when the AL Error was acknowledged.

In Table 72 two examples show the use of the weighted Sync Error Counter.

Table 72: Example for weighted Error Counter

Example	SM Event	1	0	1	0	1	0	1	0	1	0	1
1	Internal Sync Error Counter (incremented by 1) Error Counter Limit = 3	0	1	0	1	0	1	0	1	0	1	0
2	Internal Sync Error Counter (incremented by 3) Error Counter Limit = 9	0	3	2	5	4	7	6	9	9	9	9

In example 1 the Internal Sync Error Counter is incremented by 1 in case of a missed SM2 Event and decremented by 1 in case of a received SM2 Event. Even though only every second SM2 Event is received the Sync Error Counter Limit will not be reached to indicate those multiple errors.

In example 2 the Internal Sync Error Counter is incremented by 3 in case of a missed SM2 Event and decremented by 1 in case of a received SM2 Event. After 8 bus cycles the Sync Error Counter Limit is reached and the multiple error is indicated to the MainDevice by an EtherCAT state change to SAFEOP with AL Status Code 0x1A.

Example 2 explains how the error counter shall be implemented for EtherCAT devices that support the Sync Error.

If the Sync Error Counter Limit is set to 0 the device does not change its EtherCAT State to SAFEOP in case of a Sync Error. This might be used for devices with dynamic Minimum Cycle Time.

22.2.3 Error Detection in the SubDevice

The SubDevice has the Error Detection mechanisms shown by Table 73.

Table 73: Error Detection Mechanisms by SubDevice

Mechanism	Use
EtherCAT State	M
SM Change	M
Watchdog	M
PDO Control	O
PDO Toggle	O
Synchronization Monitoring	C, mandatory depending on the synchronization modes
Local Error	C, mandatory if object 0x10F1:1 is supported

The MainDevice specific Error Reasons will be detected with the mechanisms listed in Table 74.

Table 74: Error Detection of MainDevice Errors

MainDevice Error Reason	SubDevice Error Detection
Outputs or part of Outputs are invalid (e.g. PLC-Stop)	PDO Control, PDO Toggle
Outputs are not updated (e.g. Application exception)	Watchdog, PDO Toggle, Synchronization Monitoring
MainDevice Restart	EtherCAT State, Watchdog, Synchronization Monitoring
Timing Error (e.g. sending of cyclic telegrams was not finished when the next cycle started)	PDO Toggle, AL Event, Synchronization Monitoring

22.2.4 Error Detection in the MainDevice

Table 75 shows the Error Detection Mechanisms that might be supported by the MainDevice.

Table 75: Error Detection Mechanisms by MainDevice

Mechanism	Use
EtherCAT State	M
PDO State	O
PDO Toggle	O
Working Counter	M
Lost SubDevices	M
Lost Frames	M
Local Error	C, mandatory if Local error is supported

The SubDevice specific Error Reasons will be detected with the mechanisms listed in Table 76.

Table 76: Error Detection of SubDevice Errors

SubDevice Error Reason	MainDevice Error Detection
Inputs or part of Inputs are invalid (e.g. I/O-Error)	PDO State, Working Counter
Inputs are not updated	PDO Toggle, Working Counter
No communication of the Application Controller with the ESC	PDO Toggle, Working Counter
Application Controller restarted	PDO Toggle, Working Counter, EtherCAT State
Cycle Time exceeded	PDO Toggle, Working Counter, Error Counters
SubDevice powered off	Lost SubDevices, Lost Frames
SubDevice disconnected	Lost SubDevices, Lost Frames
Inputs are OK, but Outputs cannot be set (e.g. I/O-Error)	PDO State, Working Counter
SubDevice Timing Error	Error Counters, Sync Error

22.3 Error Reaction

The Error Reactions are distinguished between local reactions and bus reactions.

22.3.1 Error Reactions in the SubDevice

22.3.1.1 Local Reactions

Table 77 shows the local reactions.

Table 77: Local Reactions SubDevice Errors

Local Reactions	Explanation	Use
Use default output data	The outputs will be set to a default value which may be settable by the MainDevice.	M
Use old output data	The outputs will not be changed.	M
Use expected output data	The outputs will be adapted to the expected values.	O
SubDevice specific output reaction	The outputs will be modified according to a SubDevice specific reaction like a function. This reaction may be configurable by the MainDevice.	O

22.3.1.2 Bus Reactions

Table 78 shows the bus reactions.

Table 78: Bus Reactions SubDevice Errors

Bus Reactions	Explanation	Use
No reaction	The SubDevice will not change his EtherCAT behaviour	
TxPDO State	The SubDevice will set the TxPDO State to TRUE (only if TxPDO State is supported).	C Mandatory if TxPDO State is supported
RxPDO State	The SubDevice will set the RxPDO State to TRUE (only if RxPDO State is supported).	C Mandatory if RxPDO State is supported
EtherCAT State	The SubDevice will change the EtherCAT State.	M
Disable Sync Manager Channel	The SubDevice will disable Sync Manager channel(s) that the Working Counter will not increment any more when accessing the related memory areas.	O used if Local Error (e.g. Input error) would lead to this case. Indication mechanism. Alternative is use of PDO State
Error Counter	The SubDevice will increment of the error counters of object 0x1C3x	C Mandatory if Error Counters in Object 0x1C3x are supported (SI 11, 12, 13, 14)
Sync Error	The SubDevice will set the Sync Error bit (0x1Cx:20)	C Mandatory if Sync Error Bit in 0x1C3x:20 is supported

22.3.1.3 Relation between Error Detection and Error Reaction

Table 79 shows the relation between Error Detection and Error Reaction in the SubDevice.

Table 79: Relation Error Detection and Error Reaction in the SubDevice

SubDevice Error Detection	Local Error Reaction	Bus Error Reaction
EtherCAT State, Watchdog, SM-Change	Use default output data or SubDevice specific output reaction	EtherCAT State
PDO Control	Use default output data or SubDevice specific output reaction	No reaction
RxPDO Toggle (only single error)	Use old output data or use expected output data	Increment Error Counter (0x1C3x:0E RxPDO Toggle Failed Counter) and set Sync Error = 1
RxPDO Toggle (multiple error = Sync Error Counter Limit exceeded)	Use default output data or SubDevice specific output reaction	Increment Error Counter (0x1C3x:0E RxPDO Toggle Failed Counter) and set Sync Error = 1 and set EtherCAT State to SAFEOP with AL Status Code 0x33 (or 0x1A). See also 0x10F1:2 Sync Error Counter Limit.
Local Output Error	Use default output data or SubDevice specific output reaction	RxPDO State or (Disable Sync Manager Output Channel and set EtherCAT state to SAFEOP) (Error behaviour set in 0x10F1 Object 0x10F1: Error Settings)
Local Input Error	Use default output data or SubDevice specific input reaction	TxPDO State or (Disable Sync Manager Input Channel and set EtherCAT State to SAFEOP) (Error behaviour set in 0x10F1 Object 0x10F1: Error Settings)

SubDevice Error Detection	Local Error Reaction	Bus Error Reaction
Synchronization Monitoring 1 (only single error)	No reaction	Increment Error Counter (0x1C3x:0C Cycle Time Too Small) and acknowledge the AL Event but do not start a new cycle but dismiss the outputs. MainDevice detects the Error by the TxPDO Toggle if it is supported by the SubDevice.
Synchronization Monitoring 1 (multiple error = Internal Sync Error Counter Limit exceeded)	Use default output data or SubDevice specific output reaction	set EtherCAT State to SAFEOP with AL Status Code 0x33 (or 0x1A). See also 0x10F1:2 Sync Error Counter Limit.
Synchronization Monitoring 2 (only single error)	Use old output data or use expected output data	set Sync Error = 1
Synchronization Monitoring 2 (multiple error = Internal Sync Error Counter Limit exceeded)	Use default output data or SubDevice specific output reaction	set EtherCAT State to SAFEOP with AL Status Code 0x32 (or 0x1A). See also 0x10F1:2 Sync Error Counter Limit.
Synchronization Monitoring 3 (only single error)	Use old output data or use expected output data	Increment Error Counter (0x1C3x:0B SM-Event Missed) and set Sync Error = 1
Synchronization Monitoring 3 (multiple error = Internal Sync Error Counter Limit exceeded)	Use default output data or SubDevice specific output reaction	Increment Error Counter (0x1C3x:0B SM Event Missed) and set EtherCAT State to SAFEOP with AL Status Code 0x34 (or 0x1A). See also 0x10F1:2 Sync Error Counter Limit.
Synchronization Monitoring 4 (only single error)	Use old output data or use expected output data	Increment Error Counter (0x1C3x:0D Shift Time Too Short) and set Sync Error = 1
Synchronization Monitoring 4 (multiple error = Internal Sync Error Counter Limit exceeded)	Use default output data or SubDevice specific output reaction	Increment Error Counter (0x1C3x:0D Shift Time Too Short) and set EtherCAT State to SAFEOP with AL Status Code 0x32 (or 0x1A). See also 0x10F1:2 Sync Error Counter Limit.
Synchronization 5	Use default output data or SubDevice specific output reaction	Set EtherCAT state to SAFEOP and AL Status Code to 0x2C (or 0x1A)
Synchronization 6	Use default output data or SubDevice specific output reaction	Set EtherCAT state to SAFEOP and AL Status Code to 0x2C (or 0x1A)
Synchronization 7	Use default output data or SubDevice specific output reaction	Refuse state transition to OP and set AL Status Code to 0x2D (or 0x1A)

22.3.2 Error Reactions in the MainDevice

22.3.2.1 Local Reactions

The local reactions may be settable in the MainDevice's configuration tool. They are listed in Table 80.

Table 80: Local Reactions MainDevice Errors

Local Reactions	Explanation	Use
Status	The error will be signalized in a status field to the application.	M
Use default input data	The inputs will be set to a default value which may be settable by the MainDevice's configuration tool with a default value from the SubDevice's device description file.	O
Use old input data	The inputs will not be changed.	M
Use expected data	The inputs will be adapted to the expected values (in case of a control loop for example).	O
MainDevice specific input reaction	The inputs will be modified according to a MainDevice specific reaction like a function.	O

22.3.2.2 Bus Reactions

The bus reactions may be settable in the MainDevice's configuration tool. They are listed in Table 81.

Table 81: Bus Reactions MainDevice Errors

Bus Reactions	Explanation	Use
No reaction	The MainDevice will not change his EtherCAT behaviour	
RxPDO Control	The MainDevice will set the RxPDO Control to TRUE (only if RxPDO Control is supported).	O
Use default output data	The outputs will be set to a default value which may be settable by the MainDevice's configuration tool with a default value from the SubDevice's device description file.	O
SyncUnit to SAFEOP	The MainDevice will change the EtherCAT State of the SubDevice(s) with an application controller and will disable the Sync Manager channel(s) of the SubDevice(s) which only support the EtherCAT state machine emulation.	O

22.3.2.3 Relation between Error Detection and Error Reaction

Table 82 shows the relation between Error Detection and Error Reaction in the MainDevice.

Table 82: Relation Error Detection and Error Reaction in the MainDevice

SubDevice Error Detection	Local Error Detection	Bus Error Detection
EtherCAT State	Status and one of the other local reactions (Handling of Input data)	SyncUnit to SAFEOP (for SyncUnit definition see 22.4.2)
RxPDO State	Status	No reaction
TxPDO State	Status and one of the other local reactions (Handling of Input data)	No reaction
TxPDO Toggle	Status and one of the other local reactions (Handling of Input data)	No reaction
Working Counter Single Fault	Status and one of the other local reactions (Handling of Input data)	No reaction
Working Counter Multiple Faults	Status and one of the other local reactions (Handling of Input data)	SyncUnit to SAFEOP
Single Lost Frame	Status and one of the other local reactions (Handling of Input data)	No reaction
Multiple Lost Frames	Status and one of the other local reactions (Handling of Input data)	SyncUnit to SAFEOP
Lost SubDevices	Status and one of the other local reactions (Handling of Input data)	SyncUnit to SAFEOP
Local Error	Status	PDO Control if supported or EtherCAT State (of SubDevice(s) belonging to erroneous application)

22.4 Restart Behaviour

22.4.1 Restart Behaviour in the SubDevice

When the error is gone the SubDevice will use the outputs sent from the MainDevice instead of the values according to the local reaction.

The Restart Behaviour of the different bus reactions is shown in Table 83.

Table 83: SubDevice Restart Behaviour

Bus Error Reaction	Restart Behaviour
EtherCAT State	EtherCAT State according to the requested EtherCAT state from the MainDevice
RxPDO State	Set to FALSE
TxPDO State	Set to FALSE
Disable Sync Manager Channel(s)	Will be enabled
Error Counter	will not increment any more
SyncError	Set to FALSE

Figure 19 shows the state transitions in SAFEOP and OP of the SubDevice (changes to PREOP or INIT (AL Control Event, SM Change Event) are not shown in this figure to get a better overview). In the following figure the local output error and the local input error shall be ignored if the Error behaviour (0x10F1) is set to PDO State or Device Specific State.

The restart behaviour of the watchdog is settable. If the “watchdog enable bit” of the Sync Manager 2 (3 if no outputs are transmitted in SAFEOP or OP) Control Byte is set, “WD okay” is only TRUE, if new outputs were received (inputs are read), otherwise “WD okay” is always TRUE (MainDevice legacy mode). The MainDevice legacy mode is optional the other watchdog behaviour is mandatory.

If the SubDevice has to be synchronized in OP, it should delay the state transition confirmation from SAFEOP to OP until it is synchronized or the SAFEOP2OP-Timeout Time (defined in the SubDevice's device description) has expired. It could happen that only one SM event is received before the state transition is requested by the MainDevice.

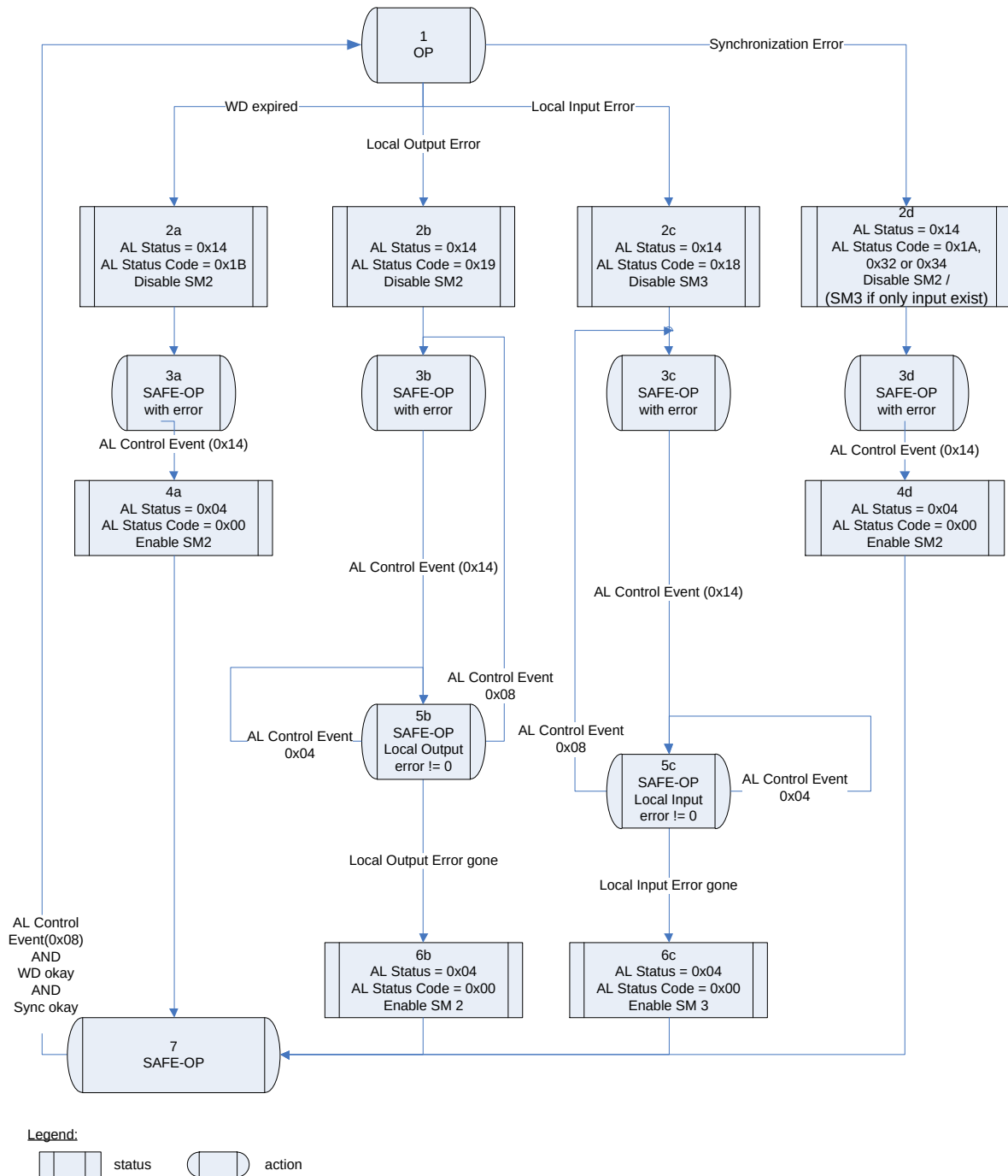


Figure 19: Restart Behaviour of the SubDevice

22.4.2 Restart Behaviour in the MainDevice

When the error is gone the MainDevice will use the inputs sent from the SubDevice(s) and reset the status to the application.

The Restart Behaviour of the different bus reactions is shown in Table 84.

Table 84: MainDevice Restart Behaviour

Bus Error Reaction	Restart Behaviour
EtherCAT State of Sync Unit	See Figure 20
Use default output data	Set to process data values
PDO Control	Set to FALSE

Figure 20 shows the optional bus error reaction and the restart behaviour in the MainDevice when changing the EtherCAT state of all SubDevices belonging to a Sync Unit. A Sync Unit consists at least of all SubDevices whose process data will be transferred with the same EtherCAT datagram. In that case a local error in one of those SubDevices could not be assigned exactly so that all SubDevices related to that EtherCAT datagram should be set to SAFE-OP.

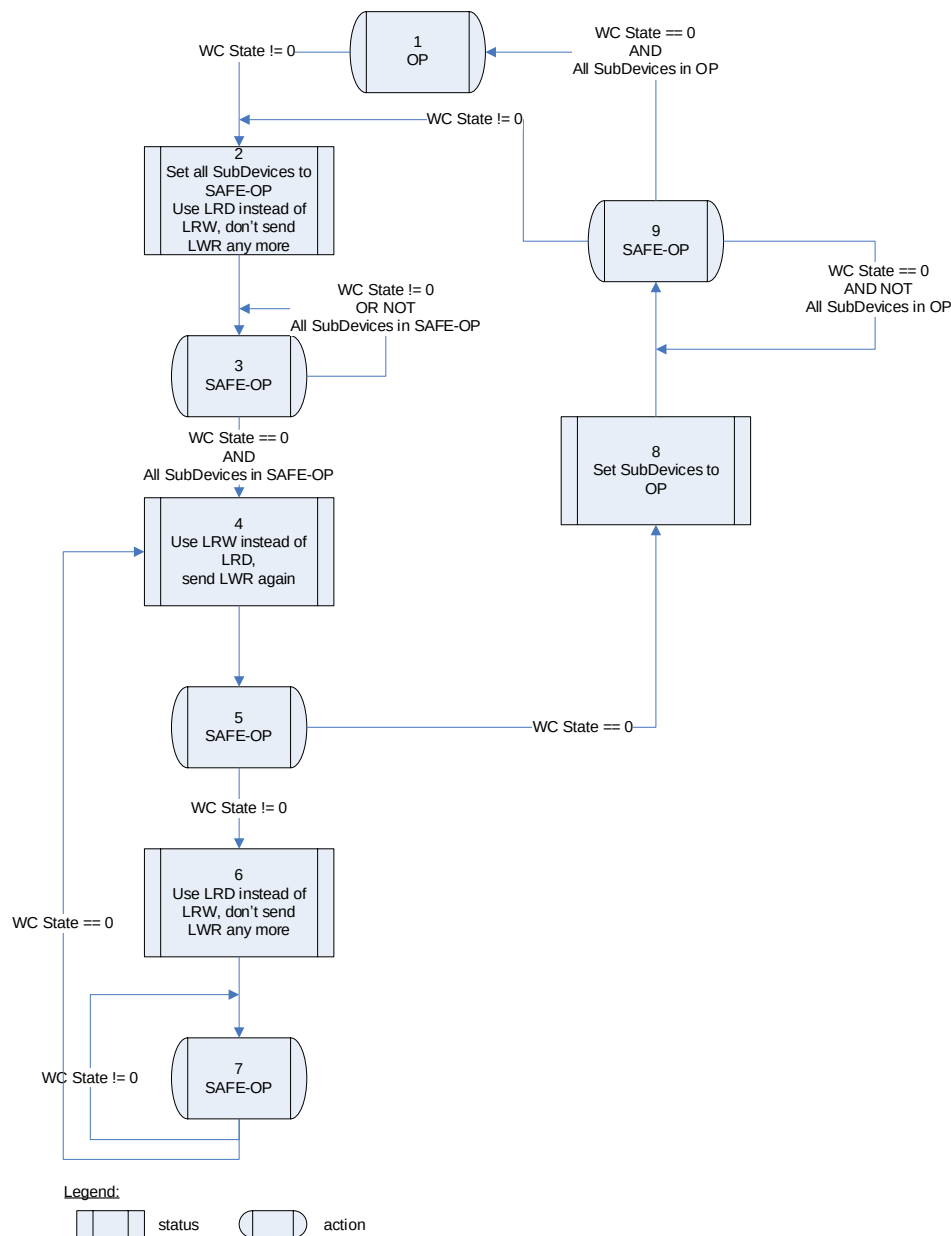


Figure 20: Restart Behaviour of a Sync Unit in the MainDevice

If the MainDevice detects a working counter failure in a process data frame (LRW, LWR, LRD) while the MainDevice is in state OP (1), all SubDevices belonging to this Sync Unit should be set to state SAFEOP (2). All LRW frames belonging to the Sync Unit should be changed to LRD frames, LWR frames should not be sent anymore, so that no invalid outputs will be sent to the SubDevices.

The MainDevice is now in state SAFEOP (3) and waits until that the process data frame(s) (sent with LRD) return(s) with a correct working counter and all SubDevices belonging to the Sync Unit are in SAFEOP.

If the working counter is correct and all SubDevices belonging to the Sync-Unit are in SAFEOP, the MainDevice should use the original process data frames again (4). If the working counter is still not okay the MainDevice should change to LRD-frames again that inputs could be updated (6).

When the working counter of the original process data frame(s) is correct, the error is gone and all SubDevices belonging to the Sync Unit should be set back to OP state (8, 9). Depending on the MainDevice's application an acknowledge is necessary to switch to state 8 and 9.

The MainDevice waits until all SubDevices are in OP (9) and is back in OP (1).

23 Synchronization

The motivation of a standard EtherCAT Synchronization Handling is a unique behavior of EtherCAT devices (MainDevices and SubDevices) for synchronized applications.

The following synchronization modes are distinguished.

Synchronization in the SubDevice

- Free Run
- Synchronization with SM Event
- Synchronization with Sync0 Event

Synchronization in the MainDevice

- Cyclic Mode
- DC Mode

Configuration of Synchronization

The supported synchronization modes, times and diagnostic features are described with the objects 0x1C32, 0x1C33, 0x1C02, 0x1400-0x15FF and 0x1800-0x19FF. The supported synchronization modes are described in the element OpModes in the EtherCAT SubDevice Information (ESI) file.

NOTE: The SyncManager Communication Type is described with the Object 0x1C00. The following SyncManager assignment shall be used. If process data in both directions input and output is used then the output data shall be handled in a SyncManager with the lower index and input data shall be handled by the SyncManager with the higher index.

If mailboxes are used the output mailbox shall be assigned to the SyncManager with the lower index (usually SM0) and the input mailbox shall be assigned to the SyncManager (usually SM1) with the higher index.

Table 85 shows when mailbox and process data is used.

Table 85: SyncManger Assignment with Mailbox

SM	Description
0	MailboxOut
1	MailboxIn
2	Output Process Data
3	Input Process Data

If no process output data is used but input process data also SM2 might be used for input process data.

Table 86 shows when only process data is used and no mailbox.

Table 86: SyncManger Assignment without Mailbox

SM	Description
0	Output Process Data
1	Input Process Data

If no process output data is used but process input data also SM0 might be used for process input data.

For description of the SyncManagers the objects 0x1C3y are used (with y = number of SM).

For the following description of the Synchronization Modes the SynManager assignment as described in Table 85 SyncManger Assignment with Mailbox is assumed.

23.1 Synchronization in the SubDevice

The motivation of a standard EtherCAT Synchronization Handling is a unique behavior of EtherCAT devices (MainDevices and SubDevices) for synchronized applications.

23.1.1 Synchronization Modes in the SubDevice

The following synchronization modes are standardized in this document:

- **Free Run**
SubDevice's application is not synchronized to EtherCAT.
- **Synchronous with SM Event (SM-Synchronous)**
SubDevice's application is synchronized to the SM2 event (if cyclic outputs are transmitted) or the SM3 Event (if only cyclic inputs are transmitted). SM events are based on the time an EtherCAT frame is received. This time can jitter in the range of a few microseconds due to the EtherCAT MainDevice implementation (delay in Stack, PHY & MAC Delay, etc).
- **Synchronous with DC SYNC Event (DC-Synchronous)**
SubDevice's application is synchronized to the Sync0 or Sync1 event, which are based on the distributed clocks (DC) unit. The jitter could be reduced to a few nanoseconds.

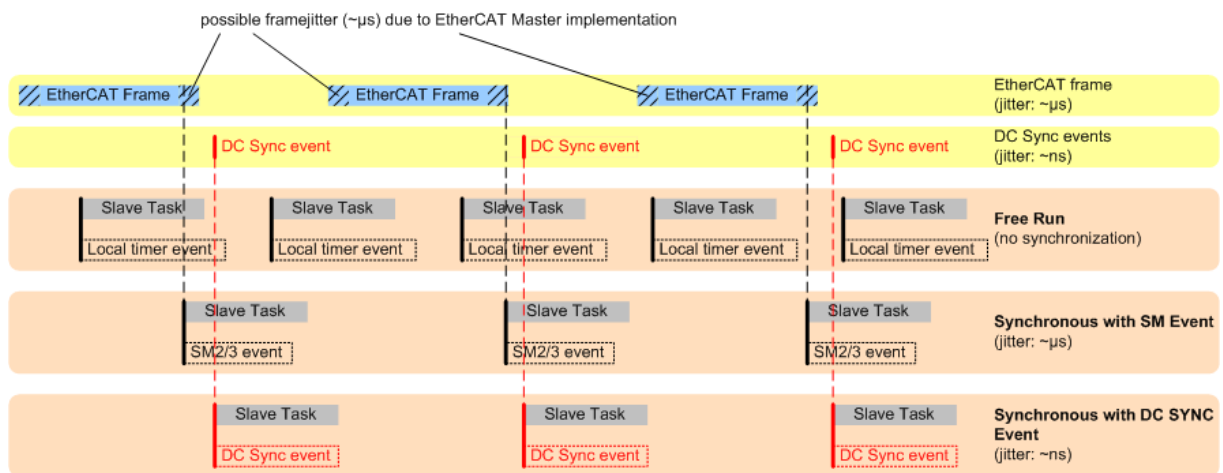


Figure 21: Differences between synchronization in the SubDevices (simplified)

23.1.1.1 Synchronization Mode Identification

The different synchronization types can be identified by the different combinations of the Subindexes of 0x1C32 and 0x1C33 as shown in Table 87.

Clause 23.1.2 specifies the SyncManager Parameters generically.

NOTE: "--" within one table field means that the SubIndex is not used and can either be 0 or not existing.

Table 87: Synchronization Mode Identification

Sync Mode	Synchro- nization Type 0x1C32 SI 01	Synchro- nization Type 0x1C33 SI 01	Output Shift Time 0x1C32 SI 03	Input Shift Time 0x1C33 SI 03	Calc and Copy Time 0x1C32 SI 06	Calc and Copy Time 0x1C33 SI 06	Delay Time 0x1C32 SI 09	Delay Time 0x1C33 SI 09	Fixed Sync0 Cycle Time
Free Run Mode									
Free Run ²	0x00	0x00	--	--	--	--	--	--	--
SM Event Mode									
SM2 ²	0x01	0x22	--	--	--	--	--	--	--
SM3 ²	--	0x01	--	--	--	--	--	--	--
SM2, Shift Input Latch ²	0x01	0x22	--	!=0 ³	--	!=0	--	--	--
SM3, Shift Input Latch ²	---	0x01	--	!=0 ³	--	!=0	--	--	--
DC Mode									
DC Sync0	0x02	0x02	--	--	>=0	!=0	>=0	--	--
DC Sync0, Shift Outputs Valid and Input Latch	0x02	0x02	>=0 ³	>=0 ³	!=0	!=0	>=0	--	--
DC, Shift of Input Latch with Sync1	0x02	0x03	--	--	!=0	!=0	>=0	>=0	--
DC Sync1	0x03	0x03	--	--	!=0	!=0	>=0	--	--
Subordinated Application Controller Cycles									
DC, Shift Outputs Valid/ Input Latch	0x03	0x02 or 0x03	>=0 ³	>=0 ³	>=0	!=0	>=0	>=0	!=0

23.1.1.2 Terminology

23.1.1.2.1 Copy and Prepare Outputs

With a trigger event (local timer event, SM2/3 event Sync0/1 event) output data is read from the SyncManager Output area and, if necessary, mathematical operations are performed on those values. Then, the physical output signal is generated. With the Outputs Valid mark the output signal is available at the process.

The Copy and Prepare Outputs time sums up the times for copying process data from the SyncManager to the local memory, mathematical operations if necessary and the hardware delays (depending on implementation including some software run time). The single times are not further

² If Outputs are available, the SubDevice is synchronized always on the SM 2 event. If no outputs are available, the SubDevice is synchronized on the SM 3 event.

³ Shall be writeable if the related action can be shifted

determined. They would comply with the values described in SyncManager Object 0x1C32 (see Table 88).

Table 88: 0x1C32 Copy and Prepare Outputs

Described Times	SyncManager Object 0x1C32
Copy process data from SyncManager and mathematical operations	Calc and Copy Time (0x1C32:06)
Hardware delay time	Delay Time (0x1C32:09) Hardware delay time shall be kept short and fixed

23.1.1.2.2 Get and Copy Inputs

The Get and Copy Inputs time sums up the hardware delay of reading in the signal, the execution of mathematical operations, if necessary, and the copying of the input process data to the SyncManager 3 area. The single times are not further determined. They would comply with the values described in SyncManager Object 0x1C33 (see Table 89).

Table 89: 0x1C33 Get and Copy Inputs

Described Times	SyncManager Object 0x1C33
Mathematical operations and copy data from local memory to SyncManager	Calc and Copy Time (0x1C33:06)
Hardware delay to prepare the Input Latch	Delay Time (0x1C33:09) Hardware delay time shall be kept short and fixed

The Inputs are available at the SyncManager 3 area after the Min Cycle Time (0x1C32:05).

23.1.1.2.3 Outputs Valid

At the Outputs Valid the outputs are available for the process (e.g. as electrical signal).

23.1.1.2.4 Input Latch

With the Input Latch the input data is acquired but not mathematical operations have been performed on the data and have not been copied to the SyncManager area.

23.1.1.2.5 User Shift Time

The User Shift Time considers jitter of the MainDevice.

23.1.1.2.6 Sync1 Cycle Time

The Sync1 Cycle Time might be used for shifting the Start Input Latch or Start Driving Outputs. The Sync1 Cycle Time is described with the register 0x0984:0x0987. It describes the shift time between the Sync0 and Sync1 signal while the Sync0 is always the reference signal.

23.1.1.2.7 Shift Time

Shift Time describes the time between the sync event (SM2 event, SM3 event, SYN0, Sync1) and the Outputs Valid or Input Latch. If the Outputs Valid or Input Latch could be shifted this time should be writable.

23.1.1.3 Free Run

In Free Run Mode the local cycle is started by a local timer interrupt of the application controller. The cycle time may be changed by the MainDevice (optional behaviour of the SubDevice) to change the timer. In Free Run mode the local cycle runs independent from the communication cycle and/ or MainDevice cycle.

Optional features

The SubDevice may support a flexible Cycle Time (0x1C32:02 is writable). In this case the Minimum Cycle Time (0x1C32:05) shall be supported by the SubDevice, too.

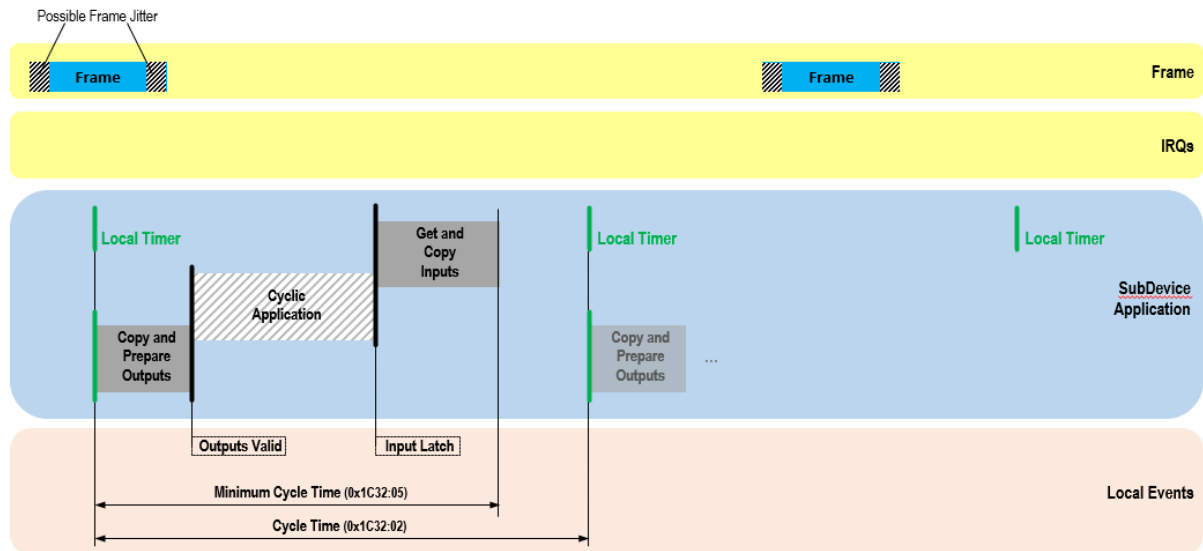


Figure 22: Local Cycle with Free Run synchronization, with local timer

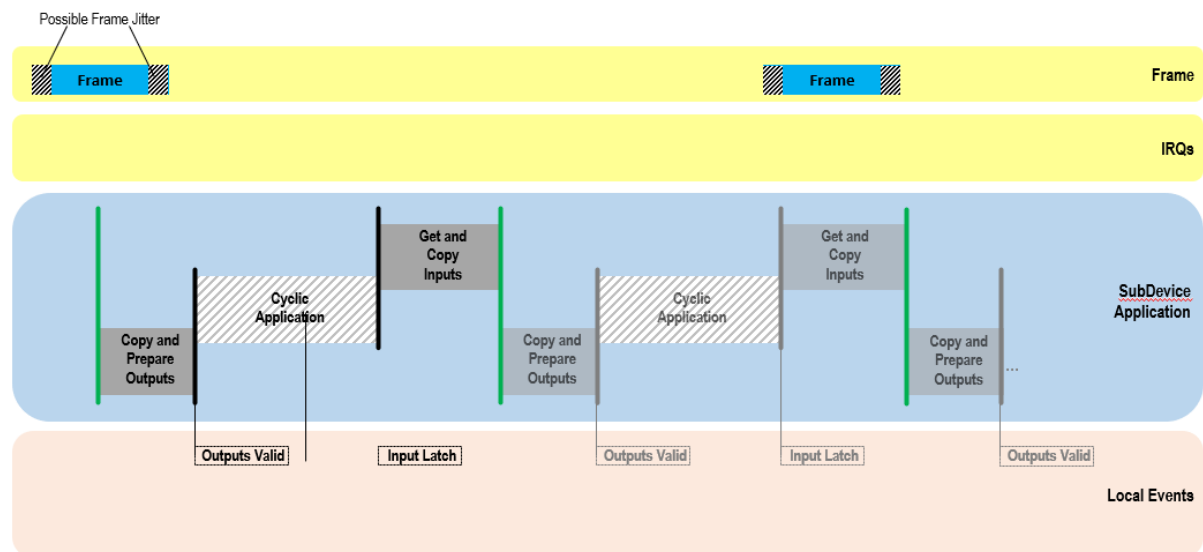


Figure 23: Local Cycle with Free Run synchronization, without local timer

Table 90 and Table 91 explain how those objects shall be used in Free Run Mode.

Table 90: 0x1C32 Free Run

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	R or RW	C	0x00: Free Run Optional if only Free Run is supported
2	Cycle Time	R or RW	O	Local Cycle Time of application controller
3	Shift Time	--	--	
4	Synchronization Types supported	R	C	Bit 0: Free Run supported Optional if only Free Run is supported
5	Minimum Cycle Time	R	C	Mandatory if 0x1C32:02 is writable
6	Calc and Copy Time	--	--	
7	Minimum Delay Time	--	--	
8	Get Cycle Time	--	--	
9	Delay Time	--	--	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed	--	--	
12	Cycle Time Too Small	--	--	
13	Shift Time Too Short	--	--	
14	RxPDO Toggle Failed	--	--	
15	Minimum cycle distance	--	--	
16	Maximum cycle distance	--	--	
17	Minimum SM SYNC distance	--	--	
18	Maximum SM SYNC distance	--	--	
31:19	--	--	--	
32	Sync Error	--	--	

Table 91: 0x1C33 Free Run

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	R or RW	C	Same value as 0x1C32:021
2	Cycle Time	R or RW	O	Same value as 0x1C32:02
3	Shift Time	--	--	
4	Synchronization Types supported	R	C	Same value as 0x1C32:04
5	Minimum Cycle Time	R	C	Same value as 0x1C32:05
6	Calc and Copy Time	--	--	
7	--	--	--	
8	Get Cycle Time	--	--	
9	Delay Time	--	--	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed	--	--	
12	Cycle Time Too Small	--	--	
13	Shift Time Too Short	--	--	
14	RxPDO Toggle Failed	--	--	
31:15	--	--	--	
32	Sync Error	--	--	

23.1.1.4 SM-Synchronous with SM Event

The local cycle is started when the SM2 (with cyclic outputs) or SM3 (without cyclic outputs) event is received.

If outputs are available the SubDevice is synchronized always on the SM 2 event. If no outputs are available, the SubDevice is synchronized on the SM 3 event.

23.1.1.4.1 SM-Synchronous with SM2/3 Event

The local cycle is started when the SM2/3 event is received.

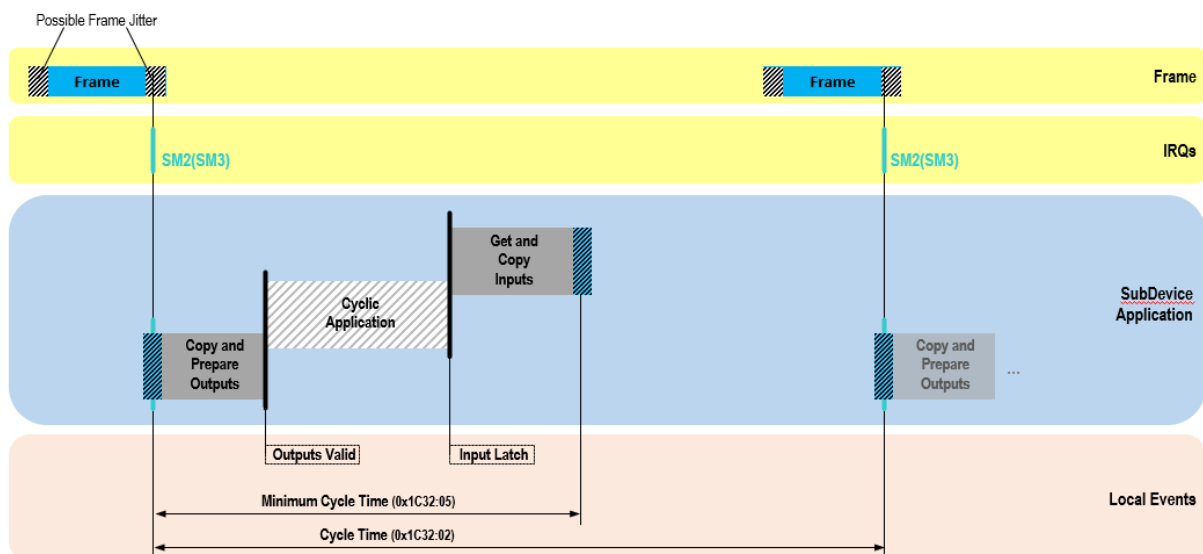


Figure 24: Local Cycle synchronous with SM2 Event

Table 92 and Table 93 explain how those objects shall be used in Synchronous with SM 2/3 Event Mode.

Table 92: 0x1C32 Synchronous with SM 2/3 Event

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x01: SM Synchronous – synchronized with SM 2 Event
2	Cycle Time	R or RW	O	Communication cycle time
3	Shift Time	--	--	
4	Synchronization Types supported	R	M	Bit 1: SM-Synchronous supported
5	Minimum Cycle Time	R	M	
6	Calc and Copy Time	--	--	
7	Minimum Delay Time	--	--	
8	Get Cycle Time	RW	C	Mandatory if Subindex 4 Bit 14 is 1
9	Delay Time	--	--	
10	Sync0 Cycle Time	--	--	
11	SM-Event Missed	R	O	
12	Cycle Time Too Small	R	M	
13	Shift Time Too Short	--	--	
14	RxPDO Toggle Failed	R	O	
15	Minimum cycle distance	R	O	
16	Maximum cycle distance	R	O	
17	Minimum SM SYNC distance	--	--	
18	Maximum SM SYNC distance	--	--	
31:19	--	--	--	
32	Sync Error	R	C	Shall be supported if SM-Event Missed Counter (SI11) is supported

Table 93: 0x1C33 Synchronous with SM 2/3 Event

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x01: SM-Synchronous - synchronized with SM 3 Event (used when only inputs are transmit in SAFEOP and OP) 0x22: SM-Synchronous with SM2 Event (used when outputs are transmit in SAFEOP and OP)
2	Cycle Time	R or RW	O	Same value as 0x1C32:02
3	Shift Time	--	--	
4	Synchronization Types supported	R	M	Same value as 0x1C32:04
5	Minimum Cycle Time	R	M	Same value as 0x1C32:05
6	Calc and Copy Time	--	--	
7	--	--	--	
8	Get Cycle Time	RW	C	Same value as 0x1C32:08
9	Delay Time	--	--	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed	R	O	Same value as 0x1C32:0B
12	Cycle Time Too Small	R	M	Same value as 0x1C32:0C
13	Shift Time Too Short	--	--	
14	RxPDO Toggle Failed	R	O	Same value as 0x1C32:0E
31:15	--	--	--	
32	Sync Error	R	C	Same value as 0x1C32:20

23.1.1.4.2 SM-Synchronous with SM2/3 Event, Shift Input Latch

Input data should be latched from the process as close as possible to the next SM2/3 event to provide the latest input data to the control system (MainDevice). To obtain this the input Latch is shifted closer to the SM2/3 event using the Shift Time (0x1C33:03).

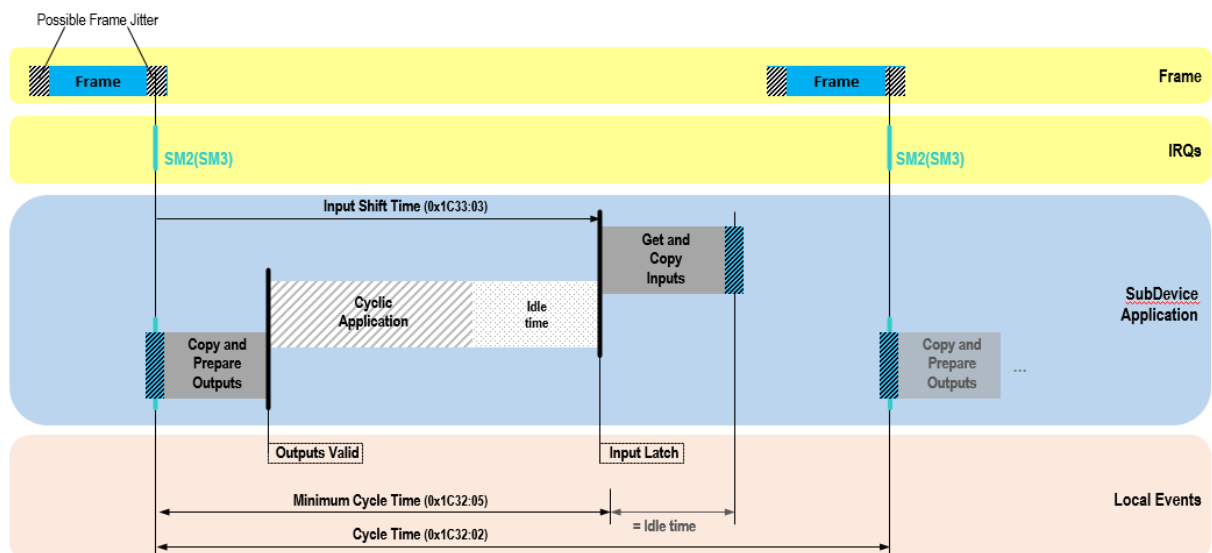

Figure 25: Local Cycle synchronous with SM2/3 Event, Shift Input Latch

Table 94 and Table 95 explain how those objects shall be used in Synchronous with SM 2/3 Event Mode and shifted Input Latch.

Table 94: 0x1C32 Synchronous with SM 2/3 Event, Shift Input Latch

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x01: SM-Synchronous – synchronized with SM 2/3 Event
2	Cycle Time	R or RW	O	Communication cycle time
3	Shift Time	--	--	
4	Synchronization Types supported	R	M	Bit 1: SM-Synchronous supported
5	Minimum Cycle Time	R	M	
6	Calc and Copy Time	--	--	
7	Minimum Delay Time	--	--	
8	Get Cycle Time	RW	C	Mandatory if Subindex 4 Bit 14 is 1
9	Delay Time	--	--	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed	R	O	
12	Cycle Time Too Small	R	M	
13	Shift Time Too Short	--	--	
14	RxPDO Toggle Failed	R	O	
15	Minimum cycle distance	R	O	
16	Maximum cycle distance	R	O	
17	Minimum SM SYNC distance	--	--	
18	Maximum SM SYNC distance	--	--	
31:19	--	--	--	
32	Sync Error	R	C	Shall be supported if SM-Event Missed Counter (S11) is supported

Table 95: 0x1C33 Synchronous with SM 2/3 Event, Shift Input Latch

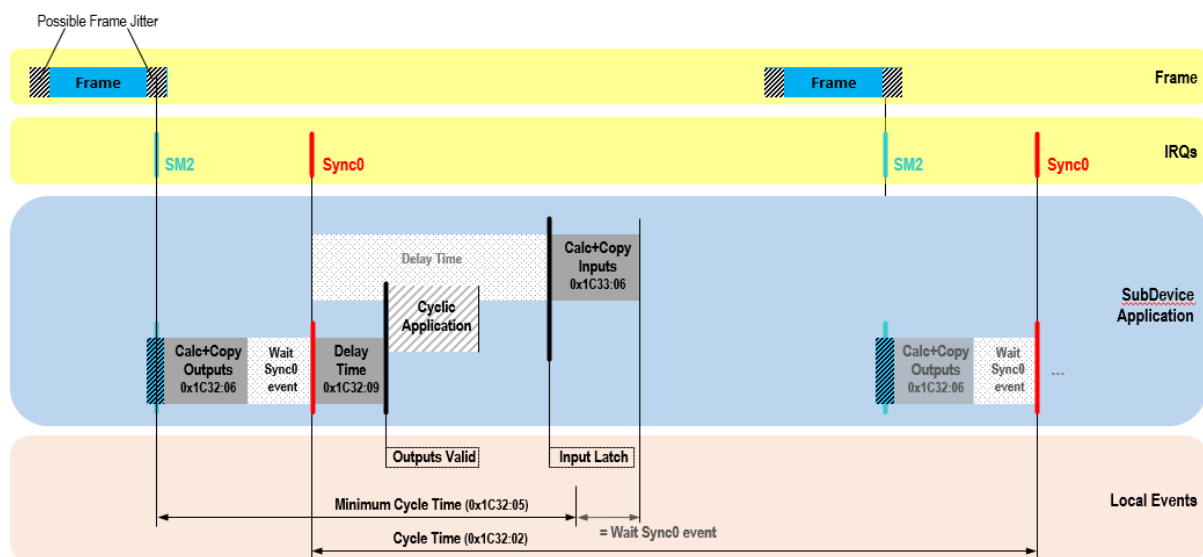
SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x01: SM-Synchronous - synchronized with SM 3 Event (used when only inputs available) 0x22: SM-Synchronous with SM2 Event (used when outputs available)
2	Cycle Time	R or RW	O	Same value as 0x1C32:02
3	Shift Time	RW	M	
4	Synchronization Types supported	R	M	Same value as 0x1C32:04
5	Minimum Cycle Time	R	M	Same value as 0x1C32:05
6	Calc and Copy Time	R	M	
7	--	--	--	
8	Get Cycle Time	RW	C	Same value as 0x1C32:08
9	Delay Time	--	--	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed	R	O	Same value as 0x1C32:0B
12	Cycle Time Too Small	R	M	Same value as 0x1C32:0C
13	Shift Time Too Short	--	--	
14	RxPDO Toggle Failed	R	O	Same value as 0x1C32:0E
31:15	--	--	--	
32	Sync Error	R	C	Same value as 0x1C32:20

23.1.1.5 DC Mode

23.1.1.5.1 DC Mode (Synchronous with Sync0 Event)

23.1.1.5.1.1 General synchronization with Sync0 Event

The local cycle is started when the Sync0 event is received. The Process Data Frame (Frame) can optionally be processed in the SubDevice with the help of SM2 Event before the next Sync0 Event is received: The Calc + Copy Time contains in this case the minimum time difference between receiving of the frame (SM2 Event) and the Sync0 Event.


Figure 26: Local Cycle synchronous to Sync0 Event with support of SM2 event

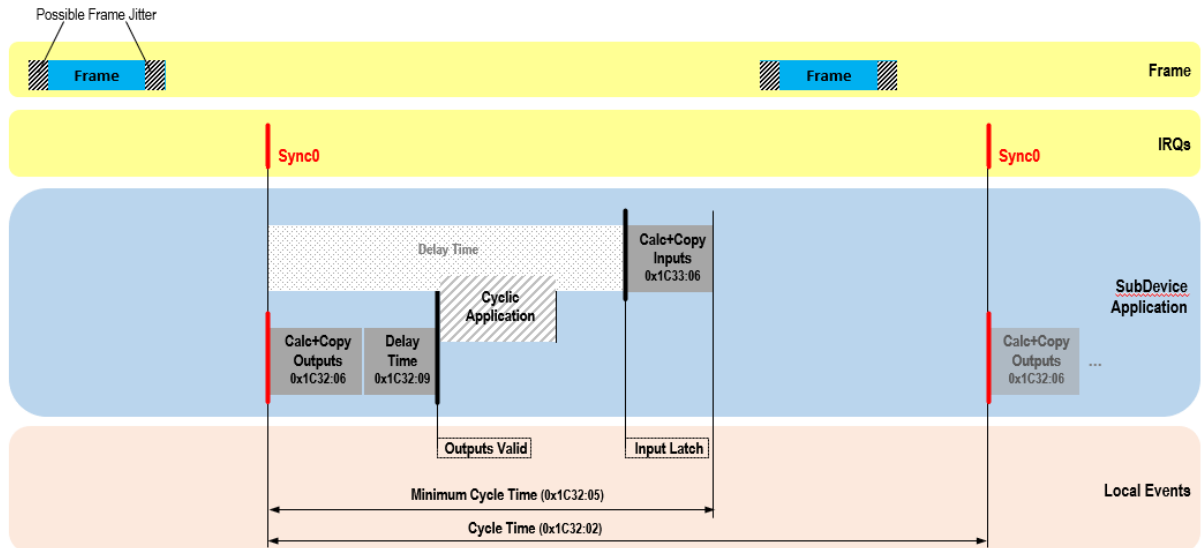


Figure 27: Local Cycle synchronous to Sync0 Event without support of SM2 event

Table 96 and Table 97 explain how those objects shall be used in DC Mode.

Table 96: 0x1C32 DC Mode

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	R or RW	M	0x02: DC Sync0 - synchronized with Sync0 Event
2	Cycle Time	R	O	Sync0 Cycle Time (register 0x09A3:0x09A0) Time between two Sync0 events. Sync0 Cycle Time shall be copied to this entry.
3	Shift Time	--	--	
4	Synchronization Types supported	R	M	Bit 4:2 : DC Type supported: 001 = DC Sync0
5	Minimum Cycle Time	R	M	
6	Calc and Copy Time	R	M	Minimum Time between the frame and the Sync0 event
7	Minimum Delay Time	--	--	
8	Get Cycle Time	RW	C	Mandatory if Subindex 4 Bit 14 is 1
9	Delay Time	R	M	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed Counter	R	O	
12	Cycle Time Too Small	R	M	
13	Shift Time Too Short Counter	R	O	incremented if the SM-event is received too late and the SYNC0-event therefore generated before the calc+copy time ends (MainDevice telegram too late)
14	RxPDO Toggle Failed	R	O	
15	Minimum cycle distance	R	O	
16	Maximum cycle distance	R	O	
17	Minimum SM SYNC distance	R	O	
18	Maximum SM SYNC distance	R	O	
31:19	--	--	--	
32	Sync Error	R	C	Shall be supported if SM-Event Missed Counter (SI11) or Cycle Time Too Small Counter (SI12) or Shift Time Too Short Counter (SI13) are supported

Table 97: 0x1C33 DC Mode

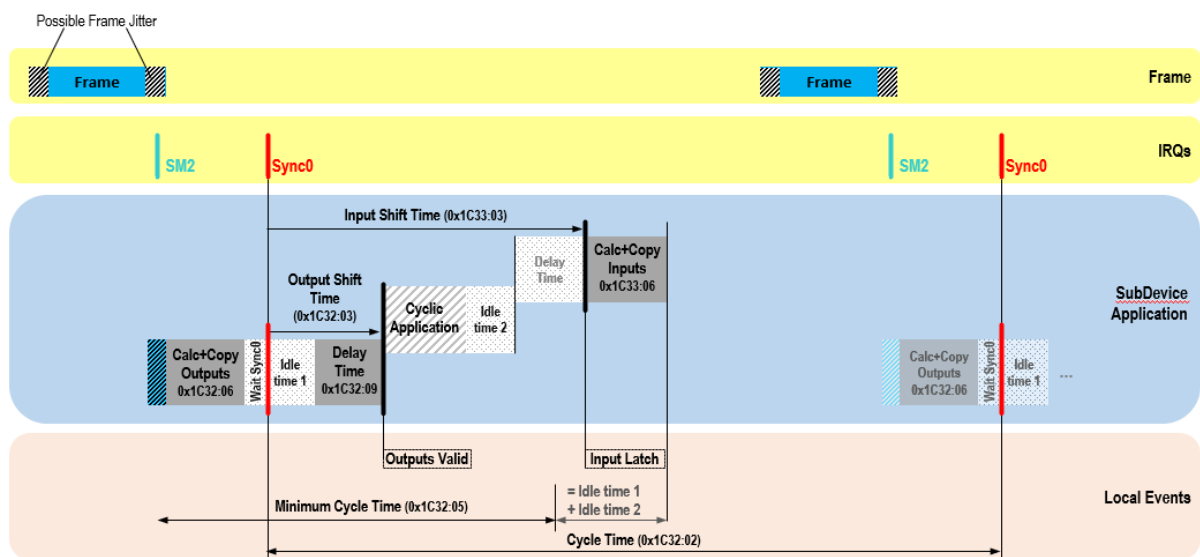
SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x02: DC Sync0 – synchronized with Sync0 Event
2	Cycle Time	R	O	Same value as 0x1C32:02
3	Shift Time	--	--	
4	Synchronization Types supported	R	M	Same value as 0x1C32:04
5	Minimum Cycle Time	R	M	Same value as 0x1C32:05
6	Calc and Copy Time	R	M	
7	--	--	--	
8	Get Cycle Time	RW	C	Same value as 0x1C32:08
9	Delay Time	--	--	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed	R	O	Same value as 0x1C32:0B
12	Cycle Time Too Small	R	M	Same value as 0x1C32:0C
13	Shift Time Too Short	R	O	Same value as 0x1C32:0D
14	RxPDO Toggle Failed	R	O	Same value as 0x1C32:0E
31:15	--	--	--	
32	Sync Error	R	C	Same value as 0x1C32:20

23.1.1.5.1.2 Shift of Outputs Valid and/or Input Latch

The Outputs Valid can be delayed by use of the Shift Time (0x1C32:03). The Shift Time describes the time between Sync0 and Outputs Valid.

The Input Latch can be delayed by the use of the Shift Time (0x1C33:03). In this case the Shift Time describes the time between Sync0 and the Input Latch.

SM2 Event shall be received prior to the Sync0 Event.


 Figure 28: Local Cycle synchronous to Sync0 Event, Shift of Outputs/ Inputs with support of SM2 event

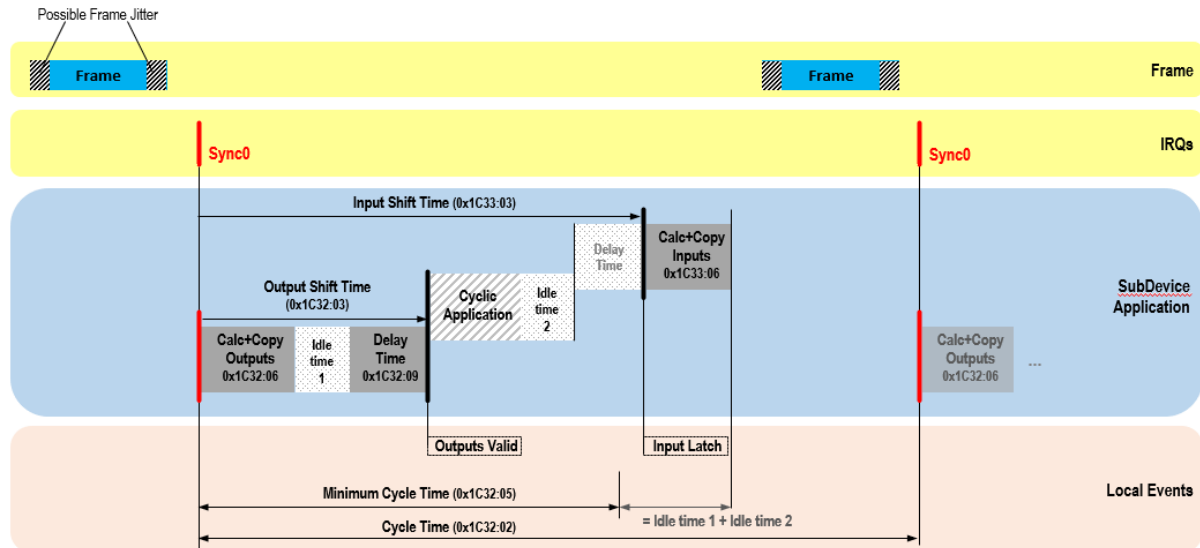


Figure 29: Local Cycle synchronous to Sync0 Event, Shift of Outputs/ Inputs without support of SM2 event

Table 98 and Table 99 explain how those objects shall be used in DC Mode, Shift of Outputs/ Inputs.

Table 98: 0x1C32 DC Mode, Shift of Outputs/ Inputs

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x02: DC Sync0 - synchronized with Sync0 Event
2	Cycle Time	R	M	Sync0 Cycle Time (register 0x09A3:0x09A0) Time between two Sync0 events. Sync0 Cycle Time shall be copied to this entry.
3	Shift Time	RW	M	Time between Sync0 event and Outputs Valid (initialized with 0x1C32:09).
4	Synchronization Types supported	R	M	Bit 4:2 : DC Type supported: 001 = DC Sync0 Bit 5-6: Shift Settings 01 = Output Shift with local timer (Shift Time)
5	Minimum Cycle Time	R	M	
6	Calc and Copy Time	R	M	Minimum Time between frame and Sync0
7	Minimum Delay Time	--	--	
8	Get Cycle Time	RW	C	Mandatory if Subindex 4 Bit 14 is 1
9	Delay Time	R	M	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed	R	O	
12	Cycle Time Too Small	R	M	
13	Shift Time Too Short	R	O	Should be supported in DC Mode, i.e. if SI4 bit 5 or bit 6 is true.
14	RxPDO Toggle Failed	R	O	
15	Minimum cycle distance	R	O	
16	Maximum cycle distance	R	O	
17	Minimum SM SYNC distance	R	O	
18	Maximum SM SYNC distance	R	O	
31:19	--	--	--	

SubIndex	Description	Access	Use	Description / Default value
32	Sync Error	R	C	Shall be supported if SM-Event Missed Counter (SI11) or Cycle Time Too Small Counter (SI12) or the Shift Time Too Short Counter (SI13) is supported

Table 99: 0x1C33 DC Mode, Shift of Outputs/ Inputs

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x02: DC Sync0 – synchronized with Sync0 Event
2	Cycle Time	R	M	Same value as 0x1C32:02
3	Shift Time	RW	M	Time between Sync0 and Input Latch (initialized with 0x1C33:05+0x1C32:06)
4	Synchronization Types supported	R	M	Bit 4:2 : DC Type supported: 001 = DC Sync0 Bit 5-6: Shift Settings 01 = Input Shift with local timer (Shift Time)
5	Minimum Cycle Time	R	M	Same value as 0x1C32:05
6	Calc and Copy Time	R	M	
7	--	--	--	
8	Get Cycle Time	RW	C	Same value as 0x1C32:08
9	Delay Time	--	--	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed	R	O	Same value as 0x1C32:0B
12	Cycle Time Too Small	R	M	Same value as 0x1C32:0C
13	Shift Time Too Short	R	O	Same value as 0x1C32:0D
14	RxPDO Toggle Failed	R	O	Same value as 0x1C32:0E
31:15	--	--	--	
32	Sync Error	R	C	Same value as 0x1C32:20

23.1.1.5.1.3 Shift of Input Latch with Sync1

The Input Latch can be delayed by the use of the Sync1 event (0x1C33:01 = 0x03). In this case the Sync1 Cycle Time describes the time between Sync0 and Start Latch. The Delay Time (1C33:09) contains the hardware delay of the latching.

In case the Process Data Frame (Frame) is processed in the SubDevice with the help of SM2 Event before the next Sync0 Event, the Calc + Copy Time contains the minimum time difference between receiving of the frame (SM2 Event) and the Sync0 Event.

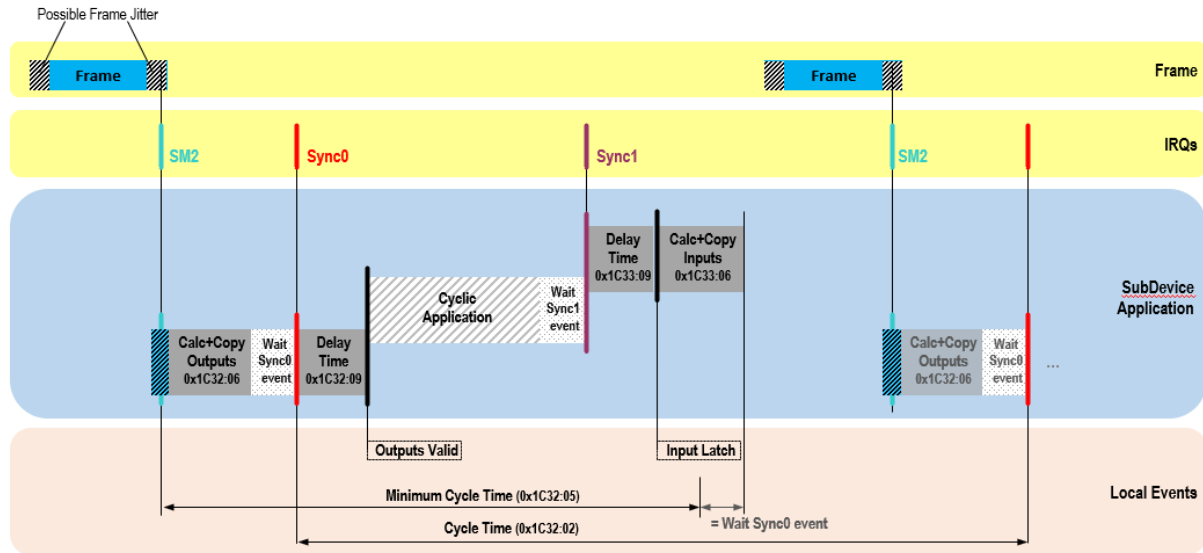


Figure 30: Local Cycle synchronous to Sync0 Event, Shift of Input Latch with Sync1 with support of SM2 event

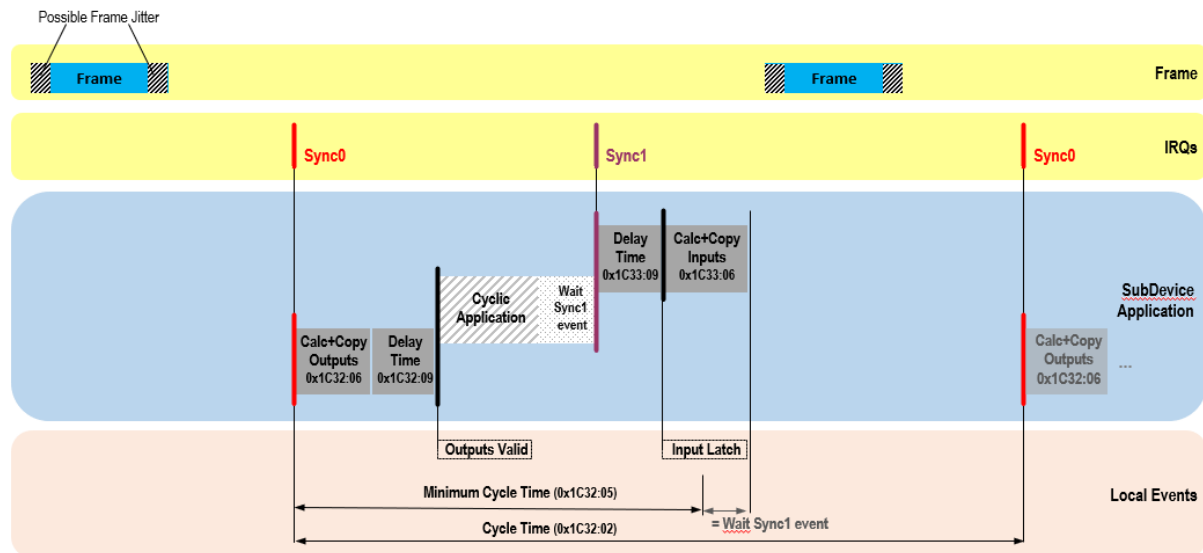


Figure 31: Local Cycle synchronous to Sync0 Event, Shift of Input Latch with Sync1 without support of SM2 event

Table 100 and Table 101 explain how those objects shall be used in DC Mode, Shift of Input Latch with Sync1.

Table 100: 0x1C32 DC Mode, Shift of Input Latch with Sync1

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x02: DC Sync0 - synchronized with Sync0 Event
2	Cycle Time	R	M	Sync0 Cycle Time (register 0x09A3:0x09A0) Time between two Sync0 events. Sync0 Cycle Time shall be copied to this entry.
3	Shift Time	--	--	
4	Synchronization Types supported	R	M	Bit 4:2 : DC Type supported: 001 = DC Sync0 Bit 5: Shift Settings 0 = No Output Shift supported

SubIndex	Description	Access	Use	Description / Default value
5	Minimum Cycle Time	R	M	
6	Calc and Copy Time	R	M	Minimum Time between frame and Sync0
7	Minimum Delay Time	--	--	
8	Get Cycle Time	RW	C	Mandatory if Subindex 4 Bit 14 is 1
9	Delay Time	R	M	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed	R	O	
12	Cycle Time Too Small	R	M	
13	Shift Time Too Short	R	O	
14	RxPDO Toggle Failed	R	O	
15	Minimum cycle distance	R	O	
16	Maximum cycle distance	R	O	
17	Minimum SM SYNC distance	R	O	
18	Maximum SM SYNC distance	R	O	
31:19	--	--	--	
32	Sync Error	R	C	Shall be supported if SM-Event Missed Counter (SI11) or Cycle Time Too Small Counter (SI12) or the Shift Time Too Short Counter (SI13) is supported

Table 101: 0x1C33 DC Mode, Shift of Input Latch with Sync1

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x03: DC Sync1 - synchronized with Sync1 Event
2	Cycle Time	R	M	Same value as 0x1C32:02
3	Shift Time	--	--	
4	Synchronization Types supported	R	M	Bit 4:2 : DC Type supported: 010 = DC Sync1 Bit 6:5: Shift Settings 10 = Input Shift with Sync1
5	Minimum Cycle Time	R	M	Same value as 0x1C32:05
6	Calc and Copy Time	R	M	
7	--	--	--	
8	Get Cycle Time	RW	C	Same value as 0x1C32:08
9	Delay Time	R	M	Time difference between Start Latch and Latch
10	Sync0 Cycle Time	--	--	
11	SM-Event missed	R	O	Same value as 0x1C32:0B
12	Cycle Time Too Small	R	M	Same value as 0x1C32:0C
13	Shift Time Too Short	R	O	Same value as 0x1C32:0D
14	RxPDO Toggle Failed	R	O	Same value as 0x1C32:0E
31:15	--	--	--	
32	Sync Error	R	C	Same value as 0x1C32:20

23.1.1.5.2 DC Mode (Synchronous with Sync1 Event)

The local cycle is started when the Sync0 event is received. The Process Data Frame (Frame) has to be received before the Sync0 event. The Sync1 is used for Outputs Valid (or Input Latch if no outputs are transmitted), so the Sync1 Cycle Time defines the time difference between Sync0 and Start Driving Outputs. The Calc+Copy Time (1C32:06) is the minimum allowed time for the Sync1 Cycle Time, the Delay Time (1C32:09) defines the hardware delay for driving the outputs.

SM2 Event shall be received prior to the Sync0 Event.

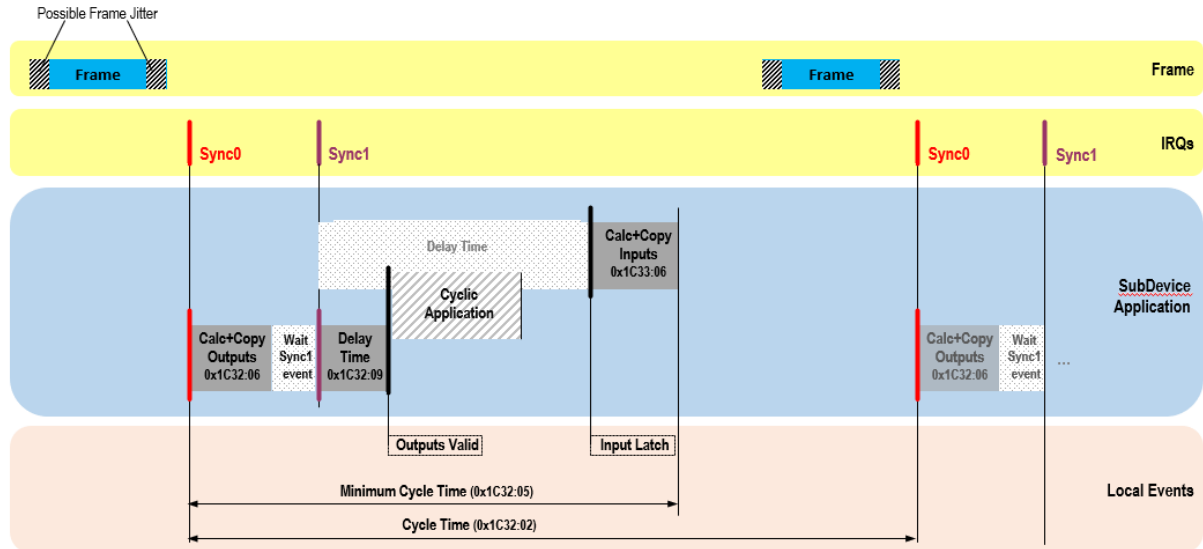


Figure 32: Local Cycle synchronous to Sync1 Event

Table 102 and Table 103 explain how those objects shall be used in DC Mode (Sync1).

Table 102: 0x1C32 DC Mode (Sync1)

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x03: DC Sync1 - synchronized with Sync1 Event
2	Cycle Time	R	O	Sync0 Cycle Time (register 0x09A3:0x09A0) Time between two Sync0 events. Sync0 Cycle Time shall be copied to this entry.
3	Shift Time	--	--	
4	Synchronization Types supported	R	M	Bit 4:2 : DC Type supported: 010 = DC Sync1
5	Minimum Cycle Time	R	M	
6	Calc and Copy Time	R	M	Minimum Time between Sync0 and Start Driving Outputs (minimum Sync1 Cycle Time)
7	Minimum Delay Time	--	--	
8	Get Cycle Time	RW	C	Mandatory if Subindex 4 Bit 14 is 1
9	Delay Time	R	M	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed Counter	R	O	
12	Cycle Time Too Small	R	M	
13	Shift Time Too Short Counter	R	O	incremented if the distance between SYNC0-event and SYNC1-event is too short (configuration problem)
14	RxPDO Toggle Failed	R	O	
15	Minimum cycle distance	R	O	
16	Maximum cycle distance	R	O	

SubIndex	Description	Access	Use	Description / Default value
17	Minimum SM SYNC distance	R	O	
18	Maximum SM SYNC distance	R	O	
31:19	--	--	--	
32	Sync Error	R	C	Shall be supported if SM-Event Missed Counter (SI11) or Cycle Time Too Small Counter (SI12) or Shift Time Too Short Counter (SI13) are supported

Table 103: 0x1C33 DC Mode (Sync1)

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	R	M	0x03: DC Sync1 - synchronized with Sync1 Event
2	Cycle Time	R	O	Same value as 0x1C32:02
3	Shift Time	--	--	
4	Synchronization Types supported	R	M	Same value as 0x1C32:04
5	Minimum Cycle Time	R	M	Same value as 0x1C32:05
6	Calc and Copy Time	R	M	
7	--	--	--	
8	Get Cycle Time	RW	C	Same value as 0x1C32:08
9	Delay Time	--	--	
10	Sync0 Cycle Time	--	--	
11	SM-Event missed	R	O	Same value as 0x1C32:0B
12	Cycle Time Too Small	R	M	Same value as 0x1C32:0C
13	Shift Time Too Short	--	--	
14	RxPDO Toggle Failed	R	O	Same value as 0x1C32:0E
31:15	--	--	--	
32	Sync Error	R	C	Same value as 0x1C32:20

23.1.1.6 DC, subordinated application controller cycles

23.1.1.6.1 General

For SubDevices with fast local cycle times (e.g. control loops) the cycle time of the application controller might be much faster than the communication cycle time. In this case two synchronization features are distinguished:

1. Shift of Outputs Valid
2. Shift of Input Latch

If a subordinated drive cycle is used with DC, Sync1 shall be used for the communication cycle time of the MainDevice. Output Shift and Input Shift shall be related to Sync1. Sync0 can be used for the subordinated internal cycle.

23.1.1.6.2 DC, subordinated μ C cycles, Shift of Outputs Valid and/or Input Latch

The Sync0 signal is only used to trigger the local microcontroller cycle. The Outputs Valid and Input Latch are triggered by the Sync1 Event and can only be shifted by the Output Shift Time and Input Shift Time.

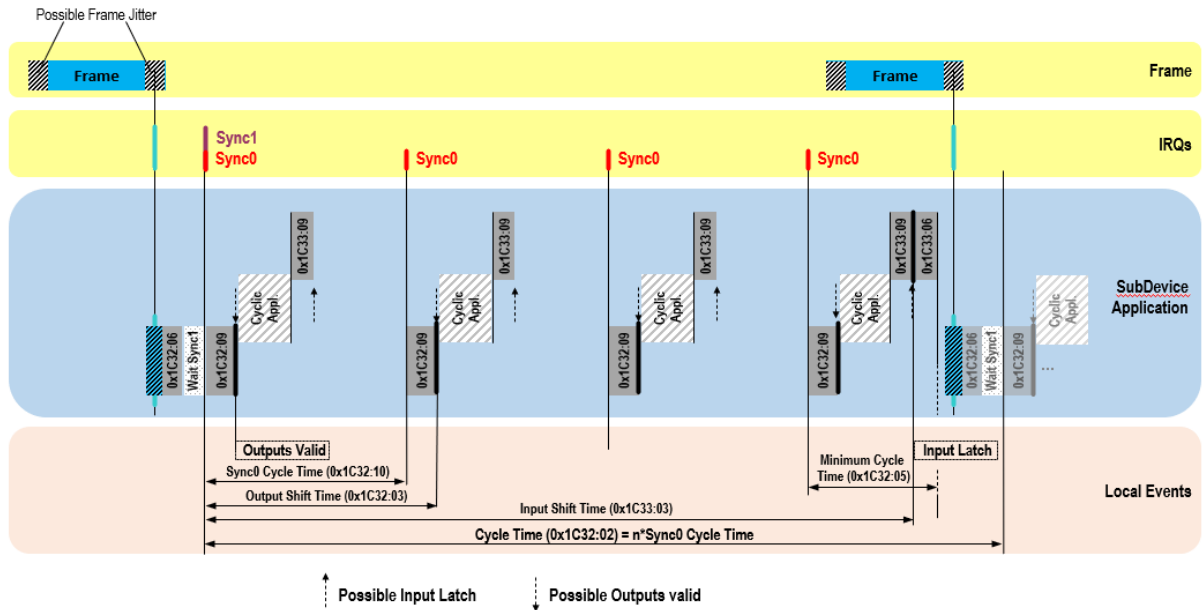


Figure 33: DC, subordinated µC cycles, Outputs/ Inputs delayed

Table 104 and Table 105 explain how those objects shall be used in DC Mode, subordinated µC cycles, Outputs Valid/ Input Latch shifted.

Table 104: 0x1C32 DC Mode, subordinated µC cycles, Shift Outputs/ Inputs

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x03: DC Sync1 – synchronized with Sync1 Event
2	Cycle Time	R	M	Sync1 Cycle Time (register 0x09A7:0x09A4) $0x09A3:0x09A0 \times \left[\text{DIV} \left(\frac{0x09A7:0x09A4}{0x09A3:0x09A0} \right) + 1 \right]$ Time between two Sync1 events Sync1 Cycle Time shall be copied by the SubDevice application.
3	Shift Time	RW	O	
4	Synchronization Types supported	R	M	Bit 4:2 : DC Type supported: 100 = Subordinated Application Bit 6:5: Shift Settings 00 = No Output Shift supported 01 = Output Shift with local timer (Shift Time)
5	Minimum Cycle Time	R	M	
6	Calc and Copy Time	R	M	
7	--	--	--	
8	Get Cycle Time	RW	C	Mandatory if Subindex 4 Bit 14 is 1
9	Delay Time	R	M	
10	Sync0 Cycle Time	R	M	
11	SM-Event missed	R	O	
12	Cycle Time Too Small	R	M	
13	Shift Time Too Short	R	O	Should be supported in DC Mode, i.e. if SI4 bit 5 or bit 6 is true.
14	RxPDO Toggle Failed	R	O	
15	Minimum cycle distance	R	O	
16	Maximum cycle distance	R	O	

SubIndex	Description	Access	Use	Description / Default value
17	Minimum SM SYNC distance	R	O	
18	Maximum SM SYNC distance	R	O	
31:19	--	--	--	
32	Sync Error	R	C	Shall be supported if SM-Event Missed Counter (SI11) or Cycle Time Too Small Counter (SI12) or the Shift Time Too Short Counter (SI13) is supported

Table 105: 0x1C33 DC Mode, subordinated µC cycles, Shift Outputs/ Inputs

SubIndex	Description	Access	Use	Description / Default value
1	Synchronization Type	RW	M	0x03: DC Sync1 – synchronized with Sync1 Event
2	Cycle Time	R	M	Same value as 0x1C32:02
3	Shift Time	R	O	
4	Synchronization Types supported	R	M	Bit 4:2 : DC Type supported: 100 = Subordinated Application Bit 6:5: Shift Settings 00 = No Output Shift supported 01 = Output Shift with local timer (Shift Time)
5	Minimum Cycle Time	R	M	Same value as 0x1C32:05
6	Calc and Copy Time	R	M	
7	--	--	--	
8	Get Cycle Time	RW	C	Same value as 0x1C32:08
9	Delay Time	R	M	
10	Sync0 Cycle Time	R	M	Same value as 0x1C32:0A
11	SM-Event missed	R	O	Same value as 0x1C32:0B
12	Cycle Time Too Small	R	M	Same value as 0x1C32:0C
13	Shift Time Too Short	R	O	Same value as 0x1C32:0D
14	RxPDO Toggle Failed	R	O	Same value as 0x1C32:0E
31:15	--	--	--	
32	Sync Error	R	C	Same value as 0x1C32:20

23.1.2 Objects 0x1C32 and 0x1C33: SyncManager Parameter objects

Table 106 defines the SyncManager Parameter object.

Table 106: 0x1C32 / 0x1C33 SyncManager Parameter objects

Attribute	Value
Index	0x1C32, 0x1C33
Name	SyncManager Parameter
Max SubIndex	32
Category	Conditional Optional if only Free Run mode supported
PDO Mapping	No

The SyncManager Parameter Objects are defined in Table 107 and Table 108.

If no outputs are available, the object 0x1C32 does not exist.

If no outputs are transmitted in SAFE-OP and OP (Sync Manager Channel 2 is disabled), the Subindex 0 of object 0x1C32 shall return 0 in SAFE-OP and OP. The other subindexes shall return the Abort-Code 0x06090011 (Subindex does not exist).

Table 107: Output SyncManager Parameter (0x1C32)

Sub-Index	Description	Data Type	Access	Use	Description / Default value
1	Synchronization Type	UNSIGNED16	R or RW	M	0x00: Free Run (not synchronized) 0x01: SM-Synchronous – synchronized with SM Event 0x02: DC Sync0 – synchronized with Sync0 Event 0x03: DC Sync1 – synchronized with Sync1 Event NOTE: Entry only writable with new value in SafeOp or OP if Synchronization can be changed in those states
2	Cycle Time	UNSIGNED32	R or RW	O	Free Run (Synchronization Type = 0x00): Time between two local timer events in ns Synchronous with SM2 (Synchronization Type = 0x01): Minimum time between two SM2 events in ns DC Sync0 (Synchronization Type = 0x02): Sync0 Cycle Time (Register 0x9A3-0x9A0) in ns NOTE: Entry only writable with new value in SafeOp or OP if Synchronization can be changed in those states
3	Shift Time	UNSIGNED32	R or RW	O	Time between related event and the associated action (outputs valid hardware) in ns Shift of Output valid equal or greater than 0x1C32:09 NOTE: Entry only writable with new value in SafeOp or OP if Synchronization can be changed in those states

Sub-Index	Description	Data Type	Access	Use	Description / Default value
4	Synchronization Types supported	UNSIGNED16	R	M	Bit 0: Free Run supported Bit 1: SM Synchronous supported Bit 4:2 : DC Type supported: 000 = No DC 001 = DC Sync0 010 = DC Sync1 100 = Subordinated Application with fixed Sync0 Bit 6:5: Shift Settings 00 = No Output Shift supported 01 = Output Shift with local timer (Shift Time) 10 = Output Shift with Sync1 Bit 9...6: Reserved for future use Bit 10: Delay Times should be measured (because they depend on the configuration) Bit 11: Delay Time is fix (synchronization is done by hardware) Bit 13...11: Reserved for future use Bit 14: Dynamic Cycle Times Times described in 0x1C32 are variable (depend on the actual configuration) This is used for e.g. EtherCAT gateway devices. The SubDevice shall support cycle time measurement in OP state. The cycle time measurement is started by writing 1 to 0x1C32:08. If this bit is set, the default values of the times to be measured (Minimum Cycle Time, Calc And Copy Time, Delay Time) could be 0. The default values could be set in INIT and PREOP state. Bit 14 should only be set, if the SubDevice cannot calculate the cycle time after receiving all Start-up SDO in transition PS. Bit 15: Reserved for future use
5	Minimum Cycle Time	UNSIGNED32	R	C	Minimum cycle time supported by the SubDevice (maximum duration time of the local cycle) in ns It might be necessary to start the Dynamic Cycle Time measurement SI04, Bit 14 and SI08, Bit 0 to get a valid value used in Synchronous or DC Mode
6	Calc and Copy Time	UNSIGNED32	R	C	Time needed by the application controller to copy the process data from the Sync Manager to the local memory and perform calculations if necessary before the data is sent to the process. Minimum time for Outputs to SYNC-Event used in DC mode
7	Minimum Delay Time	UNSIGNED32	R	O	Only important for DC Sync0/1 (Synchronization type = 0x02 or 0x03): Minimum Hardware delay time of the SubDevice. because of software synchronization there could be a distance between the minimum and the maximum delay time Distance between minimum and maximum delay time (Subindex 9) shall be smaller than 1 μs. if SubIndex 4, Bit 11 is 1, this SubIndex contains the value of SubIndex 9

Sub-Index	Description	Data Type	Access	Use	Description / Default value
8	Get Cycle Time	UNSIGNED16	RW	C	Bit 0: 0: Measurement of local cycle time stopped 1: Measurement of local cycle time started. If written again, the measured values are reset Used in Synchronous or (DC Mode with variable Cycle Time) Bit 1: 0: -- 1: Reset the error counters Bit 2...15: reserved for future use
9	Delay Time	UNSIGNED32	R	C	Only important for DC Sync0/1 (Synchronization type = 0x02 or 0x03): Hardware delay time of the SubDevice. Time from receiving the trigger (Sync0 or Sync1 Event) to drive output values to the time until they become valid in the process (e.g. electrical signal available). if Subindex 7 Minimum Delay Time is supported and unequal 0, this Delay Time is the Maximum Delay Time
10	Sync0 Cycle Time	UNSIGNED32	RW/R	O	Only important for DC Sync0 (Synchronization type = 0x03) and subordinated local cycles: Time between two Sync0 signals if fixed Sync0 Cycle Time is needed by the application
11	SM-Event Missed Counter	UNSIGNED16	R	O	This error counter is incremented when the application expects a SM event but does not receive it in time and as consequence the data cannot be copied any more. Should be supported in DC Mode
12	Cycle Time Too Small Counter	UNSIGNED16	R	C	This error counter is incremented when the cycle time is too small so that the local cycle cannot be completed and input data cannot be provided before the next SM event. used in Synchronous or DC Mode
13	Shift Time Too Short Counter	UNSIGNED16	R	C	This error counter is incremented when the time distance between the trigger (Sync0) and the Outputs Valid is too short because of a too short Shift Time or Sync1 Cycle Time. Should be supported in DC Mode, i.e. if SI4 bit 5 or bit 6 is true.
14	RxPDO Toggle Failed Counter	UNSIGNED16	R	O	This error counter is incremented when the SubDevice supports the RxPDO Toggle and does not receive new RxPDO data from the MainDevice (RxPDO Toggle set to TRUE)
15	Minimum Cycle Distance	UNSIGNED32	R	O	Minimum Cycle Distance in ns used in conjunction with SI16 to monitor the jitter between two SM-events
16	Maximum cycle distance	UNSIGNED32	R	O	Maximum cycle distance in ns used in conjunction with SI15 to monitor the jitter between two SM-events
17	Minimum SM SYNC Distance	UNSIGNED32	R	O	Minimum SM SYNC Distance in ns used in conjunction with SI18 to monitor the jitter between the SM-event and SYNC0-event in DC-SYNC0-Mode
18	Maximum SM SYNC Distance	UNSIGNED32	R	O	Maximum SM SYNC Distance in ns used in conjunction with SI17 to monitor the jitter between the SM-event and SYNC0-event in DC-SYNC0-Mode
31:19	--	--	--		Reserved for future use.
32	Sync Error	BOOL	R	C	Shall be supported if SM-Event Missed Counter (SI11) or Cycle Time Too Small Counter (SI12) or Shift Time Too Short Counter (SI13) are supported Mappable in TxPDO 0: no Synchronization Error or Sync Error not supported 1: Synchronization Error

Table 108 describes the settings for the Input SyncManager.

If no inputs are available, the object 0x1C33 does not exist.

If no inputs are transmitted in SAFE-OP and OP (Sync Manager Channel 3 is disabled), the Subindex 0 of object 0x1C33 shall return 0 in SAFE-OP and OP. The other subindexes shall return the Abort-Code 0x06090011 (Subindex does not exist).

Table 108: Input SyncManager Parameter (0x1C33)

Sub-Index	Description	Data Type	Access	Use	Description / Default value
1	Synchronization Type	UNSIGNED16	R or RW	M	Synchronization Type according to 0x1C32:01 0x00: Free Run (not synchronized) 0x01: SM-Synchronous with SM3 Event 0x02: DC Sync0 – synchronized with Sync0 Event 0x03: DC Sync1 – synchronized with Sync1 Event 0x22: Synchronous with SM2 Event NOTE: Entry only writable with new value in SafeOp or OP if Synchronization can be changed in those states
2	Cycle Time	UNSIGNED32	R	O	Same time as for 0x1C32:02
3	Shift Time	UNSIGNED32	R or RW	O	Time between related event and the associated action (inputs latched from hardware) in ns shift of Input Latch equal or greater than 0x1C33:09 NOTE: Entry only writable with new value in SafeOp or OP if Synchronization can be changed in those states
4	Synchronization Types supported	UNSIGNED16	R	M	Bit 0: Free Run supported Bit 1: SM-Synchronous supported Bit 4:2 : DC Type supported: 000 = No DC 001 = DC Sync0 010 = DC Sync1 100 = Subordinated Application with fixed Sync0 Bit 6:5: Shift Settings 00 = No Input Shift supported 01 = Input Shift with local timer (Shift Time) 10 = Input Shift with Sync1 Bit 13:6: Reserved for future use Bit 14: Dynamic Cycle Times Same meaning as in 0x1C32:04 Bit 15: Reserved for future use
5	Minimum Cycle Time	UNSIGNED32	R	C	Same time as 0x1C32:05 used in SM-Synchronous or DC-Synchronous Mode
6	Calc and Copy Time	UNSIGNED32	R	C	Time in ns needed by the application controller to perform calculations on the input values if necessary and to copy the process data from the local memory to the Sync Manager before the data is available for EtherCAT. Minimum time for Inputs after Input Latch used in DC-Synchronous mode
7	--	UNSIGNED32	--		Reserved for future use
8	Get Cycle Time	UNSIGNED16	RW	C	Same meaning as 0x1C32:08 used in SM-Synchronous or DC-Synchronous Mode (with variable Cycle Time)
9	Delay Time	UNSIGNED32	R	C	Only important for DC Sync1 (Synchronization type = 3), if the Input Latch is started by the Sync1-event Hardware delay time of the SubDevice. used in DC-Synchronous mode
10	Sync0 Cycle Time	UNSIGNED32	R	C	Same value as 0x1C32:0A used in Synchronous or DC Mode

Sub-Index	Description	Data Type	Access	Use	Description / Default value
11	SM-Event Missed Counter	UNSIGNED16	R	O	Same value as 0x1C32:0B
12	Cycle Time Too Small Counter	UNSIGNED16	R	C	Same value as 0x1C32:0C
13	Shift Time Too Short Counter	UNSIGNED16	R	O	Same value as 0x1C32:0D
14	reserved	UNSIGNED16			reserved
31:15	--	--	--		Reserved for future use.
32	Sync Error	BOOL	R	C	Same value as 0x1C32:20

23.1.3 Diagnosis of the Synchronization Mode

A cycle synchronous diagnosis can be done by setting the sync manager channels 2 and 3 in 1-buffer mode. In that case the working counter would not match if a sync error had happened.

The disadvantage of the 1 buffer mode is that all data of an EtherCAT datagram is invalid if a synchronization failure happens. Alternatively the PDO-Toggle can be used:

With the PDO-Toggle the sync manager should work in 3-buffer mode that a synchronization failure will not influence the data of other SubDevices. The corresponding toggle bit can be checked by the MainDevice and SubDevice.

If the SubDevice detects a synchronization error this information shall be reported in the Sync Error Entry (0x1C32:20, 0x1C33:20).

So the MainDevice could detect synchronization problems cycle synchronous by checking the PDO Toggle and the Sync Error.

23.1.3.1 Error Behaviour of the SubDevice

Table 109 shows the error behavior of the SubDevice.

Table 109: Error Behaviour of the SubDevice

Error Type	SubDevice Behaviour
Communication Cycle Time (0x1C32:02) < Minimum Cycle Time (0x1C32:05)	After the MainDevice has written the cycle time via an SDO service the SubDevice could check with the Minimum Cycle Time. If the Cycle Time is smaller than the Minimum Cycle Time the SubDevice should return an Abort (Abort Code 0x06090032, Value too short) on this SDO service. In DC Mode the Communication Cycle Time will be overwritten with the Sync0 Cycle Time (Register 0x9A3:0x9A0). The SubDevice should refuse the state transition from PREOP to SAFEOP with AL Status Code 0x36 (or 0x35) if the Sync0 Cycle Time (Register 0x9A3:0x9A0) is too small (or too great). If the Minimum Cycle Time is variable the SubDevice might only check the timing during SAFEOP or OP. In that case the SubDevice behaves like described for Error Type "too short interval between two sync events"
Sync1 used for Outputs Valid and Sync1 Cycle Time < Output Calc And Copy Time (0x1C32:06) or Sync1 used for Input Latch and Sync1 Cycle Time < (Input Shift Time (0x1C33:03) - Input Delay Time (0x1C33:09))	The SubDevice should refuse the state transition from PREOP to SAFEOP with AL Status Code 0x37 (or 0x35) if the Sync1 Cycle Time (Register 0x9A7:0x9A4) is too small (or too great). If the Times are variable the SubDevice might only check the timing during SAFEOP or OP. In that case the SubDevice behaves like described for Error Type "distance between Sync0 and Sync1 signal too short"
Invalid sync or latch configuration	The SubDevice should refuse the state transition from PREOP to SAFEOP with AL Status Code 0x30 or 0x31 if the sync or latch configuration does not match.
Interval between two sync events too short (corresponds to Synchronization Monitoring 1 in Table 79)	If the SubDevice detects a too short interval between two SM2/3 Events it should increment the Cycle Time Too Small Counter (0x1C3x:0C) and acknowledge the SM2/(3) event and wait for the next SM2/(3) event. The MainDevice could detect this error by checking the PDO Toggle. If the Sync Error Counter Limit exceeds, the SubDevice shall leave its state to SAFEOP and set the AL Status Code to 0x33 (or 0x1A) (See also 0x10F1:02 in Table 71). For the SAFEOP2OP-transition the AL-Status-Code 0x32 should be generated in this case.
No SM2 Event before Sync0 event (only for DC Modes) (corresponds to Synchronization Monitoring 3 in Table 79)	If the SubDevice detects that no SM2 event was received when the Sync0 event was received, the SM-Event Missed Counter (0x1C3x:0B) shall be incremented and the Sync Error Flag (0x1C32:20) shall be set. If the Sync Error Counter Limit exceeds, the SubDevice shall leave its state to SAFEOP and set the AL Status Code to 0x34 (or 0x1A) (See also 0x10F1:02 in Table 71).
Distance between Sync0 and Sync1 signal too short (only for DC Modes) (corresponds to Synchronization Monitoring 4 in Table 79)	If the SubDevice detects a too short distance between Sync0 and Sync1 signal, the Shift Too Short Counter (0x1C3x:0D) shall be incremented and the Sync Error Flag (0x1C32:20) shall be set. If the Sync Error Counter Limit exceeds, the SubDevice shall leave its state to SAFEOP and set the AL Status Code to 0x33 (or 0x1A) (See also 0x10F1:02 in Table 71).
PLL could not be synchronized any more (corresponds to Synchronization Monitoring 2 in Table 79)	If the SubDevice has a PLL is used to synchronize its local cycle to the communication cycle and the PLL be could be synchronized any more, the SubDevice shall leave its state to SAFEOP and set the AL Status Code to 0x32 (or 0x1A).
no Sync0/Sync1 signal any more (only for DC Modes) If the SubDevice detects in OP that no Sync0 or Sync1 event is generated any more it shall leave its state to SAFEOP and set the AL Status Code to 0x2C (or 0x1A).	no Sync0/Sync1 signal any more (only for DC Modes) If the SubDevice detects in OP that no Sync0 or Sync1 event is generated any more it shall leave its state to SAFEOP and set the AL Status Code to 0x2C (or 0x1A).
Timeout by waiting for first Sync0/Sync1 signal (only for DC Modes)	If the SubDevice gets a timeout by waiting for the Sync0/Sync1-event in SAFEOP it shall refuse the state transition to OP and set the AL Status Code to 0x2D (or 0x1A).

23.2 Synchronization in the MainDevice

The MainDevice normally has two different synchronization modes:

- Cyclic Mode
- DC Mode

23.2.1 Cyclic Mode

In Cyclic Mode the MainDevice should send the process data frames cyclically. The process data frames may be sent with different cycle times. The MainDevice cycle is usually controlled by a local timer on the MainDevice. The SubDevices may be operated in Free Run or synchronous with the SM event mode. For SubDevices running in synchronous mode the MainDevice should check that the cycle time of the corresponding process data frames is greater than the Minimum Cycle Time of the SubDevices (0x1C3x:05 Sync Manager Parameter).

23.2.2 DC Mode, MainDevice follows Reference Clock

In DC Mode the MainDevice is behaving like in Cyclic Mode but the local cycle should be synchronized with the MainDevice Clock. The local timer of the MainDevice should be adjusted with the ARMW datagram which distributes the MainDevice clock.

In DC Mode usually all SubDevices supporting DC should be synchronized to the DC Base Time. The MainDevice should synchronize its cycle to this DC Base Time, too.

Figure 34 shows how the local cycle of the MainDevice may be synchronized to the DC Base Time.

DC Base

The DC Base Time is a virtual time that is used to set the MainDevice cycle and communication cycle in to relation to the reference clock. The DC Base Time and the reference time have a fixed relation.

Local Timer

Time of MainDevice that is adjusted to the MainDevice clock (reference time)

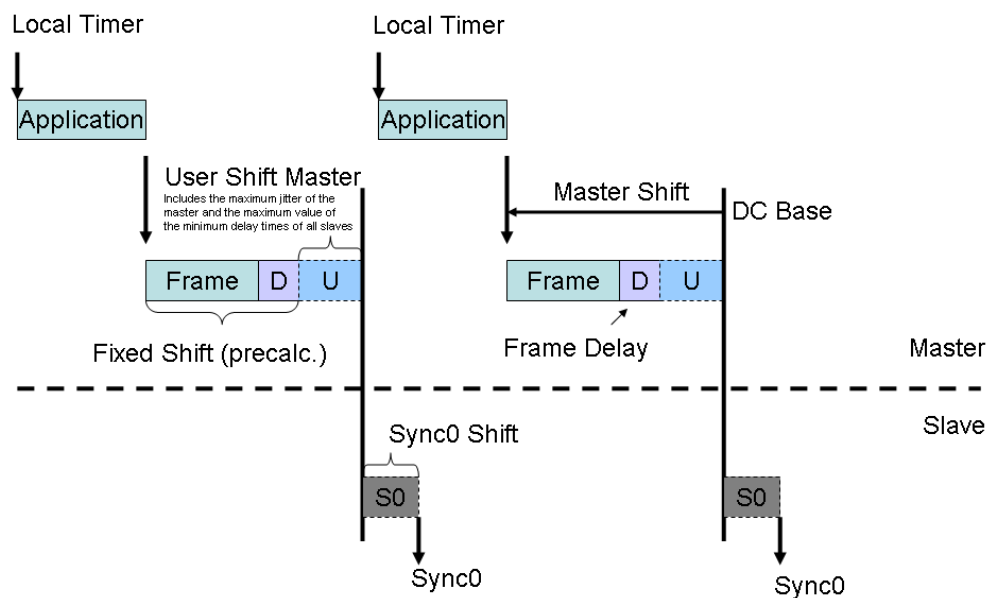
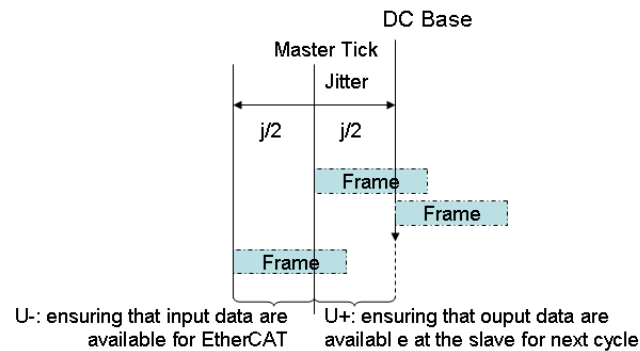


Figure 34: MainDevice synchronized to DC Base

The local application is started with a local timer. The local timer is shifted to the DC Base Time by the sum of the following times:

- Duration of the application execution time (Application)
- Frame transmission time (Frame)
- Frame transmission delay (D)

- User Shift (U) which shall include the maximum of the minimum delay times of the SubDevices and the maximum jitter of the execution of the application.
U+: positive User Shift as shown in Figure 35
U-: negative User Shift as shown in Figure 35



Master Tick: Tick on which master sends the frame

Figure 35: User Shift

The cycle time of the local timer will be controlled by reading the system time (register 0x910-0x913) of the MainDevice Clock with an ARMW command cyclically.

The Sync0 event of each SubDevice may be shifted to the DC Base Time by writing the Start Time of the cyclic Operation (Register 0x990-0x993) for the SubDevices.

MainDevice Settings for use of DCs

The DC Base signal is a virtual signal used as timing reference. Usually this should be synchronous with the Start time of the DC Reference Clock.

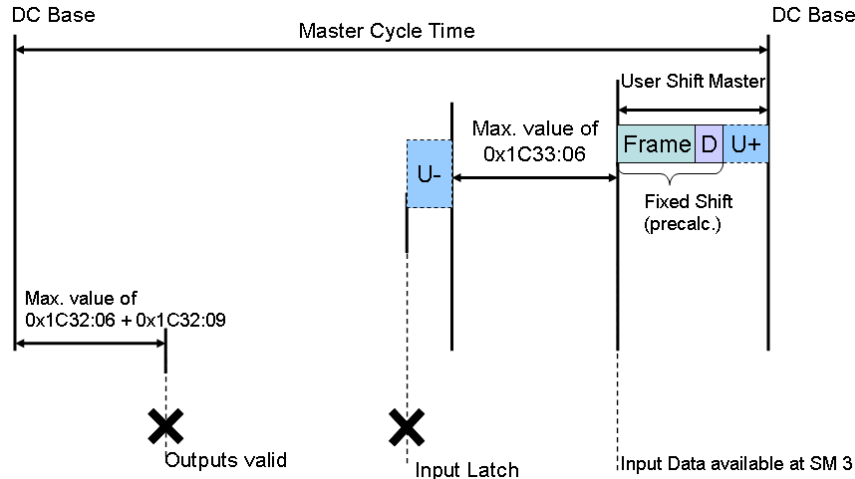


Figure 36: MainDevice Settings for use of DCs

Max value of Calc and Copy Time (0x1C32:06) plus Output Delay Time (0x1C32:09) of all DC SubDevices shall be used as reference to set the Outputs Valid.

The maximum Shift Time for each SubDevice can be $\text{MaxShiftTime} = \text{MainDevice Cycle Time} - \text{User Shift MainDevice} - \text{Calc And Copy Time} - U-$

MainDevice Settings for use of DCs with SubDevice related Settings

Figure 37 and Table 110 describes how the settings handled by the MainDevice are used for two different synchronization cases (Slv 1, Slv 2).

Table 110: MainDevice Settings for use of DCs with SubDevice related settings

SubDevice	Description
Slv1	Fixed Input Shift Time (0x1C33:03), no exact synchronization possible This device has the maximum value of 0x1C32:03 plus 0x1C32:06 in this example. Its Sync0 Signal comes with the DC Base signal. The Shift Time is fixed and thus, cannot be used to shift the Input Latch to the system Input Latch time. Synchronization of the Input Latch cannot be achieved in this case.
Slv 2	Sync0 and Shift Times (0x1C32:03, 0x1C33:03) used for synchronization in DC Mode The (Calc and Copy (0x1C32:06) + Output Delay Time (0x1C32:09)) of SubDevice 2 is smaller than the Max value of (0x1C32:06 + 0x1C32:09). To synchronize the Outputs Valid of SubDevice 2 with the system's Outputs Valid the Sync0 cycle of SubDevice 2 is set that it is (0x1C32:06 + 0x1C32:09) away. The Input Shift Time measures from the Sync0 to the Input Latch. The local application should calculate an internal Shift Time from Shift Time minus Input Delay Time so that inputs can be read in physically with the system Input Latch.

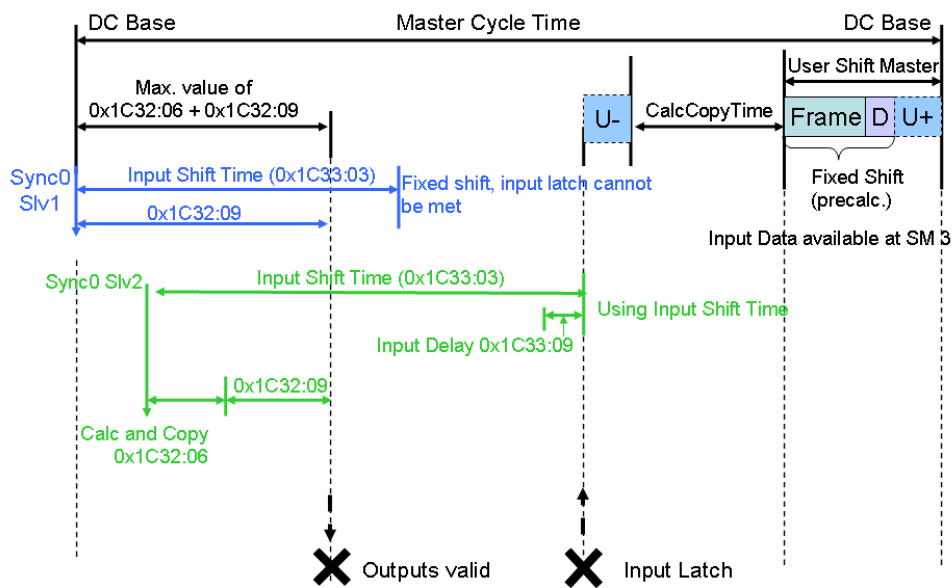


Figure 37: MainDevice Settings

23.2.3 Sync-Unit: DcOffset correction

To prevent that the setting of DcOffset register (0x0920...0x0927) during active SyncOut-Unit (0x0981.0 == 1) will jump over an active Sync-Event the following sequence shall be used:

- deactivate Sync-Unit (0x0981 = 0)
- Set DcOffset (0x0920...0x0927)
- Set StartTimeCycle (0x0990...0x09978) with new value
- activate Sync-Unit (0x0981 = 1)

24 External Synchronization

24.1 External Synchronization of EtherCAT Networks, Reference Clock follows MainDevice Clock

The synchronization of system components or production lines with several EtherCAT Segments can be done with different solutions:

- Synchronization with an external clock using IEEE 1588 protocol
- Synchronization by a bridge device.
- Synchronization by MainDevice Clock

Depending on the application the accuracy of the synchronization has different requirements. For the synchronization of two production lines often the accuracy in the range of milliseconds is enough. However if two or more motion control networks should be synchronized it may be necessary to have the Sync-signals exact at the same time in all devices (e.g. for a network wide commutation information) - that means an accuracy better than a microsecond may be needed.

Within one EtherCAT Segment the Distributed Clocks (DC) mechanism can be used, to synchronize the network devices. DC offers accuracy much better than a microsecond. The first device that should be synchronized is used as the reference clock for that segment. The System Time information of this device is distributed cyclically to the other devices to compensate dynamic drift. The static drift and the propagation delay are compensated during startup of the segment.

The synchronization of several EtherCAT Segments with a high accuracy means that the DC reference clocks in the segments must be adjusted. It can be an external Time information, e.g. GPS or DCF77 receiver (see Figure 38), but it can also be one of the DC reference clocks.

The DC Reference Clocks of the EtherCAT segments need to be adjusted by the EtherCAT MainDevice according to the time information coming from the external clock. The EtherCAT MainDevice should cyclically adjust the DC Reference Clock with a unicast write command to the System Time register. The DC following clocks should be adjusted by a following read-multiple-write command (xRMW).

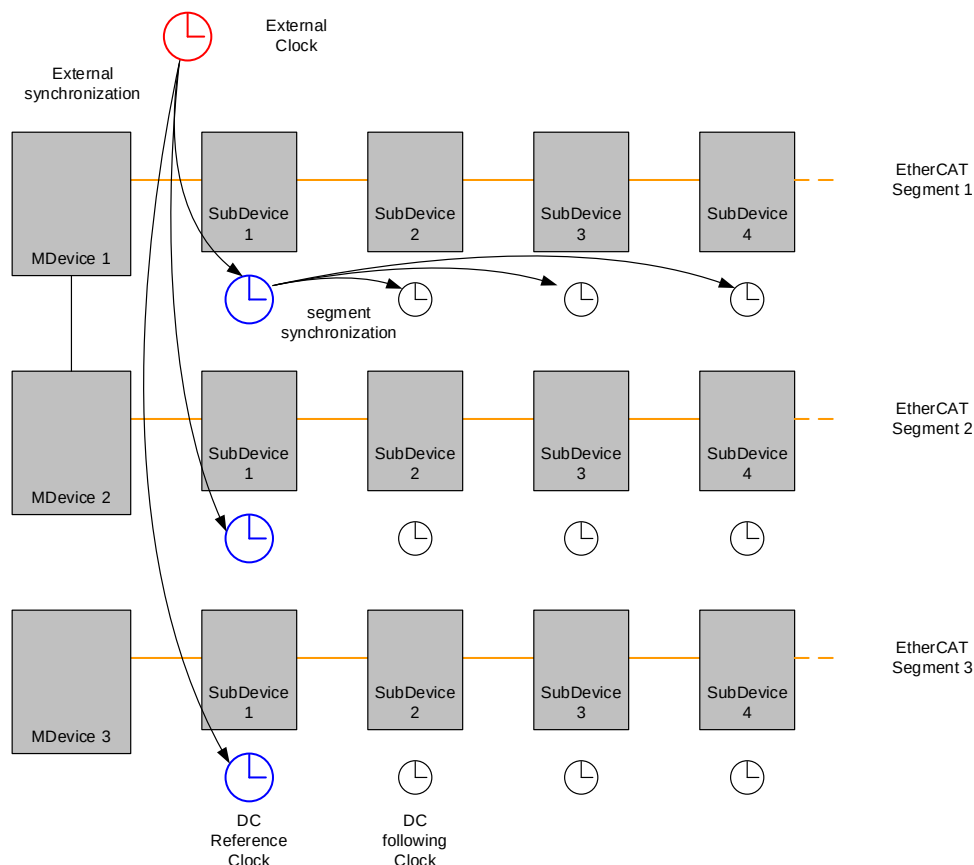


Figure 38: Synchronization of several EtherCAT Segments

24.1.1.1 Synchronization with an external clock using IEEE 1588

The clock is an external clock that supports IEEE 1588 protocol [IEC61588 Precision Clock Synchronization Protocol for Networked Measurement and Control Systems (IEEE Standard)]. This protocol defines mechanisms to synchronize devices in a Standard Ethernet Network.

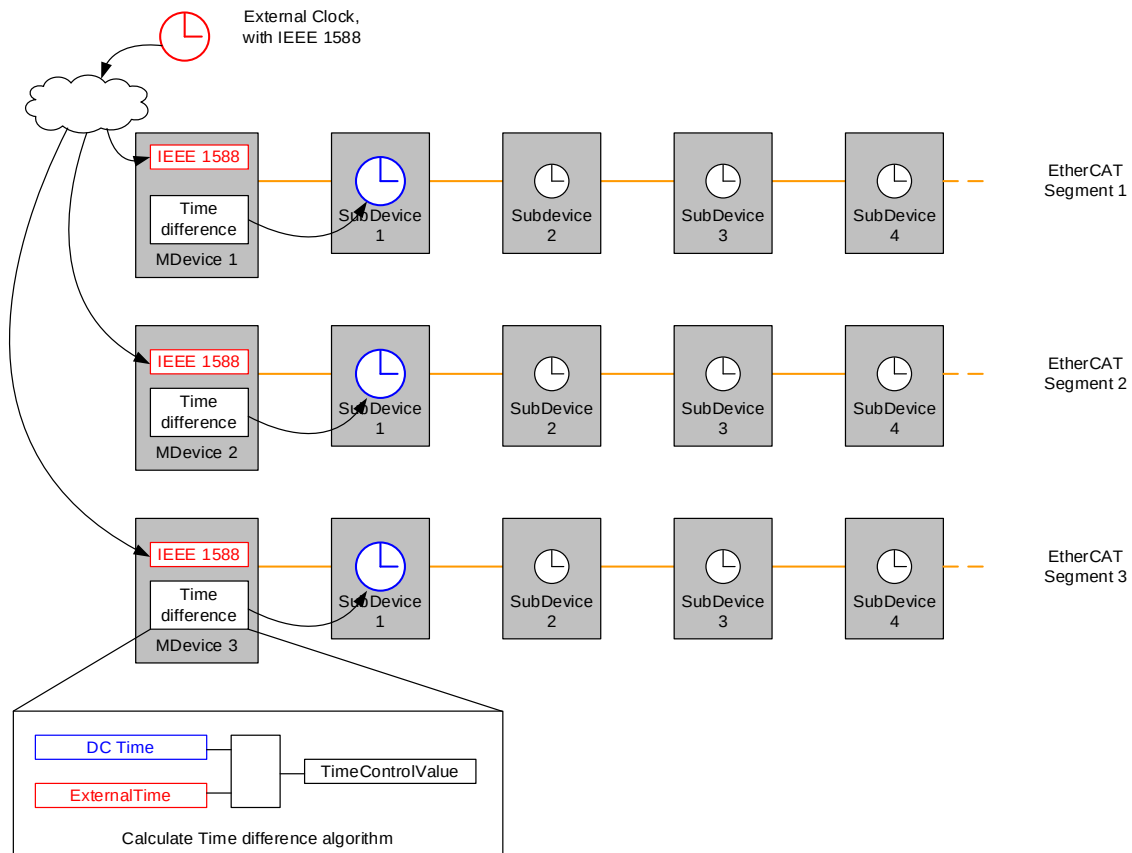


Figure 39: External IEEE 1588 Clock

The control algorithm for the calculation of the time difference between the DC reference clock within an EtherCAT Segment and the IEEE1588 clock can be located in the MainDevice (Figure 39) or it can be located in one of the SubDevices (Figure 40) - if the SubDevice offers this algorithm, the MainDevice can be kept very simple.

In Figure 39 each MainDevice is connected to the external Clock. The MainDevice can calculate the time difference of the 1588-clock to the DC reference clock. The calculated TimeControlValue can be used to adjust the DC Clocks. The accuracy depends on the implementation of the 1588 protocol in the MainDevice - when using a standard Ethernet Port in the MainDevice there is no hardware support to capture the timestamps of the incoming telegrams. Therefore the accuracy is not that high.

Figure 40 shows an example where the control algorithm is located in one of the EtherCAT SubDevices. This offers the advantage that a special hardware support can be used in the SubDevice and the MainDevice can be kept simple. The 1588 telegrams are latched at a legacy Ethernet port with a high precise timestamp in the SubDevice. This external Timestamp can be compared to the high precise DC System Time (1). The result is a TimeControlValue that holds the information if the DC Time runs to fast or to slow. This TimeControlValue can send via PDO to the MainDevice (2) that can adjust the DC Reference Clock in the segment (3).

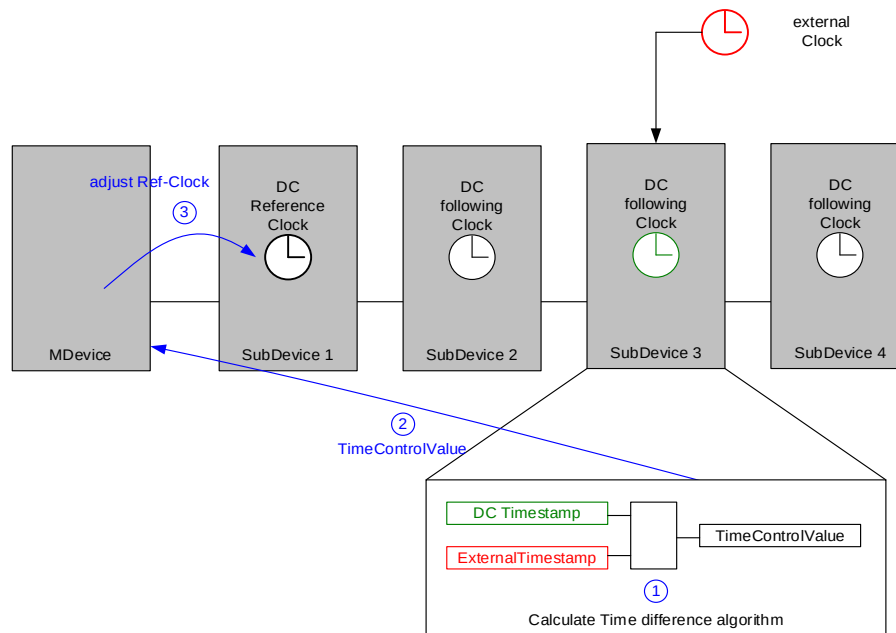


Figure 40: Control Algorithm for Time difference loaded in an EtherCAT SubDevice

During startup the absolute time information of the external Clock must be adjusted to the absolute time of the DC Reference Clock: First the EtherCAT Segment is started and boot-up and the DC Time is set according to the Time information of the MainDevice. The Time difference of this segment time to the 1588 time can be very huge so that the adjustment with the DC control algorithm is not sufficient. The setting of the system time offset register would cause a discontinuity in the SubDevices. Thus an additional system time offset is used within the SubDevice device.

24.1.2 Synchronization by a Bridge device

The synchronization of two or more EtherCAT segments can be done by a Bridge device as shown in Figure 41.

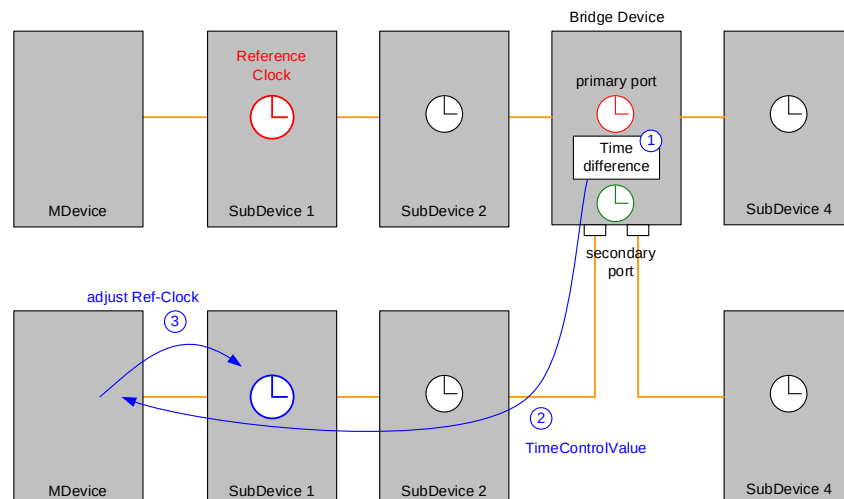


Figure 41: Synchronization of EtherCAT Segments using a Bridge device

The Bridge has two EtherCAT connections (internal two EtherCAT SubDevice Controllers are used). The primary port is connected to the first segment; the secondary port is connected to the second segment.

One of the two ports can be configured to be the leading time reference (in Figure 41 this is for example the primary port). The Bridge can calculate the time difference and offers the TimeControlValue to the MainDevice. The MainDevice can then adjust the Reference Clock.

During startup the two segments can be powered-on at different times. That means that there will be an absolute time difference between the two segments. This difference has to be taken into account with an additional system time offset as part of the calculation of the time difference algorithm.

24.2 External Synchronization of an EtherCAT SubDevice

This clause describes the unique handling for an external synchronization in an EtherCAT SubDevice device.

24.2.1 Object 0x10F4: External Synchronization Status

The External Synchronization status 0x10F4 shown in Table 111 and Table 112 contains the status to synchronize an EtherCAT segment with an external network.

Table 111: 0x10F4 External Synchronization Status

Attribute	Value
Index	0x10F4
Name	External synchronization status
Object Code	RECORD
Max SubIndex	19

Table 112: Entries of 0x10F4 External synchronization Status

SubIndex	Description	Data Type	Access	PDO Mapping	Description / Default value
1	Sync Mode	BIT2	R	(TxPDO)	0: no synchronization, 1: device operates as SYNC Reference, 2: device operates as SYNC follower
3		0, Bit-Length = 12	R	no	Align
14	Control Value Update Toggle	BOOLEAN	R	(TxPDO)	toggles every time when the control value was updated
15	Time Stamp Update Toggle	BOOLEAN	R	(TxPDO)	toggles every time when the time stamps were updated
16	External device not connected	BOOLEAN	R	(TxPDO)	TRUE: no external synchronization found
17	Internal Time Stamp	UNSIGNED64	R	(TxPDO)	only for SYNC follower. DC time stamp at the same time as the external time stamp
18	External Time Stamp	UNSIGNED64	R	(TxPDO)	only for SYNC follower. external time stamp recalculated in DC units (ns)
19	Time Control Value	INT32	R	(TxPDO)	only for SYNC follower. > 0: this value gives the information how often the System Time register of the DC Reference Clock has to be written with a higher value than the actual value of the system Time register for the dynamic drift compensation < 0: this value gives the information how often the System Time register of the DC Reference Clock has to be written with a lower value than the actual value of the system Time register for the dynamic drift compensation

24.2.2 Object 0x10F5: External Synchronization Settings

The External Synchronization status 0x10F5 shown in Table 113 and Table 114 contains the settings to synchronize an EtherCAT segment with an external network.

Table 113: 0x10F5 External Synchronization Settings

Attribute	Value
Index	0x10F5
Name	External synchronization settings
Object Code	RECORD
Max SubIndex	18

Table 114: Entries of 0x10F5 External Synchronization Settings

SubIndex	Description	Data Type	Access	PDO Mapping	Description / Default value
1	Sync Reference	BOOLEAN	RW	no	TRUE: device acts as SYNC Reference, FALSE: device acts as SYNC follower
2	32Bit Time Stamps	BOOLEAN	RW	no	TRUE: only 32 bit time stamps are used, FALSE: 64 bit time stamps are used
3		0, Bit-Length=14		no	align
17	Time Control Interval	UNSIGNED16	RW	no	only for SYNC follower: this value defines how often the time stamps will be updated in ms
18	System Time Additional Offset	UNSIGNED64	rw	no	only for SYNC follower: additional offset to the System Time (needed if the external synchronization was disconnected and the System Time differs too much because the System Time Offset of the ESC cannot be changed if the SubDevice is not in INIT)

24.2.3 TxPDO Mapping

Depending on the External synchronization settings the TxPDO differs.

24.2.3.1 SYNC follower, 64 Bit Time Stamp

Table 115 shows the TxPDO mapping for a SYNC follower with 64 bit time stamp.

Table 115: TxPDO Mapping for SYNC follower, 64 bit Time Stamp

SubIndex	Description	Data Type	Value
1	Sync Mode	UNSIGNED32	0x10F40102
2	Align	UNSIGNED32	0x0000000C
3	Control Value Update Toggle	UNSIGNED32	0x10F40E01
4	Time Stamp Update Toggle	UNSIGNED32	0x10F40F01
5	External device not connected	UNSIGNED32	0x10F41001
6	Internal Time Stamp	UNSIGNED32	0x10F41140
7	External Time Stamp	UNSIGNED32	0x10F41240
8	Time Control Value	UNSIGNED32	0x10F41310

24.2.3.2 SYNC follower, 32 Bit Time stamp

Table 116 shows the TxPDO mapping for a SYNC follower with 32 bit time stamp.

Table 116: TxPDO Mapping for SYNC follower, 32 bit Time Stamp

SubIndex	Description	Data Type	Value
1	Sync Mode	UNSIGNED32	0x10F40102
2	Align	UNSIGNED32	0x0000000E
3	Control Value Update Toggle	UNSIGNED32	0x10F40E01
4	Time Stamp Update Toggle	UNSIGNED32	0x10F40F01
5	Time Stamp invalid	UNSIGNED32	0x10F41001
6	Internal Time Stamp	UNSIGNED32	0x10F41120
7	External Time Stamp	UNSIGNED32	0x10F41220
8	Time Control Value	UNSIGNED32	0x10F41310

24.2.3.3 SYNC follower, only Status

Table 117 shows the TxPDO mapping for a SYNC follower with only status.

Table 117: TxPDO Mapping for SYNC follower, Status Only

SubIndex	Description	Data Type	Value
1	Sync Mode	UNSIGNED32	0x10F40102
2	Align	UNSIGNED32	0x0000000E
3	Control Value Update Toggle	UNSIGNED32	0x10F40E01
4	Time Stamp Update Toggle	UNSIGNED32	0x10F40F01
5	Time Stamp invalid	UNSIGNED32	0x10F41001

24.2.3.4 SYNC Reference

Table 118 shows the TxPDO mapping for a SYNC Reference.

Table 118: TxPDO Mapping for SYNC Reference

SubIndex	Description	Data Type	Value
1	Sync Mode	UNSIGNED32	0x10F40102
2	Align	UNSIGNED32	0x0000000E
3	External device not connected	UNSIGNED32	0x10F41001

24.2.4 MainDevice's Control Loop

The MainDevice has to adapt the System Time of the DC Reference clock by writing the System Time register with a higher or lower value than the actual value. This has to be done to start the dynamic drift compensation on the EtherCAT SubDevice Controller.

There are two possibilities for the MainDevice's control loop to calculate how often the system Time register of the DC Reference clock has to be written to compensate the dynamic drift:

1. Compare the Internal and External Time Stamp which are latched at the same point of time and calculate the number of writing by the MainDevice's own algorithm.
2. Use the Time Control Value generated by the device's algorithm.

With each DC start-up procedure the Control loop Parameter 1 (Address 0x930...0x931) and Control Loop Parameter 3 (Address 0x934...0x935) of all DC capable nodes should be set to following values:

- 0x0934=4
- 0x0935=12
- 0x0930=0x1000 (shall be the last command to reset the DC control unit)

When synchronizing the EtherCAT Network by adjusting the DC reference Clock the following DC Control Loop Parameter in the reference Clock can be used to minimize synchronization jitter:

- 0x0935: Speed Counter Filter = 0
- 0x930=0x1000 (shall be the last command to reset the DC control unit)

NOTE: There are ESCs that only support Control Loop Parameter 1 (Address 0x930...0x931) value of 0x1000.

For compensation of the static deviations of the clock speeds, the MainDevice should send at least 15,000 ARMW/FRMW commands to compensate static drift properly.

25 ESM behaviour

25.1.1 Start of process data exchange

The MainDevice should transmit process data in state transition PREOP-SAFEOP.

The MainDevice shall transmit process data as soon as possible after the SubDevice has confirmed SAFEOP state.

NOTE: The MainDevice may not wait until SubDevices have started their Input Update because in some operation modes the SubDevices may require process data transmission from MainDevice to start Input Update.

The SubDevice should provide process data in state transition PREOP-SAFEOP.

The SubDevice shall update input data as soon as possible after entering state SAFEOP. The SubDevice shall not go to OP state until the inputs are updated in SAFEOP state.

25.1.2 Details on SubDevice ESM handler in MainDevice

With the ESM transition of the MainDevice to INIT it should request all SubDevices to change their state to INIT. This can be done by a Broadcast Write command to register 0x0120 = 0x01.

26 Base Data Types

Base Data Types are defined in ETG.1000.6, [5]. This chapter defines additional data types and enhanced descriptions of the existing data types.

Table 119 describes the Base Data Types.

Table 119: Base Data Types

Index	Name	Base Data Type	Bit Size	Description	Range	GUID
0x0001	BOOLEAN	BOOL BIT	1	'0': FALSE '1': TRUE		{18071995-0000-0000-0000-000000000030}
0x001E	BYTE	BYTE	8	Unsigned Byte, Octet		{18071995-0000-0000-0000-000000000001}
0x001F	WORD	WORD	16	Two Octet		{18071995-0000-0000-0000-000000000004}
0x0020	DWORD	DWORD	32	Four Octet		{18071995-0000-0000-0000-000000000007}
0x000C	TIME_OF_DAY	--	48	See [13]		
0x000D	TIME_DIFFERENCE	--	48	See [13]		
0x000F	DOMAIN	--		Application specific, see [13]		
		<u>Bit String</u>				
0x0030	BIT1	BIT1	1	Can be used as UNSIGNED1		{18071995-0000-0000-0000-000000000010}
0x0031	BIT2	BIT2	2	Can be used as UNSIGNED2		{18071995-0000-0000-0000-000000000011}
0x0032	BIT3	BIT3	3	Can be used as UNSIGNED3		{18071995-0000-0000-0000-000000000012}
0x0033	BIT4	BIT4	4	Can be used as UNSIGNED4		{18071995-0000-0000-0000-000000000013}
0x0034	BIT5	BIT5	5	Can be used as UNSIGNED5		{18071995-0000-0000-0000-000000000014}
0x0035	BIT6	BIT6	6	Can be used as UNSIGNED6		{18071995-0000-0000-0000-000000000015}
0x0036	BIT7	BIT7	7	Can be used as UNSIGNED7		{18071995-0000-0000-0000-000000000016}
0x0037	BIT8	BIT8	8	Can be used as UNSIGNED8		{18071995-0000-0000-0000-000000000017}
0x0038	BIT9	BIT9	9	Can be used as UNSIGNED9		{18071995-0000-0000-0000-000000000018}
0x0039	BIT10	BIT10	10	Can be used as UNSIGNED10		{18071995-0000-0000-0000-000000000019}
0x003A	BIT11	BIT11	11	Can be used as UNSIGNED11		{18071995-0000-0000-0000-000000000020}
0x003B	BIT12	BIT12	12	Can be used as UNSIGNED12		{18071995-0000-0000-0000-000000000021}
0x003C	BIT13	BIT13	13	Can be used as UNSIGNED13		{18071995-0000-0000-0000-000000000022}

Index	Name	Base Data Type	Bit Size	Description	Range	GUID
0x003D	BIT14	BIT14	14	Can be used as UNSIGNED14		{18071995-0000-0000-0000-000000000023}
0x003E	BIT15	BIT15	15	Can be used as UNSIGNED15		{18071995-0000-0000-0000-000000000024}
0x003F	BIT16	BIT16	16	Can be used as UNSIGNED16		{18071995-0000-0000-0000-000000000025}
0x002D	BITARR8	BITARR8	8	8 individual Bits		{18071995-0000-0000-0000-000000000032}
0x002E	BITARR16	BITARR16	16	16 individual Bits		{18071995-0000-0000-0000-000000000033}
0x002F	BITARR32	BITARR32	32	32 individual Bits		{18071995-0000-0000-0000-000000000034}
		<u>Signed Integer</u>				
0x0002	INTEGER8	SINT	8	Short Integer	-128 to 127	{18071995-0000-0000-0000-000000000003}
0x0003	INTEGER16	INT	16	Integer	-32 768 to 32 767	{18071995-0000-0000-0000-000000000006}
0x0010	INTEGER24	INT24	24		-2^{23} to 2^{23-1}	{18071995-0000-0000-0000-000000000023}
0x0004	INTEGER32	DINT	32	Double Integer	-2^{31} to 2^{31-1}	{18071995-0000-0000-0000-000000000009}
0x0012	INTEGER40	INT40	40			{18071995-0000-0000-0000-000000000027}
0x0013	INTEGER48	INT48	48			{18071995-0000-0000-0000-000000000029}
0x0014	INTEGER56	INT56	56			{18071995-0000-0000-0000-00000000002B}
0x0015	INTEGER64	LINT	64	Long Integer	-2^{63} to 2^{63-1}	{18071995-0000-0000-0000-00000000000C}
		<u>Unsigned Integer</u>				
0x0005	UNSIGNED8	USINT	8	Unsigned Short Integer	0 to 255	{18071995-0000-0000-0000-000000000002}
0x0006	UNSIGNED16	UINT	16	Unsigned Integer / Word	0 to 65 535	{18071995-0000-0000-0000-000000000005}
0x0016	UNSIGNED24	UINT24	24			{18071995-0000-0000-0000-000000000022}
0x0007	UNSIGNED32	UDINT	32	Unsigned Double Integer	0 to 2^{32-1}	{18071995-0000-0000-0000-000000000008}
0x0018	UNSIGNED40	UINT40	40			{18071995-0000-0000-0000-000000000026}
0x0019	UNSIGNED48	UINT48	48			{18071995-0000-0000-0000-000000000028}
0x001A	UNSIGNED56	UINT56	56			{18071995-0000-0000-0000-00000000002A}
0x001B	UNSIGNED64	ULINT	64	Unsigned Long Integer	0 to 2^{64-1}	{18071995-0000-0000-0000-00000000000B}

Index	Name	Base Data Type	Bit Size	Description	Range	GUID
		Floating				
0x0008	REAL32	REAL	32	Floating point		{18071995-0000-0000-0000-00000000000D}
0x0011	REAL64	LREAL	64	Long float		{18071995-0000-0000-0000-00000000000E}
		GUID				
0x001D	GUID	GUID	128	Globally unique identifier with a length of 128 Bit		{18071995-0000-0000-0000-000000000021}

Table 120 describes the Base Data Types with variable length.

Table 120: Base Data Types with variable length

Index	Name	Base Data Type	Bit Size	Description	Range	GUID
		Strings				
0x0009	VISIBLE_STRING	STRING(n)	8*(n)	VisibleString (1 octet per character)		{18071995-0000-0000-0000-00010000xxxx} with xxxx equals n (in HEX)
0x0268	UNICODE_STRING	WSTRING(n)	16*(n)	Word String (2 octets per character,)		"{18071995-0000-0000-0000-00020000xxxx} with xxxx equals n+1 (in HEX)"
		Octet field				
0x000A	OCTET_STRING	ARRAY [0..n] OF BYTE	8*(n+1)	Sequence of octets (data type BYTE)		{18071995-0000-0000-0000-00030000xxxx} with xxxx equals n+1 (in HEX)
0x000B	ARRAY_OF_UINT	ARRAY [0..n] OF UINT	16*(n+1)	Sequence of UINT		{18071995-0000-0000-0000-000B0000xxxx} with xxxx equals n+1 (in HEX)
0x0260	ARRAY_OF_INT	ARRAY [0..n] OF INT	16*(n+1)	Sequence of INT		{18071995-0000-0000-0000-000A0000xxxx} with xxxx equals n+1 (in HEX)
0x0261	ARRAY_OF_SINT	ARRAY [0..n] OF SINT	8*(n+1)	Sequence of SINT		{18071995-0000-0000-0000-00080000xxxx} with xxxx equals n+1 (in HEX)
0x0262	ARRAY_OF_DINT	ARRAY [0..n] OF DINT	32*(n+1)	Sequence of DINT		{18071995-0000-0000-0000-000C0000xxxx} with xxxx equals n+1 (in HEX)
0x0263	ARRAY_OF_UDINT	ARRAY [0..n] OF UDINT	32*(n+1)	Sequence of UDINT		{18071995-0000-0000-0000-000D0000xxxx} with xxxx equals n+1 (in HEX)
0x0264	ARRAY_OF_BITARR8	ARRAY [0..n] OF BITARR8	8*(n+1)	Sequence of BITARR8		{18071995-0000-0000-0000-00230000xxxx} with xxxx equals n+1 (in HEX)

Index	Name	Base Data Type	Bit Size	Description	Range	GUID
0x0265	ARRAY_OF_BITARR16	ARRAY [0..n] OF BITARR16	16*(n+1)	Sequence of BITARR16		{18071995-0000-0000-0000-00240000xxxx} with xxxx equals n+1 (in HEX)
0x0266	ARRAY_OF_BITARR32	ARRAY [0..n] OF BITARR32	32*(n+1)	Sequence of BITARR32		{18071995-0000-0000-0000-00250000xxxx} with xxxx equals n+1 (in HEX)
0x0267	ARRAY_OF_USINT	ARRAY [0..n] OF USINT	8*(n+1)	Sequence of USINT		"{18071995-0000-0000-0000-00090000xxxx} with xxxx equals n+1 (in HEX)"
0x0269	ARRAY_OF_REAL	ARRAY [0..n] OF REAL	32*(n+1)	Sequence of REAL		{18071995-0000-0000-0000-00170000xxxx} with xxxx equals n+1 (in HEX)
0x026A	ARRAY_OF_LREAL	ARRAY [0..n] OF LREAL	64*(n+1)	Sequence of LREAL		{18071995-0000-0000-0000-00180000xxxx} with xxxx equals n+1 (in HEX)

Table 121 describes pre-defined records.

Table 121: Pre-defined records

Index	Name	Base Data Type	Bit Size	Description	Range	GUID
0x0021	PDO_MAPPING	--		For PDO_MAPPING definition see ETG.1000		{7FF678B5-BA48-42E6-9A0A-CBFCBECC8412}
0x0022	<i>Reserved</i>			<i>Reserved for compatibility reasons</i>		
0x0023	IDENTITY	--		For IDENTITY definition see ETG.1000		{7E3EE6E2-BBAB-4468-9F5A-68E5EBBF4412}
0x0024	<i>Reserved</i>			<i>Reserved for compatibility reasons</i>		
0x0025	COMMAND_PAR	--		For COMMAND_PAR definition see ETG.1000		{0F324177-734C-4D75-89B0-468BB8E8F222}
0x0026	<i>Reserved</i>			<i>Reserved for compatibility reasons</i>		
0x0027	PDO_PARAMETER	--		For PDO_PARAMETER definition see ETG.1020		
0x0028	ENUM	--		For DEFTYPE_ENUM definition see ETG.1020		{18071995-0000-0000-0000-00000000001e}
0x0029	SM_SYNCHRONISATION	--		For SYNC_PAR definition see ETG.1000		{36BEED8B-6733-4853-A49C-3E5EA40DF14E}
0x002A	RECORD			No pre-defined Record structure. Can be used as a general data type for Records.		{FB01EF56-963F-48E7-9B0A-09691C15038B}

Index	Name	Base Data Type	Bit Size	Description	Range	GUID
0x002B	BACKUP_PARAMETER	--		For BACKUP_PARAMETER definition see ETG.1020, Object 0x10F0		{D354C782-6ECB-492E-BAAD-96BA69834B4E}
0x002C	MODULAR_DEVICE_PROFILE	--		For MODULAR_DEVICE_PROFILE definition see ETG.5001, Object 0xF000		{BE8233A1-BFBD-4A98-9220-B58C21910B27}
0x0281	ERROR_SETTING	--		For DEFTYPE_ERRORHANDLING definition see ETG.1020, Object 0x10F1		{A7AE60A4-C53D-4802-B5AB-E3A1BD83A6F1}
0x0282	DIAGNOSIS_HISTORY	--		For DEFTYPE_DIAGHISTORY definition see ETG.1020, Object 0x10F3		{DF7DD567-FF9E-4CAB-BD30-0DA54F9898D0}
0x0283	EXTERNAL_SYNC_STATUS	--		For DEFTYPE_SYNCSTATUS definition see ETG.1020, Object 0x10F4		{1265D0D5-7626-4469-A2F4-67993EB72708}
0x0284	EXTERNAL_SYNC_SETTINGS	--		For DEFTYPE_SYNCSETTINGS definition see ETG.1020, Object 0x10F5		{9E0628D3-D369-41C9-AA18-0556FCA2D9D6}
0x0285	DEFTYPE_FSOEFRAME	--		For DEFTYPE_FSOEFRAME definition see ETG.5120		{CD5B2900-7A5E-41DC-972A-85B642FDCCA1}
0x0286	DEFTYPE_FSOECOMMPAR	--		For DEFTYPE_FSOECOMMPAR definition see ETG.5120		{2DB92E24-DA89-482F-85E4-3AFB6B534F3D}
0x0287 – 0x07FF	Reserved			Reserved for future use		